

测量程序设计

C#--WPF

朱学军 编著

xazhuxj@qq.com

西安科技大学

测绘科学与技术学院

二零二六年八月

本文档仅适用于西安科技大学测绘相关专业本科二年级以上学生教学使用。

也可用于其他专业人员参考。

内部文档，版权所有 © 2026，朱学军。

未经作者本人授权，严谨出版发行！

教学计划及课程目标

教学计划

教学课时

该课程标准学时为 32+8 学时。总学时为 48 学时，上课 16 次 32 学时，上机 4 次 8 学时。必修课，考试。笔试（50%）+ 上机实验与作业（50%）

学习内容

学习面向对象的程序设计方法；
学习基本的测量算法设计方法，学习软件界面的编写与设计。

编程语言的选择

注意：该课程不是再次学习某种编程语言，而是运用某种编程语言进行测量数据处理及测量算法的编写。

可能大家只学习了 C 语言，C 语言是面向过程的功能强大的语言，执行效率高，但界面程序编写较复杂。现代编程语言更多的是面向对象的编程语言，高效而且兼容 C 语言的是 C++，但界面的编写仍较复杂。因此我们选择更加现代化的编程语言 C#，它语法优美，界面编写方式有 WinForm 与 WPF 方式，虽然执行效率没有 C++ 高，但在 Windows7、Windows8/8.1、Windows10+ 中都几乎预装了运行库 .Net Framework，并且得到了 Microsoft 的大力推广，Microsoft 的许多软件都在使用 C# 开发，学习与开发成本相比与 C++ 显著减少，开发效率显著提升。

C# 的语法与 JAVA 的许多语法是极其相似的，现代软件工程的许多方法也可以用 C# 去实现。因此，在本课程中我们将学习 C# 编程语言，并且会学习如何运用 C# 语言进行测量程序设计及测量数据处理。

现代软件基本上都具有简洁易用的界面，我们还将学习 WPF 的界面编写技术，为我们的程序设计处美观简洁的界面，这些都包含在 C# 之中。

编程环境与工具

传统的 .Net Framework 向 .Net Core 发展，最后统一到 .Net 中，将各种技术整合，目前 .Net6、.Net 8 正在迅速普及。目前 C# 已经更新到 8.0、9.0、10.0，且 11 也在预览之中。

为了探究新的技术，本课程的基本编程工具定为 Visual Studio，版本理论上为 2017 及更新版，推荐大家用 Visual Studio 2019 或 2022 社区版（Community）。

参考教材

C# 语言及 WPF 界面编写参考马骏主编，人民邮电出版社出版，《**C#** 程序设计及应用教程》第 3 版。

测量算法参考本教程（一直更新中.....）。

目录

教学计划及课程目标	i
第 1 章 C# 开发环境与程序基础	1
1.1 C# 开发环境与工具	1
1.1.1 C# 开发环境	1
1.1.2 版本控制与源代码管理	2
1.2 第一个 C# 程序	2
1.2.1 单个 C# 源代码文件的编写	2
1.2.2 单个 C# 源代码文件的编译与执行	2
1.2.3 使用 IDE 编辑、编译与运行 C# 程序	3
1.2.4 Visual Studio 文件组织形式分析	3
1.2.5 顶级语句	4
1.3 C# 程序结构分析	5
1.4 扩展：从 C 走向 C#	6
1.4.1 Client/Server 程序设计模式	6
1.4.2 从面向过程走向面向对象的程序设计	7
1.4.3 与 C 相对应的 C# 代码	9
1.4.4 面向对象的 C# 代码	11
1.5 作业	16
第 2 章 C# 基础语法	17
2.1 C# 中的变量与数据类型	17
2.1.1 值类型	17
2.1.2 引用类型	18
2.1.3 自定义类型	19
2.1.4 装箱与拆箱	19
2.1.5 延伸阅读值类型	20
2.1.6 变量的定义与 var 关键字	20
2.1.7 变量的定义位置	20
2.1.8 常量与只读变量	20
2.2 表达式与语句	21

2.2.1	表达式	21
2.2.2	语句	21
2.3	C# 中字符与字符串	25
2.3.1	字符 (char)	25
2.3.2	字符串 (string)	25
2.3.3	字符串的格式化	26
2.3.4	字符串与数值类型之间的转换	27
2.3.5	string 的性能与 StringBuilder	27
2.4	C# 特殊语法	27
2.4.1	元组 (tuple)	27
2.4.2	记录 (record)	28
2.4.3	泛型	29
2.4.4	集合	30
2.4.5	委托与事件	32
2.4.6	Lambda 语句	33
2.4.7	迭代器	33
2.5	模式匹配 (Pattern Matching)	34
2.5.1	Null 检查 (is 表达式)	34
2.5.2	类型测试 (is 表达式)	35
2.5.3	比较离散值 (switch 表达式)	35
2.5.4	关系模式	36
2.5.5	多个输入	36
2.5.6	列表模式	37
2.6	小结	38
2.7	上机实验	38
第 3 章	C# 面向对象基础	39
3.1	类 (class)	39
3.1.1	类与实例对象 (instance object)	39
3.1.2	类的实例与静态类、静态属性 (static)	39
3.1.3	类的定义	39
3.1.4	类的实例对象	40
3.1.5	类的字段 (Filed)	40
3.1.6	类字段的可访问性	41
3.1.7	类的构造函数	42
3.1.8	类的属性 (property)	44
3.1.9	类的实例方法 (method)	47
3.1.10	类的静态成员	49
3.1.11	类的实例成员与静态成员的关系	50
3.1.12	类成员扩展阅读	54

3.2 静态类与静态构造函数	55
3.2.1 静态类	55
3.2.2 静态构造函数	56
3.3 用静态类与静态函数建立测量基础函数库	56
3.3.1 C# 知识点	57
3.3.2 测绘常用算法设计	57
第 4 章 C# 面向对象程序设计	67
4.1 封装	67
4.2 继承 (Inheritance)	67
4.2.1 使用继承的目的	68
4.2.2 C# 类继承	68
4.2.3 继承的写法	68
4.2.4 子类继承基类的成员	69
4.2.5 派生类对基类方法的隐藏: 使用关键字 new	69
4.2.6 继承中的构造函数	70
4.2.7 小结	72
4.3 多态与虚函数	72
4.4 抽象方法与抽象类	82
4.5 接口 (interface)	83
4.5.1 面向接口编程	85
4.5.2 接口跳转	86
4.6 封闭类 (sealed class)	86
4.7 扩展阅读: 面向对象程序设计原则	87
4.7.1 开闭原则 (OCP)	88
4.7.2 里氏替换原则 (Liskov Substitution Principle , LSP)	89
4.7.3 迪米特原则 (最少知道原则) (Law of Demeter , LoD)	89
4.7.4 单一职责原则	90
4.7.5 接口分隔原则 (Interface Segregation Principle , ISP)	90
4.7.6 依赖倒置原则 (Dependency Inversion Principle , DIP)	91
4.7.7 组合/聚合复用原则 (Composite/Aggregate Reuse Principle , CARP)	91
4.8 小结	92
第 5 章 WPF 界面编写基础	93
5.1 WPF 设计原则	93
5.1.1 概述	93
5.1.2 WPF 核心设计原则	94
5.2 XAML 基本语法	94
5.2.1 WPF 项目结构	94
5.2.2 项目中的 xaml 文件及含义	95

5.3	WPF 常用控件	96
5.4	坐标方位角计算图形界面程序编写	96
5.4.1	建立解决方案与项目	96
5.4.2	界面编写	99
5.4.3	界面程序的优化	102
第 6 章	通用高斯投影程序设计	107
6.1	高斯投影的数学模型	107
6.1.1	椭球基础	107
6.1.2	高斯投影正算	109
6.2	程序功能分析与设计	110
6.2.1	高斯投影的主要内容	110
6.2.2	参考椭球类的设计	111
6.2.3	高斯投影正算功能的实现	114
6.2.4	高斯投影反算功能的实现	116
6.2.5	换带计算	118
6.3	图形界面程序编写	119
6.3.1	单点高斯投影正反算图形界面编写	119
6.3.2	界面程序的优化	121
6.3.3	点类的进一步优化	125
6.4	更加实用的多点计算图形程序	126
6.4.1	程序的功能	127
6.4.2	程序的面向对象分析与实现	127
6.4.3	多点的高斯投影计算	131
6.4.4	点坐标数据的读入与写出	132
6.4.5	界面设计与实现	136
6.4.6	换带功能的实现	141
6.5	扩展	141
6.5.1	UTM 投影	141
6.5.2	注意事项	143
6.5.3	空间直角坐标系与大地坐标之间的转换	143
6.5.4	同基准下的椭球膨胀法	144
第 7 章	线路要素计算程序设计	147
7.1	圆曲线程序设计	147
7.1.1	圆曲线的数学模型与算法分析	147
7.1.2	圆曲线上点的坐标计算算法分析	147
7.1.3	圆曲线坐标计算中的类设计	148
7.1.4	任意桩号点的坐标计算	156
7.1.5	圆曲线算例与测试代码	157

7.1.6	线路上的点按间距批量计算	157
7.1.7	界面的实现	157
7.2	有缓和曲线的圆曲线	158
7.2.1	缓和曲线的数学模型	158
7.2.2	有缓和曲线的圆曲线算例	163
7.3	较为完整的线路里程桩计算	163

第 1 章 C# 开发环境与程序基础

经过大学一年或两年的学习，我们有了 C 语言程序设计基础，有一定的计算机编程的基本知识。C 语言是功能强大的高效程序设计语言，但在现代的软件开发中特别是 Windows 桌面界面程序开发中的效率太低了。为了提高软件开发效率，微软公司在 2000 年之后推出了 .Net Framework，在新版 Windows 中也越来越多的进行了 .Net Framework 预装，.Net Framework 也进化到 .NET 10+，具备了跨平台等多项功能，应用场景日益广泛。

C# 语言是 .NET 平台开发的主流语言，C# 语言面向对象、语法优美、功能强大、开发效率高，是现代软件工程的结晶，其应用也越来越广泛。因此我们将从 C 语言程序设计转向 C# 程序设计语言，学习更多的现代程序设计方法，满足现代测绘大数据的处理与算法编写需求。

1.1 C# 开发环境与工具

1.1.1 C# 开发环境

Microsoft Visual Studio 是微软出品的经典集成开发环境 (Integrated Development Environment, 简称 IDE)，也是 C# 语言开发的经典工具，目前常用的版本为 2017、2019、2022、2026 版，最新版本为 2026 版。为了学习与练习 C# 新的语法，建议使用较新版本的 IDE，本课程将使用 Microsoft Visual Studio Community 2022/2026 进行代码的编写与调试。建议你的 Microsoft Visual Studio 版本不要低于 2017 版。

安装好 Microsoft Visual Studio Community 2022/2026 后，基本上 C# 的开发环境就准备好了。

JetBrains Rider 是另一款非常优秀的 .NET IDE，号称 The world's most loved .NET and game dev IDE，其开发环境与 PyCharm(Python 开发)、Android Studio 等本质上是一样的，使用习惯也高度类似，还可以跨平台使用，非常值得尝试。

Microsoft Visual Code 是微软开源的文本编辑软件，功能比 Windows 操作系统自带的记事本功能强大的多，在程序编写中可以快速查看与修改源代码。也可以配合 dotnet 命令进行 .NET 程序开发。

如果不考虑界面编写，可用使用 LINQPad 进行 C# 练习，LINQPad 是一款非常优秀的 .NET 代码编辑器，支持 C#、VB、F# 等多种语言，可以直接编写、调试与运行 C# 代码，支持 .NET 6/7/8/9/10 等多种版本、多种运行环境，支持 NuGet 包管理器，支持 LINQ 查询语法，支持多种数据库，支持图形化输出，支持 Markdown 文档等，功能非常强大，值得尝试。

1.1.2 版本控制与源代码管理

git 软件是目前最流行的开源分布式版本控制软件，也称为软件业的时光机。在我们的软件编写过程中，将使用它进行源代码管理。

TortoiseGit 是 git 的图形化客户端软件，开源软件，支持中文等语言。

git 软件不但能用于编程时的源代码管理与协作，在其他领域也有诸多应用，建议多练习多使用。

github.com 是全球最大的开源代码托管平台，支持 git 软件进行源代码管理与协作，是软件开发者的宝藏库，非常值得一探究竟。

gitee.com 是国内的开源代码托管平台，本课程的示例代码托管在 gitee.com 上，可以使用命令 `git clone git@gitee.com:zhuxj911/SurApp.git` 下载源码。

1.2 第一个 C# 程序

1.2.1 单个 C# 源代码文件的编写

我们用 C# 语言编写经典的 C 语言入门程序 Hello World。打开 Microsoft Visual Code，首先将文件保存为 HelloWorld.cs 文件，在其中输入以下代码：

```
1 // HelloWorld.cs
2 using System;
3
4 namespace HelloWorld
5 {
6     class Program
7     {
8         static void Main(string[] args)
9         {
10             Console.WriteLine("Hello World!");
11         }
12     }
13 }
```

1.2.2 单个 C# 源代码文件的编译与执行

点击开始菜单中的 Visual Studio 文件夹下的 Developer Command Prompt for VS 或 Developer PowerShell for VS，切换到 HelloWorld.cs 文件所在的文件夹，输入：`csc HelloWorld.cs` 将编译生成 HelloWorld.exe 可执行程序。

再输入：`HelloWorld`，程序执行输出：`Hello World!`

以上命令中 `csc` 为 C# 语言的编译器，作用为检查 C# 源代码文件中的语法，将其翻译成 .NET 能识别的中间语言 (Microsoft Intermediate Language，简称为 IL)，最后在 .NET 的公共语言运行库 (Common Language Runtime，简称为 CLR) 支持下解释为可执行的计算机命令。

整个过程如图1.1所示。

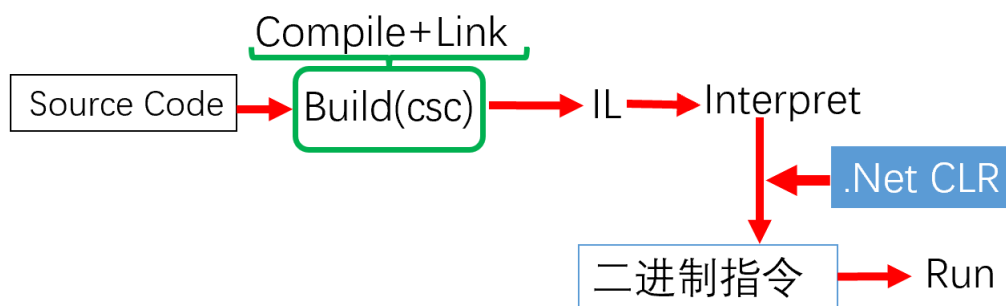


图 1.1 HelloWorld 程序构建过程

1.2.3 使用 IDE 编辑、编译与运行 C# 程序

一个程序除了源代码之外，还有图标、图片、配置信息等等文件，源代码文件也可能有多个，用以上方式编辑、构建可执行文件太复杂了，这时使用 IDE 的效率会更高。

用 Visual Studio 生成一个 HelloWorld 控制台（Console）项目，在 Program.cs 文件中会自动生成以上代码，点击工具栏上的运行按钮，即可编译运行程序。

1.2.4 Visual Studio 文件组织形式分析

在文件资源管理器中打开 HelloWorld 项目，文件组织形式如图 1.2 所示：

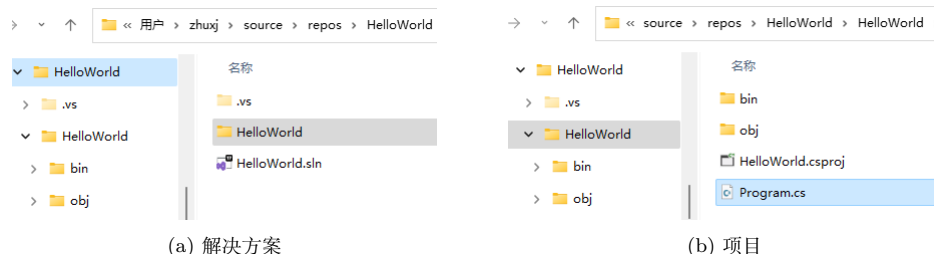


图 1.2 HelloWorld 程序文件结构

图1.2a 为解决方案的文件夹，系统在生成解决方案时，会根据工程名（或解决方案名）自动创建一个文件夹，如本例中的 HelloWorld，并在文件夹中创建一个解决方案的文件 HelloWorld.sln 与工程文件夹 HelloWorld。

文件 HelloWorld.sln 为文本文件，可以用 Visual Code 或 Notepad 打开，其内容如下：

```
1 Microsoft Visual Studio Solution File, Format Version 12.00
2 # Visual Studio Version 17
3 VisualStudioVersion = 17.0.31612.314
4 MinimumVisualStudioVersion = 10.0.40219.1
5 Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "HelloWorld", "HelloWorld\
  HelloWorld.csproj", "{770A121E-217D-435D-AAB7-2EDE7EC5A7C9}"
```

```
6 EndProject
7 Global
8     .....省略.....
9 EndGlobal
```

第 5 行说明了 HelloWorld 项目的相关信息。

图1.2b为项目 HelloWorld 的文件夹，该文件夹中有一个工程文件 HelloWorld.csproj，一个 C# 源代码文件 Program.cs、两个文件夹 bin 与 obj，bin 用于存放最后生成的可执行文件，obj 缓存系统中间文件。

文件 HelloWorld.csproj 内容如下：

```
1 <Project Sdk="Microsoft.NET.Sdk">
2
3     <PropertyGroup>
4         <OutputType>Exe</OutputType>
5         <TargetFramework>net10.0</TargetFramework>
6     </PropertyGroup>
7
8 </Project>
```

这个文件记录了项目的生成类型与运行目标框架。

从以上看出，Visual Studio 是用文件夹与文件的形式来组织管理项目的。一个项目由多个源代码文件、资源文件等组成，经过编译链接最终生成为程序集。程序集通常具有文件扩展名.exe 或.dll，具体取决于它们是实现应用程序 (application) 还是实现库 (library)。

一个或多个项目组成一个解决方案 (Solution)。

一个项目的构建过程如图1.3所示。

1.2.5 顶级语句

使用 Visual Studio 2022/2026 生成.NET 6 或更高版本项目 (C#9.0 版本) 时，如果不勾选“不使用顶级语句”，则只会在 Program.cs 文件中生成一个语句：

```
1 Console.WriteLine("Hello World!");
```

顶级语句 (top-level statements) 相当于去掉了 using、namespace 以及默认类 Program 与默认的 Main 函数，这些由编译器在编译时自动生成。具体请参看 微软顶级语句官方文档。

顶级语句允许直接在文件的顶层编写代码，无需定义类或方法，包含顶级语句的文件被视为程序的入口点，类似于 C# 之前的 Main 方法，编译器会自动生成一个 Main 方法，并将顶级语句作为 Main 方法的主体。尽管不需要定义类，但顶级语句的文件中仍然可以定义局部函数。

顶级语句只能在一个源文件中使用，如果一个项目中有多个使用顶级语句的文件，会导致编译错误。

顶级语句简化了基于控制台应用的程序，使创建简单程序更容易，更加方便探索新的语法与算法。

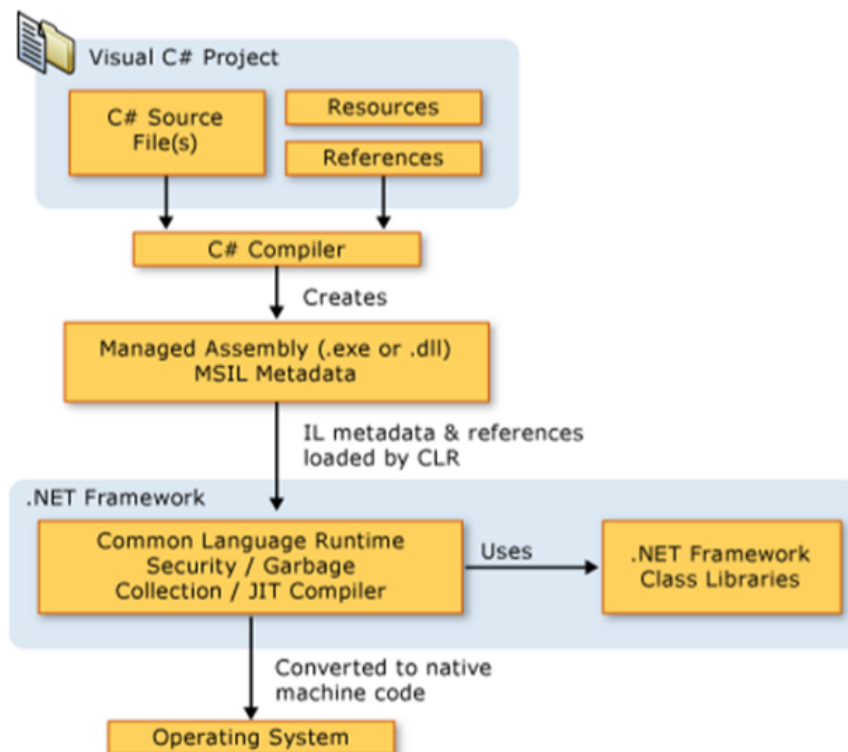


图 1.3 项目构建过程

1.3 C# 程序结构分析

分析以上 C# 示例中的 HelloWorld.cs 源代码可以看出，C# 程序中的一些关键概念：

- using 指令

引入所需的外部命名空间，使用其中的类型，避免在使用这些类时需要写完整的命名空间路径。

- 命名空间 (namespace)

组织与管理源代码，避免名称冲突。

命名空间有两种形式：一种是命名空间块，另一种是文件范围的命名空间。

命名空间块的形式为：

```
1 namespace MyNamespace
2 {
3     // 代码内容
4 }
5
```

文件范围的命名空间的形式为：

```
1 namespace MyNamespace;
```

```
2 // 代码内容
3
```

一般一个文件中只会有一个命名空间，命名空间的名称一般与文件夹的名称相同。

- 类 (class)

C# 代码的载体，包含数据和方法的容器，**C#** 中所有的代码都必须写在类中。

- 类的静态成员方法 (static void Main)

静态成员方法是指不依赖于类的实例而存在的方法，意味着整个类只有唯一的一份，Main 方法是程序的入口点，当程序启动时从这里开始执行。

- Console.WriteLine 语句

Console 代表控制台类，WriteLine 是控制台类的一个方法，作用是在控制台输出一行文本。

C# 程序由一个或多个源代码文件与资源文件组成。每个源代码中声明类，类包含成员，并且可按命名空间进行组织。类的成员有字段 (field)、方法、函数、属性和事件等。

1.4 扩展：从 C 走向 C#

1.4.1 Client/Server 程序设计模式

现代软件往往是多人协作开发的成果，单个人进行大型软件的开发是比较少的。在软件开发中需要遵循软件工程的组织原则，寻求代码的可复用性、可测试性、可阅读性与可维护性。

在学习编程时我们首先应该改掉以下不良编码习惯：

改变如某些 C 语言入门教材中将代码直接写入 main 函数中的习惯；

改变在 **C#** 语言中将代码直接写入 Main 函数中的习惯；

改变在 UI (WinForm 或 WPF 或其它的界面) 中直接写入逻辑算法代码的习惯。

以上三种形式的代码书写处的 Main 函数或界面调用函数我们可以称为 Client，我们编写的逻辑算法代码可以称为 Server。试想如果我们去商场买东西，我们就是 Client，提供需求；商场就是 Server，提供服务。如果我们要买一只圆珠笔，也许我们只需简单的告诉商场人员，然后付钱拿笔走人就行了。但如果商场要让我们自己去仓库里找笔、查价钱、再在商场登记簿上登记库存等等一些动作，你想想会是怎么样的一个结果？买只笔都要把我们累死了。

这说明了什么呢？提供服务功能的商场应该对有功能需求的客户简化和屏蔽各种复杂的中间环节。在软件开发时同样如此，我们的 main 或 Main 函数或 UI 处的代码应该简洁，基本上只是调用各个功能算法而已。

如果我们像以上这三种方式组织代码，将会带来一系列的问题，尤其在团队开发与多人协作时代码不能复用，不能进行 unit test (单元测试)、不能用 git 工具（著名的源代码管理工具）进行源代码的自动合并。软件系统稍微复杂一点，我们的开发就会面临不可控甚至失败的危险。

万丈高楼从地起，因此，在学习编程时首先我们要有 Client/Server 模式意识，要遵循界面与算法相分离的原则进行程序设计。

1.4.2 从面向过程走向面向对象的程序设计

良好的面向过程设计程序设计程序是可以很好的转向面向对象的程序设计的，我们将从一个简单的 C 程序开始设计结构较为良好的 C 代码，再将其用面向对象的 C# 进行实现。从中体会面向过程与面向对象的程序设计方法的不同。

面向过程的 C 代码示例

在测绘专业中我们经常与测量点打交道，因此我们定义一个点 Point（测点除了它的坐标 x, y 和高程 z 之外，还需至少有点名），另外我们定义一个简单的为大家所熟知的圆 Circle，来使用点 Point 定义它的圆心，并实现判断两圆是否相交，计算圆的面积和周长等功能。代码如下所示：

```
1 // Ch01Ex01.cpp : Defines the entry point for the console application.
2 //
3
4 #include "stdafx.h"
5
6 #define _USE_MATH_DEFINES
7 #include <math.h>
8 #include <string.h>
9 #define PI M_PI
10
11 typedef struct _point {
12     char name[11];
13     double x, y, z;
14 }Point;
15
16 typedef struct _circle {
17     Point center;
18     double r;
19     double area;
20     double length;
21 }Circle;
22
23 //计算圆的面积
24 void Area(Circle * c) {
25     c->area = PI * c->r * c->r;
26 }
27
28 //计算两点的距离
29 double Distance(const Point * p1, const Point * p2) {
30     double dx = p2->x - p1->x;
31     double dy = p2->y - p1->y;
32     return sqrt(dx*dx + dy * dy);
33 }
```

```
34
35 //判断两圆是否相交
36 bool IsIntersectWithCircle(const Circle * c1, const Circle * c2) {
37     double d = Distance(&c1->center, &c2->center);
38     return d <= (c1->r + c2->r);
39 }
```

对 C 代码的解释

代码中的 11 和 16 行定义结构体时使用了 typedef，这样在标准 C 中再定义 Point 类型或 Circle 类型时可以避免在其前面加关键词 struct。

第 24-26 行计算圆的面积，传入了圆的指针，这是 C 及 C++ 中传递自定义数据类型的常用的高效方式。由于计算的圆面积会保存在结构体 Circle 中的 area 中，所以函数不需要返回值，返回值定义为 void。

第 29-33 行为计算两点的距离，由于该函数不会修改 p1、p2 两个 Point 内的任何成员，故应该将这两个指针定义为 const 类型，防止函数内部无意或故意的修改。

同理第 36-39 行判断两圆是否相交，由于该函数不会修改 c1、c2 两个圆内的任何成员，也应将这两个指针定义为 const 类型。

第 37 行由于 Distance 函数的参数需要两个 Point 指针类型的参数，尽管 c1 与 c2 均为指针，但 c1->center 与 c2->center 不是指针，所以需要在其前用取地址符 &。

第 38 行的关系运算符本身的运算结果就为 bool 型，故不需要进行 if、else 类型的判断。相应的 main 函数测试代码如下：

```
1 int main()
2 {
3     Point pt1;
4     strcpy_s(pt1.name, 11, "pt1");
5     pt1.x = 100; pt1.y = 100; pt1.z = 425.324;
6
7     Point pt2;
8     strcpy_s(pt2.name, 11, "pt2");
9     pt2.x = 200; pt2.y = 200; pt2.z = 417.626;
10
11     Circle c1;
12     c1.center = pt1; c1.r = 80;
13
14     Circle c2;
15     c2.center = pt2; c2.r = 110;
16
17     Area(&c1) ; //计算圆c1的面积
18     printf("Circle1 的面积 = %lf \n", c1.area );
19
20     bool yes = IsIntersectWithCircle(&c1, &c2);
```

```
21     printf("Circle1 与 Circle2 是否相交 : %s\n", (yes ? "是" : "否") );
22
23     return 0;
24 }
```

程序的运行结果如下：

Circle1 的面积 = 20106.192983

Circle1 与 Circle2 是否相交 : 是

1.4.3 与 C 相对应的 C# 代码

将以上代码相对应的 C 代码直接翻译为 C# 代码为：

```
1 using System;
2
3 namespace Ch01Ex02
4 {
5     class Point
6     {
7         public string name;
8         public double x;
9         public double y;
10        public double z;
11
12        public double Distance(Point p2)
13        {
14            double dx = x - p2.x;
15            double dy = y - p2.y;
16            return Math.Sqrt(dx * dx + dy * dy);
17        }
18    }
19
20    class Circle
21    {
22        public Point center;
23        public double r;
24        public double area;
25        public double length;
26
27        public void Area()
28        {
29            area = Math.PI * r * r;
30        }
31
32        public bool IsIntersectWithCircle(Circle c2)
33        {
```

```
34         double d = this.center.Distance(c2.center);
35         return d <= (r + c2.r);
36     }
37 }
38 }
```

由以上 C# 代码可以看出与 C 代码的不同。这是因为 C# 语言是纯面向对象语言的缘故。

程序或软件的基本概念是：程序 = 数据结构 + 算法。在以上 C 或 C# 代码中，Point 和 Circle 都可以看作是数据结构，计算两点距离和圆的面积或判断两圆是否相交都可以看作是算法。在 C 语言中，数据结构和算法是分离的，尽管函数的参数是数据结构，但函数并不属于数据结构。在 C# 中则不同，数据结构用 class 定义，则数据与算法都属于同一个 class。

计算两点距离的函数设计为 Point 类的一个方法，可以看作是计算当前点与另一个点的距离，因此函数的参数就只有一个 Point p2，另一个自然是 this（当前类本身）了。

同样的道理圆的面积计算也不需要参数了（可以想象为计算知道自己的半径，计算自己的面积并存入自己的成员变量中了）。判断两圆是否相交也可以看作是判断当前圆与另一个圆 Circle c2 是否相交了，股只需要一个参数。

相应 C# 的 Main 函数测试代码如下：

```
1 using System;
2
3 namespace Ch01Ex02
4 {
5     class Program
6     {
7         static void Main(string[] args)
8         {
9             Point p1 = new Point();
10            p1.name = "p1";
11            p1.x = 100;
12            p1.y = 100;
13            p1.z = 425.324;
14
15
16            Circle c1 = new Circle();
17            c1.center = p1;
18            c1.r = 80;
19
20            Point p2 = new Point();
21            p2.name = "p2";
22            p2.x = 200;
23            p2.y = 200;
24            p2.z = 417.626;
25
26            Circle c2 = new Circle();
27            c2.center = p2;
28            c2.r = 110;
```

```
29
30     c1.Area(); //计算圆c1的面积
31     Console.WriteLine("Circle1的面积={0}", c1.area );
32
33     c2.Area(); //计算圆c2的面积
34     Console.WriteLine("Circle2的面积={0}", c2.area);
35
36     bool yes = c1.IsIntersectWithCircle(c2);
37     Console.WriteLine("Circle1与Circle2是否相交:{0}",
38         yes ? "是" : "否" );
39 }
40 }
41 }
```

程序的运行结果如下：

Circle1的面积=20106.1929829747

Circle2的面积=38013.2711084365

Circle1与Circle2是否相交：是

1.4.4 面向对象的 C# 代码

上面的 C# 不符合面向对象的软件设计原则（违背封装、继承与多态中的封装原则），且在 Main 函数中，如果由于疏漏忘记第 30 行或 33 行对圆的面积进行计算，圆 c1 或 c2 将不会有正确的面积。

因此，我们将其优化。首先对 Point 类进行封装，将其字段定义为 private，用属性方法将其向外暴露接口，相应的 C# 代码为：

```
1     private string name;
2     public string Name
3     {
4         get { return name; }
5         set { name = value; }
6     }
7
8     private double x;
9     public double X
10    {
11        get { return x; }
12        set { x = value; }
13    }
14    private double y;
15    public double Y
16    {
17        get { return y; }
18        set { y = value; }
19    }
```

```
20     private double z;  
21     public double Z  
22     {  
23         get { return z; }  
24         set { z = value; }  
25     }
```

以上是用 C# 的属性对 private 字段的进行简单的封装，也可以写成如下形式：

```
1 public string Name {get; set ;}  
2 public double X {get; set ;}  
3 public double Y {get; set ;}  
4 public double Z {get; set ;}
```

为了简化 Main 函数对 Point 类的调用及对其各字段值的初始化，我们定义构造函数如下：

```
1     public Point(double x, double y)  
2     {  
3         this.name = "";  
4         this.x = x;  
5         this.y = y;  
6         this.z = 0;  
7     }  
8  
9     public Point(string name, double x, double y, double z)  
10    {  
11        this.name = name;  
12        this.x = x;  
13        this.y = y;  
14        this.z = z;  
15    }
```

同样道理，也应对 Circle 进行封装，将其各个字段定义为 private，并用属性方法将其向外简单的暴露接口，相应的 C# 代码为：

```
1     public Point Center { get; set; }  
2  
3     private double r;  
4     public double R  
5     {  
6         get { return r; }  
7         set  
8         {  
9             if(value != r && value >= 0)  
10            {  
11                r = value;  
12                CalArea(); //r的值发生改变，重新计算圆的面积  
13            }  
14        }  
15    }
```

```
15 }
```

圆心 Center 只是简单的封装，圆的半径则不同。由圆的特性知，当圆的半径确定，其面积与周长也就确定了。为了计算的高效，封装时，当圆的半径值发生改变时，就需要重新计算圆的面积，确保面积的正确性。

由于圆的面积只与圆的半径有关，我们不需要在其他的情况对圆的面积进行修改，因此封装的 C# 代码为：

```
1 private double area;
2 public double Area
3 {
4     get { return area; }
5 }
```

在此我们去掉了属性 Area 中的 set 语句，并将 area 成员定义为了 private，使外部无法修改 area 的值，来保证圆的面积 Area 的正确性。

还有一个地方可能会修改圆的半径，就是圆的初始化时，因此圆的构造函数应定义为：

```
1 public Circle(double x, double y, double r)
2 {
3     Center = new Point(x, y);
4
5     this.R = r; //赋值给R而不是this.r，确保计算圆的面积
6 }
```

为确保在半径 r 的值发生改变后能重新计算圆的面积和周长，此处赋值给半径属性 R。完整的优化后的 C# 代码如下：

```
1 using System;
2
3 namespace Ch01Ex03
4 {
5     class Point
6     {
7         private string name;
8         public string Name
9         {
10             get { return name; }
11             set { name = value; }
12         }
13
14         private double x;
15         public double X
16         {
17             get { return x; }
18             set { x = value; }
19         }
20     }
21 }
```

```
19     }
20     private double y;
21     public double Y
22     {
23         get { return y; }
24         set { y = value; }
25     }
26     private double z;
27     public double Z
28     {
29         get { return z; }
30         set { z = value; }
31     }
32
33     public Point(double x, double y)
34     {
35         this.name = "";
36         this.x = x;
37         this.y = y;
38         this.z = 0;
39     }
40
41     public Point(string name, double x, double y, double z)
42     {
43         this.name = name;
44         this.x = x;
45         this.y = y;
46         this.z = z;
47     }
48
49     public double Distance(Point p2)
50     {
51         double dx = X - p2.X;
52         double dy = Y - p2.Y;
53         return Math.Sqrt(dx * dx + dy * dy);
54     }
55 }
56
57 class Circle
58 {
59     public Point Center { get; set; }
60
61     private double r;
62     public double R
63     {
64         get { return r; }
```

```
65         set
66         {
67             if(value != r && value >= 0)
68             {
69                 r = value;
70                 CalArea(); //r的值发生改变，重新计算圆的面积
71             }
72         }
73     }
74
75     private double area;
76     public double Area
77     {
78         get { return area; }
79     }
80
81     private double length;
82
83     public Circle(double x, double y, double r)
84     {
85         Center = new Point(x, y);
86
87         this.R = r; //赋值给R而不是this.r，确保计算圆的面积
88     }
89
90     //计算圆的面积
91     private void CalArea()
92     {
93         area = Math.PI * r * r;
94     }
95
96     //判断两圆是否相交
97     public bool IsIntersectWithCircle(Circle c2)
98     {
99         double d = this.Center.Distance(c2.Center);
100         return d <= (r + c2.r);
101     }
102 }
103 }
```

代码中由于计算圆的面积将会在属性 R 与构造函数中多次调用,我们将其定义为函数 CalArea 供复用,并将可访问性定义为 private 限制为内部使用。

相应 C# 的 Main 函数测试代码如下:

```
1 using System;
2
3 namespace Ch01Ex03
```

```
4 {  
5     class Program  
6     {  
7         static void Main(string[] args)  
8         {  
9             Circle c1 = new Circle(100, 100, 80);  
10            Circle c2 = new Circle(200, 200, 110);  
11  
12            Console.WriteLine("Circle1的面积={0}", c1.Area );  
13            Console.WriteLine("Circle2的面积={0}", c2.Area);  
14  
15            bool yes = c1.IsIntersectWithCircle(c2);  
16            Console.WriteLine("Circle1与Circle2是否相交:{0}",  
17                               yes ? "是" : "否" );  
18        }  
19    }  
20 }
```

由优化后的 C# 代码可以看出 Main 十分简洁。

1.5 作业

1. 请完成示例中的圆的周长计算功能；
2. 请试着完成多段线 Polyline 的面积与长度计算功能（提示：多段线是点序的集合）；
3. 请试着完成多边形 Polygon 的面积与长度计算功能。

提示：多边形面积计算公式为：

$$S = \frac{1}{2} \sum_{k=1}^6 (x_k y_{k+1} - x_{k+1} y_k)$$

多边形的顶点坐标依次为 $p_1(-1, 0)$, $p_2(2, 3)$, $p_3(4, 2)$, $p_4(4, 4)$, $p_5(6, 8)$, $p_6(-2, 5)$, 则依上式计算该多边形面积为 28.

第 2 章 C# 基础语法

C# 程序设计语言是类 C 语言，基本语法与 C 或 C++ 基本类似，比如块注释为 `/* */`，行注释为 `//`，每行语句用分号 `;` 分隔，C# 标识符区分大小写。C# 使用相同的大括号 `{ }`、中括号 `[ ]` 与圆括号 `( )`，控件语句 `if`、`else`、`switch`，循环语句 `for`、`while`、`do while` 等。

本章主要讲解 C# 中不同于 C 或 C++ 的语法。

2.1 C# 中的变量与数据类型

为保持 C# 语言的高效性，C# 中的数据类型分为两类：值类型 (value type) 和引用类型 (reference type)。从概念上讲，值类型变量直接存储其值，引用类型变量存储对其值的引用。

值类型变量的值通常直接在栈上保存，变量直接存储相应的值，因此变量的值自动生成、自动释放。当值类型的变量进行赋值或传参操作时，会创建该值的副本，对一个值类型变量的修改不会影响另一个。基本数据类型如 `int`、`float`、`double`、`decimal`、`bool`、`char` 与结构体 (struct)、枚举 (enum) 等都是值类型。

引用类型变量在堆中生成变量的值，在栈中保存堆中值的地址，当变量被释放时，在堆中生成的值不会自动释放，靠 .NET 架构回收。引用类型包括类 (class)、接口 (interface)、委托 (delegate)、数组 (array) 以及字符串 (string) 等。

值类型和引用类型在内存中的存储方式差异导致了它们在 C# 程序中的不同行为。例如，值类型在赋值时会复制整个数据，而引用类型在赋值时只复制引用。因此对引用类型的一个实例的修改会影响所有指向该实例的变量，而值类型的修改则限定于单个实例。

值类型适用于表示简单的数据结构，如数值、布尔值等。值类型通常具有更好的性能，因为它们可以直接存储在栈上，栈上的内存分配和回收速度非常快，在内存管理方面更高效，且不需要垃圾回收。

引用类型适用于表示更复杂的数据结构和对象，如字符串和数组等。它们支持继承和多态，但需要垃圾回收来管理内存。

2.1.1 值类型

C# 的值类型进一步划分为基本类型、枚举 (enum)、结构 (struct) 和可为 null 的值类型 (nullable type 或 `T?`)。

对于值类型，每个变量都有它们自己的数据副本（除 `ref` 和 `out` 参数变量外），因此对一个变量的操作不可能影响另一个变量。

表 2.1 常见值类型

类别	说明
基本类型	有符号整型: sbyte 、 short 、 int 、 long 无符号整型: byte、ushort、uint、ulong Unicode 字符: char IEEE 浮点: float、double 高精度小数型: decimal 布尔: bool
结构 (struct)	struct S ... 形式的用户定义的类型
枚举 (enum)	enum E ... 形式的用户定义的类型
可空 (T?)	派生自 System.Nullable<T> 泛型结构体, T? 为其别名

常见值类型如表 2.1 所示，在定义时可直接赋初值，如 `double x = 3785342.235;`。

C# 中常用的 DateTime 的定义为 `public readonly struct DateTime : .....`，因此 DateTime 是值类型数据。

2.1.2 引用类型

C# 的引用类型进一步划分为类 (class)、接口 (interface)、数组 (array) 和委托 (delegate)。对于引用类型，两个变量可能引用同一个对象，因此对一个变量的操作可能影响另一个变量所引用的对象。

表 2.2 常见引用类型

类别	说明
类 (class)	派生于 System.Object, 其别名为 object , 是所有其他类型的基类
接口 (interface)	interface I ... 形式的用户定义的类型
数组 (Array)	派生于 System.Array, 一维和 multidimensional 数组, 例如 int[] 和 int[,]
委托 (delegate)	派生于 System.Delegate, delegate int D(...) 形式的用户定义的类型
事件 (event)	派生于 System.Event

C# 中的 string 的类名为 System.String, string 是其别名，因此 string 是引用类型数据，string 类型数据可以在定义时直接赋初始值，如 `string s = "Hello, World!";`。

C# 的所有类型都继承自 System.Object 类，System.Object 是所有类型的基类。

除 string 等经过特殊处理的类型外，其他引用类型都需要通过 new 关键字来创建实例或将其他实例对象的引用赋值给它。如以下代码所示：

```
1 Point p1 = new Point(1, 2); //创建了一个 Point 类的实例对象 p
2 Point p2 = p1; // p2 引用 p1 所指向的对象
```

2.1.3 自定义类型

C# 程序使用类型声明 (type declaration) 创建新类型。类型声明指定新类型的名称和成员。在 C# 类型分类中，有七类是用户可定义的：类 (class)、结构 (struct)、接口 (interface)、枚举 (enum) 和委托 (delegate)、元组 (tuple) 和记录 (record)。

注：C#7.0 提供了元组 (tuple)，C#9.0 提供了记录 (record)。

类是定义了一个包含数据成员（字段）和函数成员（方法、属性等）的数据结构，类支持单一继承和多态。

结构与类相似，表示一个带有数据成员和函数成员的结构。但是，与类不同，结构是值类型，不需要堆分配。结构类型不支持用户指定的继承，所有结构类型都从类 System.ValueType 继承。

接口定义了一个协定，是一个公共函数成员的命名集。实现某个接口的类或结构必须提供该接口的函数成员的实现。一个接口可以从多个接口继承，而一个类或结构可以实现多个接口。

委托表示对具有特定参数列表和返回类型的方法的引用。通过委托，我们能够将方法作为实体赋值给变量和作为参数传递。委托类似于在其他某些语言中的函数指针的概念，但是与函数指针不同，委托是面向对象的，并且是类型安全的。

类、结构、接口和委托都支持泛型，因此可以通过其他类型将其参数化。

枚举是具有命名常量的独特的类型。每种枚举类型都具有一个基础类型，该基础类型必须是八种整型之一。枚举类型的值集和它的基础类型的值集相同。

C# 支持由任何类型组成的一维和多维数组。与以上列出的类型不同，数组类型不必声明就可以使用。实际上，数组类型是通过在某个类型名后加一对方括号来构造的。例如，int[] 是一维 int 数组，int[,] 是二维 int 数组，int[][] 是一维 int 数组的一维数组。

可以为 null 的类型也不必声明就可以使用。对于每个不可以为 null 的值类型 T，都有一个相应的可以为 null 的类型 T?，该类型可以容纳附加值 null。例如，int? 类型可以容纳任何 32 位整数或 null 值。

2.1.4 装箱与拆箱

C# 的类型系统是统一的，因此任何类型的值都可以按对象处理。C# 中的每个类型直接或间接地从 object 类类型派生，而 object 是所有类型的根基类。引用类型的值都被视为 object 类型，被简单地当作对象来处理。值类型的值则通过对其执行装箱和拆箱操作按对象处理。下面的示例将 int 值转换为 object，然后又转换回 int。

```
1 int i = 123;
2 object o = i;      // Boxing
3 int j = (int)o;    // Unboxing
```

当将值类型的值转换为类 object 时，将分配一个对象实例（也称为“箱子”）以包含该值，并将值复制到该箱子中。反过来，当将一个 object 引用强制转换为值类型时，将检查所引用的对象是否含有正确的值类型，如果有，则将箱子中的值复制出来。

在程序中频繁地装箱和拆箱操作会带来性能开销，应尽量避免不必要的装箱和拆箱操作。

2.1.5 延伸阅读值类型

所有值类型隐式继承自 `System.ValueType` ,而 `System.ValueType` 又继承自 `System.Object` 类。这听起来是不是有点自我矛盾? 其中有什么奥秘?

实际上值类型和引用类型都是从 `System.Object` 继承的, `System.Object` 类是 C# 中所有类的基类。所有值类型都隐式继承自类 `System.ValueType` ,而类 `System.ValueType` 又继承自类 `System.Object` (别名 `object`), 奥秘的地方在于 `System.ValueType` 用更合适的值类型实现重写了 `System.Object` 中的虚方法。

但任何类型都不能直接从值类型 `System.ValueType` 派生。

这样就有一个问题, 值类型与引用类型都是从 `System.Object` 继承的, 怎么 `object` 类型与 `int` 类型就不一样呢?

这里大致可以这样理解, .NET 是动态加载运行 C# 代码的, CLR 在运行时把值类型装载到栈, 把引用类型装载到堆, 栈中只提供一个引用指向堆, 即引用。

有关具体的值类型与引用类型的区别, 请参考有关微软官方文档。

2.1.6 变量的定义与 var 关键字

C# 中变量是前置定义的, 需先指定类型, 然后是变量名, 最后是其值, 如语句 `int i = 0;` 定义 `i` 为 `int` 类型的变量, 并赋初始值为 0。

变量也可隐式定义, 让编译器根据变量的值进行推定, 如语句 `var i = 0;` 中的 `i` 将由编译器推断出类型为 `int` 。

在程序设计中, 为了保持代码简洁, 定义变量时可以使用 `var` 关键字让编译器推断变量的数据类型。

值类型数据可以在声明时赋初始值, 引用类型变量在定义时需用 `new` 生成实例对象或将其它实例对象的引用赋值给它, 或直接赋值为 `null`。

2.1.7 变量的定义位置

C# 中变量的定义位置有三种: 字段、局部变量和参数。作为字段的变量在类或结构中定义, 作为局部变量的变量在方法、构造函数、属性或索引器中定义, C# 中的参数是方法、构造函数、属性或索引器的参数列表中的变量。

C# 中不存在全局变量, 所有变量都必须在类或结构中定义。

2.1.8 常量与只读变量

常量是一个在编译时就已知其值的变量, 常量的值在程序运行时不能被修改。常量的定义使用 `const` 修饰符, 常量必须在定义时初始化。

只读变量是一个在定义时初始化且在程序运行时不能被修改的变量。只读变量的定义使用 `readonly` 修饰符。

2.2 表达式与语句

2.2.1 表达式

表达式由操作数 (operand) 和运算符 (operator) 构成。表达式的运算符指示对操作数进行什么样的运算。运算符包括 +、-、*、/、% 和 new，操作数包括文本、字段、局部变量和表达式。常见的表达式有算术表达式、逻辑表达式、关系表达式、位运算表达式、条件表达式等，如表 2.3 所示：

表 2.3 常用表达式

类别	说明
x.m	成员访问
x(...)	方法和委托调用
x[...]	数组和索引器访问
x++	后增量
x--	后减量
new T(...)	对象和委托创建
new T(...)...	使用初始值设定项创建对象
new ...	匿名对象初始值设定项
new T[...]	数组创建
typeof(T)	获取 T 的 System.Type 对象
checked(x)	在 checked 上下文中计算表达式
unchecked(x)	在 unchecked 上下文中计算表达式
default(T)	获取类型 T 的默认值
delegate ...	匿名函数 (匿名方法)
(T)x	将 x 显式转换为类型 T
await x	异步等待 x 完成
x is T	如果 x 为 T，则返回 true，否则返回 false
x as T	返回转换为类型 T 的 x，如果 x 不是 T 则返回 null
(T x) => y	匿名函数 (lambda 表达式)

其它的与 C 或 C++ 语言基本相同。

2.2.2 语句

程序的操作是通过语句 (statement) 来表示与实现的。C# 中常见的操作句有声明变量、赋值、调用方法、循环访问集合、条件判断等，相应的语句有声明语句、表达式语句、循环语句、选择语句等。语句可以是以分号结尾的单个代码行，或是块中的一系列单行语句。语句块在大括号 { } 中，可以嵌套。

具体的语句类型如下所示：

- 声明语句 (declaration statement)

声明语句用于声明变量和常量，变量声明可以选择为变量赋值，在常量声明中必须赋值。

- 表达式语句 (expression statement)

表达式语句包括方法调用、使用 new 运算符的对象分配、使用 = 和复合赋值运算符赋值、使用增量 ++、减量 - 运算符以及 await 表达式等。

计算值的表达式语句必须将值存储在变量中。

- 选择语句

用于根据表达式的值从若干个给定条件的语句中选择一个来执行。选择语句有：

if ... else if ... else ... 、switch case、?:、??、?.

?? 表示如果左值为 null，则用右值进行计算，如下代码所示：

```
1 int? a = null;
2 int b = a ?? -1; // 如果a为null, 则 b 为 -1
3 Console.WriteLine(b); // output: -1
```

?.Method(...) 表示如果左值不为 null，则执行 ?. 后相应的操作，如下代码所示：

```
1 string? str1 = null;
2 int? length1 = str1?.Length; // str1 为 null, 则 length1 为 null
3 Console.WriteLine(length1); // output: null
4
5 string? str2 = "Hello, World!";
6 int? length2 = str2?.Length; // str2 不为 null, 则 length2 为 13
7 Console.WriteLine(length2); // output: 13
```

- 循环与迭代语句 (iteration statement)

用于重复执行嵌入语句，常用语句有 while、do、for 和 foreach 语句。

C# 中常用 foreach 语句进行循环，示例代码如下：

```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         foreach (string s in args)
6         {
7             Console.WriteLine(s);
8         }
9     }
10 }
```

假设编译出的程序名为 Demo，在命令行中输入 Demo.exe Hello World of C#，则输出结果为：

```
Hello
World
of
C#
```

第 3 行代码中的 `args` 表示命令行中以空格分隔的参数数组。

- 跳转语句 (jump statement)

用于转移控制，常用语句有 `break`、`continue`、`goto`、`throw`、`return` 和 `yield` 语句。

`break` 常表示中断当前的循环或 `switch case` 分支；`continue` 表示继续从头开始循环，`continue` 语句只能用于循环语句中；`goto` 表示跳转到指定标签处执行，`goto` 语句与跳转标签只能用于同一方法中，不能跨方法跳转；`throw` 表示抛出异常，`return` 表示返回方法的调用处，`yield` 表示返回迭代器的下一个元素。

这些语句之后的语句不能被执行。

- 异常捕捉

`try ... catch ... finally` 语句用于捕获在块的执行期间发生的异常，`finally` 语句用于指定终止代码，不管是否发生异常，该代码都始终要执行。

```
1 // x/y 在y==0 时可能会抛出异常
2 static double Divide(double x, double y)
3 {
4     if (y == 0) throw new DivideByZeroException();
5     return x / y;
6 }
7
8 static void Main(string[] args)
9 {
10    //使用 try...catch...finally 捕获异常
11    try
12    {
13        if (args.Length != 2)
14        {
15            throw new Exception("Two numbers required");
16        }
17        double x = double.Parse(args[0]);
18        double y = double.Parse(args[1]);
19        Console.WriteLine(Divide(x, y));
20    }
21    catch (Exception e)
22    {
23        Console.WriteLine(e.Message);
24    }
25    finally
```

```
26 {  
27     Console.WriteLine( "Good bye!" );  
28 }  
29 }
```

- using 语句

用于获得一个资源，执行一个语句，然后释放该资源。

```
1 static void Main()  
2 {  
3     using (TextWriter w = File.CreateText("test.txt"))  
4     {  
5         w.WriteLine("Line one");  
6         w.WriteLine("Line two");  
7         w.WriteLine("Line three");  
8     }  
9 }
```

在较新的语法中，上面代码可写成如下形式，从而减少 { } 的嵌套层次。

```
1 static void Main()  
2 {  
3     using TextWriter w = File.CreateText("test.txt");  
4     w.WriteLine("Line one");  
5     w.WriteLine("Line two");  
6     w.WriteLine("Line three");  
7 }
```

- is 与 as 语句

is 和 as 是两个用于类型转换和检查的关键字。

is 关键字用于检查一个对象是否兼容于指定的类型。如果兼容，则返回 true；否则，返回 false。

```
1 Object obj = new Object();  
2 if (obj is ClassA)  
3 {  
4     ClassA a = (ClassA)obj; // 使用强制类型转换  
5     Console.WriteLine("类型转换成功");  
6 }  
7 else  
8 {  
9     Console.WriteLine("类型转换失败");  
10 }
```

as 关键字用于尝试将一个对象转换为指定的类型。如果转换成功，则返回转换后的对象；如果转换失败，则返回 null，而不会抛出异常。

```
1 Object obj = new Object();
2 ClassA a = obj as ClassA; // 尝试将obj转换为ClassA类型
3 if (a == null)
4 {
5     Console.WriteLine("类型转换失败");
6 }
7 else
8 {
9     Console.WriteLine("类型转换成功");
10 }
```

C#7.0 引入了模式匹配，允许在 `is` 与 `as` 表达式中进行类型检查和转换，功能更加强大。

- 其他语句

`checked` 语句和 `unchecked` 语句，用于控制整型算术运算和转换的溢出检查上下文。

`lock` 语句用于获取某个给定对象的互斥锁，执行一个语句，然后释放该锁。

还有其它一些语句，在我们的程序设计中不太常用，在此不做深入探讨。

2.3 C# 中字符与字符串

2.3.1 字符 (char)

C# 有强大的字符与字符串处理能力，字符 `char` 是值类型，用单引号表示，存储 unicode 字符，如 `'A'`、`'中'`、`'\u4E2D'` (中的 unicode 编码) 等。

2.3.2 字符串 (string)

字符串 `string` 是引用类型，`string` 是 `System.String` 的别名，字符串一般用双引号表示，内容为系列字符的只读集合。表示字符串的字符可以是任何 Unicode 字符，如 `"Hello"`、`"世界"` 等。

字符串定义如下：

```
1 string message1;
2 string? message2 = null;
3 string message3 = System.String.Empty;
4 string message4 = "";
5 string oldPath = "c:\\Program Files\\Microsoft Visual Studio 8.0";
6 string newPath = @"c:\Program Files\Microsoft Visual Studio 9.0";
7 string greeting = "Hello World!";
8 var temp = "I'm still a strongly-typed System.String!";
9 const string message5 = "You can't get rid of me!";
10 char[] letters = { 'A', 'B', 'C' };
11 string alphabet = new string(letters);
12 string raw = ""
13 {
```

```
14     "path": "C:\\Users\\Name",  
15     "message": "Hello"  
16 }  
17 "";
```

上面的代码展示了字符串的多种定义方式，创建字符串对象不使用 `new` 运算符，除非使用字符数组初始化字符串；长度为零的字符串表示形式为 `""`，也可以使用 `String.Empty` 来表示长度为零的字符串。

通过使用 `String.Empty`（不是 `null`）初始化字符串，可以减少 `NullReferenceException` 发生的可能性。在访问字符串前，可以先使用静态 `String.IsNullOrEmpty(...)` 方法验证字符串的值是否为 `null` 或空。

字符串有三种形式：第 5 行用双引号包含起来的字符串称为字面字符串 (Quoted string literals)，在这种字符串中，转义字符起作用；第 6 行用 `@` 符号表示的字符串称为逐字字符串 (Verbatim string literals)，在这种字符串中，转义字符不起作用；第 7 行用三个双引号表示的字符串称为原始字符串 (Raw string literals)，在这种字符串中，在首尾 `"""` 中的任何内容都是字符串的一部分。

2.3.3 字符串的格式化

在很多情况下，我们需要将字符串与变量结合起来输出，这时就需要使用字符串格式化。`C#` 提供了多种字符串格式化的方法，其中最常用的是使用字符串插值和 `String.Format` 方法。

- 字符串插值

字符串插值是 `C#6.0` 引入的一种新的字符串格式化方式，它使用 `$` 符号和花括号 `{}` 来表示变量或表达式。例如已知某六十进制的角度值为 `123°3'35.78"`：

```
1 int d = 123;  
2 int m = 3;  
3 double s = 35.78;  
4 Console.WriteLine($"Angle value is {d}°{m}'{s}\".");
```

- `String.Format` 方法

`String.Format` 方法是 `C#` 中最常用的字符串格式化方法，它使用占位符 `index` 来表示变量或表达式。上面的代码可以改写为如下形式：

```
1 int d = 123;  
2 int m = 3;  
3 double s = 35.78;  
4 string angle = String.Format("Angle value is {0}°{1:00}'{2}\".", d, m, s);  
5 Console.WriteLine(angle); // 输出 123°03'35.78"
```

第 4 行代码中，`0`、`1:00` 和 `2` 分别表示 `d`、`m` 和 `s` 的值，其中 `1:00` 表示 `m` 的值以两位数的形式输出，不足两位时在前面补零。

- ToString() 方法

除了字符串插值和 String.Format 方法，C# 还提供了其它一些字符串格式化的方法，如 ToString() 方法，它可以将数值类型转换为字符串，并且可以指定格式。例如：

```
1 double pi = 3.14159265358979323846;  
2 Console.WriteLine(pi.ToString("F2")); // 输出 3.14  
3 Console.WriteLine(pi.ToString("E")); // 输出 3.141593E+000
```

2.3.4 字符串与数值类型之间的转换

很多情况下我们需要把字符串中的数据解析为可计算的数值型变量，最直接的方法是使用 Convert.ToInt32()、Convert.ToDouble() 或 int.Parse()、double.Parse() 等方法进行转换，如下代码所示：

```
1 string str = "123.521";  
2 var num = Convert.ToDouble(str);  
3 Console.WriteLine(num);
```

或

```
1 string str = "123.521";  
2 var num = Double.Parse(str);  
3 Console.WriteLine(num);
```

上面代码中，Convert.ToDouble() 方法和 Double.Parse() 方法都可能转换失败从而抛出异常。比较好的方法是使用 Double.TryParse() 方法，它会返回一个布尔值，表示转换是否成功，如果成功，则将转换后的值赋给指定的变量，如下代码所示：

```
1 string str = "123.521";  
2 if (Double.TryParse(str, out var num))  
3 {  
4     Console.WriteLine(num);  
5 }
```

更加严密的转换方法可能是先用正则表达式判断字符串是否符合数值格式，再进行转换。

2.3.5 string 的性能与 StringBuilder

string 字符串的内容是不能被修改的，每次对字符串进行修改时，都会创建一个新的字符串对象，这会导致频繁的内存分配和垃圾回收，从而影响性能。频繁的字符串修改与大量字符串拼接应尽可能使用 StringBuilder 类，最后用函数 ToString() 将其转换为 string。

2.4 C# 特殊语法

2.4.1 元组 (tuple)

元组是 C# 中一种轻量级的数据结构，用于将多个值组合成一个复合值。它在 C#4.0 中首次引入，随后在 C#7.0 中通过 ValueTuple 提供了更强大的功能。元组可以存储不同类型的元

素，并且无需显式定义字段类型和名称。

如测绘中常用的六十进制的角度值可以用显示的元组表示为：

```
1 (int d, int m, double s) a1 = (45, 30, 15.5);
2 Console.WriteLine($"Angle value is {a1.d}°{a1.m}'{a1.s}\");
3 // Output: Angle value is 45°30'15.5".
```

也可以隐式的定义为：

```
1 var a1 = (45, 30, 15.5);
2 Console.WriteLine($"Angle value is {a1.Item1}°{a1.Item2}'{a1.Item3}\");
3 // Output: Angle value is 45°30'15.5".
```

元组类型是值类型，元组元素是公共字段，若无显式类型指定，默认名称为 Item1、Item2、Item3 等。

还可以定义为：

```
1 var (d, m, s) = (45, 30, 15.5);
2 Console.WriteLine($"Angle value is {d}°{m}'{s}\");
3 // Output: Angle value is 45°30'15.5".
```

上面代码利用 var 关键字，由编译器根据右边的值来推断变量 d、m、s 的类型，从而保持代码的简洁。

2.4.2 记录 (record)

record 修饰符是用于封装数据的内置功能，提供了一种简洁的语法来定义不可变的数据对象。record 与 record class 语法定义引用类型，record struct 定义值类型。

如下代码定义了一个 record Point：

```
public record Point(string Name, double X, double Y, double Z);
```

在 record 上声明主构造函数时，编译器会为主构造函数参数生成公共属性。上面代码相当于如下代码：

```
1 public record Point
2 {
3     public required string Name { get; init; }
4     public required double X { get; init; }
5     public required double Y { get; init; }
6     public required double Z { get; init; }
7 };
```

其中 init 修饰符表示属性只能在对象初始化时设置值，之后不能修改。

record 类型的对象是不可变的，意味着一旦创建了一个 record 对象，其属性值就不能被修改。这有助于确保数据的一致性和安全性，尤其是在多线程环境中。

我们在 Top-Level Statement 中测试使用 record 类型，如下代码所示：

```
1 Point point1 = new("GP10", 3804770.542, 608960.290, 471.823);
2 // point1.Name = "GP11";
```

```
3 Console.WriteLine(point1);
4 // output: Point { Name = GP10, X = 3804770.542, Y = 608960.290, Z = 471.823 }
5
6 Point point2 = new("GP10", 3804770.542, 608960.290, 471.823);
7 Console.WriteLine(point2);
8 // output: Point { Name = GP10, X = 3804770.542, Y = 608960.290, Z = 471.823 }
9
10 Console.WriteLine( point1 == point2); // Output: True
11 Console.WriteLine(point1.Equals(point2)); // Output: True
12
13 public record Point(string Name, double X, double Y, double Z);
```

point 对象的属性值在初始化后不能被修改，尝试修改会导致编译错误，所以上面代码中第 2 行代码被注释掉了。

第 9、10 行代码中，point1 和 point2 对象的属性值完全相同，因此它们被认为是相等的，输出结果为 True。这也是 record 类型不同于 class 的一个重要特性，即它们的相等性是基于属性值的，而不是基于对象引用。

record struct 的相等性定义基本与 struct 等同。

2.4.3 泛型

泛型是现代编程语言非常重要的语法。在 C# 中，泛型方法是一种强大的编程特性，它允许方法在声明时使用类型参数，从而实现对多种数据类型的灵活支持。泛型在代码复用和类型安全性方面具有显著优势。

- 泛型的定义

在定义类时，在类名后用尖括号括起来的类型参数列表，称之为泛型。

以下代码展示了在类中使用泛型：

```
1 public class Pair<T1, T2>
2 {
3     public T1 first;
4     public T2 second;
5 }
```

代码中的 T1 与 T2 为类型参数。

以下代码展示了在方法中使用泛型交换两个数的值：

```
1 static void Swap<T>(ref T l, ref T r)
2 {
3     T temp = l;
4     l = r;
5     r = temp;
6 }
```

上述代码中，T 是类型参数，表示该方法可以操作任意类型的数据。参数为 ref T l 和 ref T r，表示传入的参数是引用类型，可以在方法内部修改它们的值。

- 泛型的调用

泛型方法的调用需要指定类型参数，编译器会根据传入的参数类型推断出类型参数的具体类型。

```
1 int a = 1, b = 2;
2 Swap<int>(ref a, ref b); // 显式指定类型
3 Swap(ref a, ref b); // 类型推断
4 Console.WriteLine($"{a}, {b}"); // 输出: 2, 1
```

- 泛型的约束

泛型约束用于限制类型参数的范围，以确保类型参数满足特定的条件。这有助于在编译时捕获潜在的错误，并提供更好的类型安全性。C# 中泛型通过 where 关键字对类型参数进行约束。

```
1 void SwapIfGreater<T>(ref T l, ref T r) where T : IComparable<T>
2 {
3     if (l.CompareTo(r) > 0)
4     {
5         T temp = l;
6         l = r;
7         r = temp;
8     }
9 }
```

上述代码中，where T : IComparable<T> 表示类型参数 T 必须实现 IComparable<T> 接口，这样我们就可以在方法内部使用 CompareTo 方法来比较两个值。

2.4.4 集合

C# 以类库形式提供了数组 (Array)、列表 (List)、词典 (Dictionary)、栈 (Stack)、队列 (Queue) 等集合数据结构，集合中的元素可以是任意类型的数据。

- 数组

数组是相同数据类型的元素按一定顺序排列的集合，在存储数据时是连续存储的，可以快速遍历元素。数组的大小与元素类型一般在定义时需确定，这类数组称为静态数组，如 C# 中的 System.Array，如下代码所示：

```
1 int[] a1 = new int[10]; // 长度为10的一维整型数组
2 int[] a11 = new int[]{1,2,3,4,5,6,7,8,9,10}; // 由初始化元素等个数确定数组
   的长度
3 int[,] a2 = new int[10, 10]; // 长度为10x10的二维整型数组
4 int[,] a21 = new int[,]{{1,2,3},
5                        {4,5,6},
```

```

6         {7,8,9} }; // 长度为3x3的二维整型数组
7 int[][] a3 = new int[10][]; // 长度为10的一维数组的数组，也称为交错数组(
    jagged array)
8 int[][] a31 = new int[10][]; // 长度为10的交错数组
9 for(int i=0; i<10; i++)
10 {
11     a31[i] = new int[10]; // 每个一维数组的长度为10
12 }

```

上述代码中，a1、a2、a3 是静态数组，a11、a21、a31 是动态数组。

- 列表

列表是一个可变大小的集合，可以动态地添加或删除元素。列表在内存中不是连续存储的，但它提供了类似于数组的索引访问方式。列表的大小可以根据需要动态调整，这类集合称为动态数组，如 C# 中的 System.Collections.Generic.List<T>，如以下代码所示：

```

1 List<int> numbers = [1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15];
2 numbers.ForEach(number => Console.Write(number + ", "));
3 Console.WriteLine();
4
5 Console.WriteLine(numbers[1]);
6 Console.WriteLine(string.Join(', ', numbers[1..5]));
7 Console.WriteLine(numbers[^1]);
8 Console.WriteLine(string.Join(', ', numbers[^5..^1]));
9 Console.WriteLine(string.Join(', ', numbers[^5..]));

```

上述代码第 1 行，定义了一个 List<int> 类型的动态数组 numbers，这是采用了表达式初始化器的方式进行了定义，还可以用如下之一的定义方式：

```

1 List<int> numbers1 = new List<int>(){1,2,3,4,5,6,7,8,10,11,12,13,14,15};
2 List<int> numbers2 = new(){1,2,3,4,5,6,7,8,10,11,12,13,14,15};
3 var numbers3 = new List<int>(){1,2,3,4,5,6,7,8,10,11,12,13,14,15};

```

第 2 行代码使用了 Lambda 表达式的方式来遍历列表中的每个元素，并将其打印出来。输出结果为：1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15,

传统的遍历输出方式为：

```

1 foreach (int number in numbers)
2 {
3     Console.Write(number + ", ");
4 }

```

List<T> 几乎是 C# 中最常用的集合类（动态数组），List<T> 是一个泛型集合类，T 表示集合中元素的类型。

- 切片 (Slice)

切片是对数组或列表的一个连续子集的引用，它允许我们在不复制数据的情况下访问和操作原始集合的一部分。..¹ 表示切片一个区域，[^] 表示从倒数第几个元素开始。

在上小节代码第 5 行代码使用了索引访问方式来获取列表中索引为 1 的元素，输出结果为：2

第 6 行代码使用了切片 (Slice) 的方式来获取列表中索引为 1 到 4 的元素，输出结果为：2,3,4,5

第 7 行代码使用了索引访问方式来获取列表中最后一个元素，输出结果为：15

第 8 行代码使用了切片 (Slice) 的方式来获取列表中倒数第 5 到倒数第 1 的元素，注意切片是左闭右开区间，也就是不包括最后一个元素，所以输出结果为：11,12,13,14

第 9 行代码使用了列表中倒数第 5 个元素至末尾，输出结果为：11,12,13,14,15

在使用这些集合类时，应尽量使用泛型集合，如动态数组：List<T>、栈：Stack<T>、队列：Queue<T>、字典：Dictionary<TKey, TValue> 等，而不是使用非泛型集合，如 ArrayList、Stack、Queue、Hashtable 等。

2.4.5 委托与事件

- 委托 (delegate)

委托是一种类型，表示对具有特定参数列表和返回类型的方法的引用。委托类似于 C、C++ 中的函数指针。语法为：

```
public delegate <return type> <delegate-name>(<parameter list>);
```

从语法上可以看出，delegate 之后实际上就是函数的定义。

.NET 中预定义了 Action、Func 和 Predicate 委托。

Action 委托表示没有返回值的委托，参数最多有 16 个，语法形式为：

```
public delegate void Action<in T1, ..., in T16>(T1 arg1, ..., T16 arg2);
```

Func 委托表示有返回值的委托，参数最多有 16 个，最后一个参数必须是返回参数 TResult，语法形式为：

```
public delegate TResult Func<in T1, ..., in T15, out TResult>(T1 arg1, ..., T15 arg15);
```

Predicate 委托表示返回 bool 值的方法，通常用于条件判断。

可以将 delegate 看作是一种数据类型，它是存储对方法的引用。委托可以像普通方法一样被调用，并且可以作为参数传递给其他方法。

- 事件 (event)

事件 (event) 是一种特殊类型的委托，用于发布和订阅机制。在 C# 中，事件本质上就是一个封装了委托的类型，它用于响应程序中的某些操作。

```
1 public class Program
2 {
3     static void Main()
4     {
5         var button = new Button();
6         button.Click += ButtonClick; // 订阅事件
7         button.OnClick(); // 引发事件, 输出 "Button clicked!"
8     }
9
10    static void ButtonClick(object? sender, EventArgs e)
11    {
12        Console.WriteLine("Button clicked!");
13    }
14 }
15
16 public class Button
17 {
18     // 定义一个事件
19     public event EventHandler? Click;
20
21     // 引发事件的方法
22     public void OnClick()
23     {
24         Click?.Invoke(this, EventArgs.Empty); // 调用事件
25     }
26 }
```

有关委托与事件更多的知识请参考相应的微软官方文档。

2.4.6 Lambda 语句

Lambda 表达式主要用于简化委托的编写形式, 是一种可用于创建委托或表达式树类型的匿名函数。语法为:

(parameter list) => { 表达式或语句块 }

Lambda 表达式实质是一个匿名函数, 如下代码所示:

```
1 List<int> list = new(){1,2,3,4,5,6,7,8,10,11,12,13,14,15};
2 var q = list.Where( x => x%2 == 0); // 取偶数
3 Console.WriteLine(string.Join(",", q)); // 输出 2,4,6,8,10,12,14
```

list.Where 函数中调用的就是 Lambda 表达式。

2.4.7 迭代器

C# 迭代器是一种用来遍历集合对象的方法, 核心要素包括 yield return、yield break 和 foreach 循环。它能按需逐个返回元素, 不需要一次性把所有数据都加载到内存中。

迭代器方法或 `get` 访问器可对集合执行自定义迭代。迭代器方法使用 `yield return` 语句返回元素，每次返回一个。到达 `yield return` 语句时，会记住当前在代码中的位置，下次调用迭代器函数时，将从该位置重新开始执行。

如下所示代码，`Range` 函数是迭代器函数，返回从 `from` 到 `to` 间隔为 2 的整数序列。

```
1 Console.WriteLine(string.Join(", ", Range(-10, 10, 2)));
2
3 static IEnumerable<int> Range(int from, int to, int step = 1)
4 {
5     for (int i = from; i < to; i += step)
6     {
7         yield return i;
8     }
9 }
```

上述代码输出为：-10, -8, -6, -4, -2, 0, 2, 4, 6, 8

迭代器方法或 `get` 访问器的返回类型可以是 `IEnumerable`、`IEnumerable<T>`、`IEnumerator` 或 `IEnumerator<T>`。

`yield break` 语句用来提前结束迭代过程。

2.5 模式匹配 (Pattern Matching)

模式匹配是 C# 语言中比较特别的语法，许多现代程序设计语言也支持模式匹配，模式匹配使得编程代码更加与自然语言接近，让代码更加友好、可读，已成为现代编程语言的强大核心特性。

模式匹配也是 C# 的强大语言特性，是一种测试表达式是否具有特定特征的方法，能够测试表达式的类型、值、属性或结构并在匹配时直接解构或执行某些功能。

C# 的模式匹配主要包含 `is` 表达式、`switch` 表达式和各种内置模式。`is` 表达式支持通过模式匹配来测试表达式，并根据测试结果有条件声明一个新的变量。`switch` 表达式允许根据表达式的首次匹配模式执行操作。这两个表达式支持丰富的模式词汇。

2.5.1 Null 检查 (is 表达式)

模式匹配最常见的方案之一是确保值不是 `null`。

```
1 int? maybe = 12;
2
3 if (maybe is int number) // is
4 {
5     Console.WriteLine($"The nullable int 'maybe' has the value {number}");
6 }
7 else
8 {
9     Console.WriteLine("The nullable int 'maybe' doesn't hold a value");
10 }
```

上述代码是声明模式，用于测试变量类型并将其分配给新变量。

```
1 string? message = ReadMessageOrDefault();
2
3 if (message is not null) // is not
4 {
5     Console.WriteLine(message);
6 }
```

上述代码使用常数模式将变量与之进行比较，not 为一种逻辑模式，在否定模式不匹配时与该模式匹配。

2.5.2 类型测试 (is 表达式)

模式匹配的另一常见用途是测试变量是否与给定类型匹配。

```
1 public static T MidPoint<T>(IEnumerable<T> sequence)
2 {
3     if (sequence is IList<T> list)
4     {
5         return list[list.Count / 2];
6     }
7     else if (sequence is null)
8     {
9         throw new ArgumentNullException(nameof(sequence), "Sequence can't be
10         null.");
11     }
12     else
13     {
14         int halfLength = sequence.Count() / 2 - 1;
15         if (halfLength < 0) halfLength = 0;
16         return sequence.Skip(halfLength).First();
17 }
```

2.5.3 比较离散值 (switch 表达式)

```
1 public State PerformOperation(Operation command) =>
2     command switch
3     {
4         Operation.SystemTest => RunDiagnostics(),
5         Operation.Start => StartSystem(),
6         Operation.Stop => StopSystem(),
7         Operation.Reset => ResetToReady(),
8         _ => throw new ArgumentException("Invalid enum value for command",
9         nameof(command)),
9     };
```

第 8 行代码中的 `_` 为弃元模式，处理值与定义的 `enum` 值之一不匹配的任何错误条件。

可用数值常量、常量字符串值或 `Span<char>` 或 `ReadOnlySpan<char>` 代替枚举值集，示例代码如下：

```
1 //public State PerformOperation(string command) =>
2 public State PerformOperation(ReadOnlySpan<char> command) =>
3     command switch
4     {
5         "SystemTest" => RunDiagnostics(),
6         "Start" => StartSystem(),
7         "Stop" => StopSystem(),
8         "Reset" => ResetToReady(),
9         _ => throw new ArgumentException("Invalid string value for command",
10         nameof(command)),
11     };
```

`Span<char>` 表示连续字符集，`ReadOnlySpan<char>` 表示只读的连续字符集。

2.5.4 关系模式

关系模式测试如何将数值与常量进行比较。以下代码基于华氏温度返回水源状态：

```
1 string WaterState(int tempInFahrenheit) =>
2     tempInFahrenheit switch
3     {
4         < 32 => "solid",
5         32 => "solid/liquid transition",
6         (> 32) and (< 212) => "liquid",
7         212 => "liquid / gas transition",
8         > 212 => "gas",
9     };
```

上述代码中的 32°F 和 212°F 为熔点和沸点值。

2.5.5 多个输入

```
1 public record Order(int Items, decimal Cost); //订单项数、价值
2
3 public decimal CalculateDiscount(Order order) => //折扣
4     order switch
5     {
6         { Items: > 10, Cost: > 1000.00m } => 0.10m,
7         { Items: > 5, Cost: > 500.00m } => 0.05m,
8         { Cost: > 250.00m } => 0.02m,
9         null => throw new ArgumentNullException(nameof(order), "Can't calculate
10         discount on null order"),
11         var someObject => 0m,
12     };
```

上面代码可以用位置模式简写成如下形式，其中表达式的属性已析构：

```
1 public record Order(int Items, decimal Cost); //订单数量、价值
2
3 public decimal CalculateDiscount(Order order) =>
4     order switch
5     {
6         ( > 10, > 1000.00m) => 0.10m,
7         ( > 5, > 50.00m) => 0.05m,
8         { Cost: > 250.00m } => 0.02m,
9         null => throw new ArgumentNullException(nameof(order), "Can't calculate
        discount on null order"),
10        var someObject => 0m,
11    };
```

还可以将属性与 { } 匹配，{ } 可与任何非 null 值匹配。

2.5.6 列表模式

可以使用列表模式检查列表或数组中的元素。列表模式提供了一种方法，将模式应用于序列的任何元素。此外，还可以应用弃元模式 (_) 来匹配任何元素，或者应用切片模式来匹配零个或多个元素。

如有一包含银行交易信息的文本文件内容如下：

04-01-2020,	DEPOSIT,	Initial deposit,	2250.00
04-15-2020,	DEPOSIT,	Refund,	125.65
04-18-2020,	DEPOSIT,	Paycheck,	825.65
04-22-2020,	WITHDRAWAL,	Debit,	Groceries, 255.73
05-01-2020,	WITHDRAWAL,	#1102,	Rent, apt, 2100.00
05-02-2020,	INTEREST,		0.65
05-07-2020,	WITHDRAWAL,	Debit,	Movies, 12.57
04-15-2020,	FEE,		5.55

一个包含弃元模式、常量模式和 var 模式的列表模式用于捕获数值数据处理过程代码如下：

```
1 decimal balance = 0m;
2 foreach (string[] transaction in ReadRecords())
3 {
4     balance += transaction switch
5     {
6         [_, "DEPOSIT", _, var amount] => decimal.Parse(amount),
7         [_, "WITHDRAWAL", .., var amount] => -decimal.Parse(amount),
8         [_, "INTEREST", var amount] => decimal.Parse(amount),
9         [_, "FEE", var fee] => -decimal.Parse(fee),
10        - => throw new
        InvalidOperationException($"Record {string.Join(", ", transaction)} is not
        in the expected format!"),
```

```
11     };  
12     Console.WriteLine($"Record: {string.Join(", ", transaction)}, New balance:  
    {balance:C}");  
13 }
```

C# 模式匹配功能强大，更多有关高级用法请参考相应文档说明。

注：该节示例代码来自 Microsoft Learn。

2.6 小结

这一章我们学习了 **C#** 基本语法：数据与语句，数据分为数值类型与引用类型，数据定义时可以使用 `var` 关键字让编译器推断变量的数据类型，还可以使用泛型。

C# 语句相对于 **C**、**C++** 而言，出现了一些新的如 `lambda` 语句、迭代器等，这些是许多现代语言中必不可少的语言特征，我们将在后续课程中将其继续学习与应用。

2.7 上机实验

1. 冒泡排序 (Bubble Sort)

冒泡排序是一种较简单的排序算法。它重复地走访过要排序的元素列，依次比较两个相邻的元素，如果顺序错误就把他们交换过来。走访元素的工作是重复地进行直到没有相邻元素需要交换，也就是说该元素列已经排序完成。这个算法的名字由来是因为越小的元素会经由交换慢慢“浮”到数列的顶端（升序或降序排列），就如同碳酸饮料中二氧化碳的气泡最终会上浮到顶端一样，故名“冒泡排序”。

原理如下：

比较相邻的元素。如果第一个比第二个大，就交换他们两个。

针对所有的元素重复以上的步骤，除了最后一个。

持续每次对越来越少的元素重复上面的步骤，直到没有任何一对数字需要比较；

2. 快速排序 (Quick Sort)

基本思想：通过一趟排序将待排记录分隔成独立的两部分，其中一部分记录的关键字均比另一部分的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。

算法描述：

快速排序使用分治法来把一个串（list）分为两个子串（sub-lists）。具体算法描述如下：

从数列中挑出一个元素，称为“基准”（pivot）；

重新排序数列，所有元素比基准值小的摆放在基准前面，所有元素比基准值大的摆在基准的后面（相同的数可以到任一边）。在这个分区退出之后，该基准就处于数列的中间位置。这个称为分区（partition）操作；

递归地（recursive）把小于基准值元素的子数列和大于基准值元素的子数列排序。

第 3 章 C# 面向对象基础

类 (class) 是面向对象程序设计的基础与核心，类也是 C# 语言最基础最重要的类型，是一种将数据（字段或属性）与对数据的操作（方法、函数等）组合在一起的数据结构。

3.1 类 (class)

3.1.1 类与实例对象 (instance object)

类 (class) 是创建对象的蓝图或模板，对象 (object) 是根据类创建的具体实例 (instance)。我们以点 (Point) 与具体的某个点 (如 11 号教学楼旁的那个名叫 GP12 的点) 为例，点就是类，它是一个抽象的概念，定义了点的属性与行为，而具体点 GP12 就是点的实例对象。

3.1.2 类的实例与静态类、静态属性 (static)

不是所有的类都有实例对象。没有实例或不需要实例的类定义为静态类 (static class)。一个被定义为静态的类不能生成类的实例。如 C# 中的数学类 System.Math，在这个类中定义各种常数与常用数学算法，但没有任何实例数据和方法，因此将其定义为静态类 (static class)。

一个实例类中也可以有静态成员 (static member)，静态成员属于类本身，而不是类的实例，C# 中在成员前加关键字 static 将其定义为静态成员。

类的静态成员调用方法为：类名. 静态成员名，而类的实例成员调用方法为：实例对象名. 实例成员名。

在类的静态方法中不能直接调用非静态成员，需通过实例对象进行调用；但在实例方法中可以直接调用静态成员。

在此声明，若非特别语境下，以后本书中所指的类均为实例类。

3.1.3 类的定义

类的关键字为 class，定义一个类的方式如下：

```
[public/internal] class ClassName [: BaseClass, Interface1, ..., InterfaceN]
{
    .....
}
```

先指定类的修饰符，class 关键字后是类名称（类名首字母一般大写），接着是要继承的基类与需实现的接口。之后是类的主体，它由一组位于一对大括号“{”和“}”之间的成员声明组成。

由于测绘行业的基本任务是提供位置服务的，点位是我们经常使用的概念，我们就对它来定义一个类 Point：

```
public class Point : System.Object
{
    .....
}
```

上面是测量点的简单定义，关键字 class 前的 public 为访问限定符，表示访问不受限制。也可以为 internal，表示该类仅限于当前程序集访问，外部程序集不能访问。如果在 class 前不加访问修饰符，默认访问权限为 internal。

点名后的“: System.Object”表示 Point 类从 System.Object 类（System 为命名空间）继承。C# 中 System.Object 类为所有类的根基类，如果不写类的基类，则默认从 System.Object 继承，上例中“: System.Object”可以省略。

上面的代码可以简写为：

```
public class Point
{
    .....
}
```

3.1.4 类的实例对象

在 C# 中用 new 操作符生成类的实例对象。该运算符为新的实例分配内存、调用构造函数初始化实例，并返回对该实例的引用。

下面的语句是生成 Point 实例对象，可以合理使用 C# 的推断功能，在较新的 C# 语法中，在知道类型的情况下，可以简化 new 的写法：

```
1 {
2     .....
3     Point p1 = new Point();
4     var p2 = new Point();
5     Point p3 = new();
6     .....
7 }
```

当程序执行到这段代码后边的“}”时，将自动释放变量 p1、p2、p3，p1、p2 与 p3 所指向的内存，将由.NET 系统自动收回。

3.1.5 类的字段 (Filed)

一个类通常由两部分组成：数据与对数据的操作。字段是与类或类的实例相关的变量，用于存储类的数据。

如上面的点 Point 类，我们可以为其定义数据成员如下：

```
1 public class Point
2 {
3     public string? name;
4     public string? code;
5     public double x;
6     public double y;
7     public double z;
8 }
```

在定义点时，点的位置属性 (x, y, z) 很容易理解，但很容易忽略点的点名与编码两个属性，其实，测绘中往往用点名指代某个位置，点也有控制点与一般点之分，需要用编码区分点的类型，所以点名与编码这两个字段也就很有必要。

3.1.6 类字段的可访问性

从以上点 Point 类的定义看出，每个字段的定义前都有 public，代表着对字段的访问权限。类的每个成员都可以关联可访问性，以控制访问该类成员的权限，默认的可访问性为 private。

有六种可能的可访问性形式，如下表所示：

表 3.1 字段的可访问性

可访问性	含义
public	访问不受限制
protected	访问仅限于该类及该类的派生类
internal	访问仅限于当前程序集
protected internal	访问仅限于当前程序集或该类的派生类
private	访问仅限于包含类
private protected	访问限于包含类或当前程序集中派生自包含类的类型。

一个类可以访问自己的属性，可以通过 this 关键字进行引用访问，在不引起混淆的情况下，可以省略 this。this 关键字指类的实例本身。

在 Point 类的定义 public 访问权限的数据成员后，就可以按照“实例. 数据成员”进行引用了，如下代码所示：

```
1 {
2     .....
3     Point p1 = new Point();
4     p1.name = "p1";
5     p1.x = 100.0;
6     p1.y = 100.0;
7     p1.z = 410.0;
8
9     Point p2 = new Point();
```

```
10     p2.name = "p2";
11     p2.x = 200.0;
12     p2.y = 200.0;
13     p2.z = 420.0;
14     .....
15 }
```

3.1.7 类的构造函数

从上面的 p1、p2 的引用语句看，实际上是在给 p1、p2 两个点赋初值。如果每个点都这样做的话，十分不方便。初始化实例对象更好的方法是使用构造函数。

构造函数是函数名称与类名完全相同，且没有返回类型（即使返回类型为 void 也不行）的函数，它的作用是做类的初始化工作（主要是初始化类的数据）。

定义与使用 Point 类的代码如下：

```
1 //在Point.cs文件中定义
2 public class Point : System.Object
3 {
4     public string? name;
5     public string? code;
6     public double x;
7     public double y;
8     public double z;
9
10    public Point(string name, string code, double x, double y, double z)
11    {
12        this.name = name;
13        this.code = code;
14        this.x = x;
15        this.y = y;
16        this.z = z;
17    }
18 }
19
20 //在Program.cs文件中Main()函数中使用
21 Point p1 = new Point("p1", "", 100.0, 100.0, 410.0);
22 Point p2 = new Point("p2", "", 200.0, 200.0, 420.0);
```

构造函数支持重载 (overload)。所谓的**函数重载**就是函数名称相同，但函数的参数类型或参数个数不一样。请注意，函数重载没有关注函数的返回值，只关注了函数的参数。

没有参数的构造函数称为**默认构造函数**。如果类中不定义任何构造函数，系统会为我们生成一个默认构造函数，一旦我们定义了一个构造函数，系统就不会为我们再生成这个默认构造函数。如果仍然要使用这个默认的构造函数，需要显式定义。

为了方便 Point 类的初始化，我们为 Point 类定义多个构造函数如下：

```
1 //在Point.cs文件中定义
```

```
2 public class Point : System.Object
3 {
4     public string? name;
5     public string? code;
6     public double x;
7     public double y;
8     public double z;
9
10    public Point() //默认构造函数
11    {
12        this.name = null;
13        this.code = null;
14        this.x = 0.0;
15        this.y = 0.0;
16        this.z = 0.0;
17    }
18
19    public Point(double x, double y)
20    {
21        this.name = null;
22        this.code = null;
23        this.x = x;
24        this.y = y;
25        this.z = 0.0;
26    }
27
28    public Point(double x, double y, double z)
29    {
30        this.name = null;
31        this.code = null;
32        this.x = x;
33        this.y = y;
34        this.z = z;
35    }
36
37    public Point(string name, double x, double y)
38    {
39        this.name = name;
40        this.code = null;
41        this.x = x;
42        this.y = y;
43        this.z = 0.0;
44    }
45
46    public Point(string name, double x, double y, double z)
47    {
```

```
48     this.name = name;
49     this.code = null;
50     this.x = x;
51     this.y = y;
52     this.z = z;
53 }
54
55 public Point(string name, string code, double x, double y, double z)
56 {
57     this.name = name;
58     this.code = code;
59     this.x = x;
60     this.y = y;
61     this.z = z;
62 }
63 }
```

注意以上构造函数中的 `this` 关键字不可省略, `this.name` 限定了 `name` 是类中的 `name`, 而“=”后的 `name` 根据就近原则指的是函数参数中的 `name`。其含义为将参数 `name` 的值赋值给类中的 `name` 字段。

有了上面重载形式的构造函数, 我们就可以多种方式生成 `Point` 类的实例了, 如下代码所示:

```
1 // 在Program.cs文件中Main()函数中使用
2 Point p1 = new Point();
3 Point p2 = new Point(100.0, 100.0);
4 Point p3 = new Point(300.0, 300.0, 430.0);
5 Point p4 = new Point("p4", 400.0, 400.0);
6 Point p5 = new Point("p5", 500.0, 500.0, 450.0);
7 Point p6 = new Point("p6", "001", 600.0, 600.0, 460.0);
```

构造函数只能用于 `new` 操作符生成类的实例, 不能显式调用。

构造函数的访问权限一般定义为 `public`, 在某些特殊的场景中也可以定义为 `private`。如果定义为 `private`, 该类的 `new` 操作只能在该类的静态函数中使用, 在其它类中是无法生成该类的实例。

构造函数不能被继承, 但子类在其构造函数中可以使用 `base` 关键字进行调用。

3.1.8 类的属性 (property)

在前面的 `Point` 类中我们将各个字段定义为 `public` 访问权限, 这会导致在任何能引用到类的实例的地方都能修改类中字段的数据, 很显然这不符合封装的原则。在很多情况下, 我们需要对类中数据的访问与修改添加一定的规则, 比如测量行业中的坐标 (`x`、`y`) 一般情况下不能为负值。

因此, 我们需将 `Point` 类中的字段访问权限定义为 `private` 或 `protected`, 代码如下:

```
1 // 在Point.cs文件中定义
```

```
2 public class Point
3 {
4     private string name;
5     private string code;
6     private double x;
7     private double y;
8     private double z;
9
10    //此处省略掉Point的构造函数，代码如前面所示
11 }
```

为了支持对 Point 类中字段的快捷访问，C# 中引入了**属性 (Property)**。属性 (Property) 是字段的自然扩展，是 C# 核心语法，也是后面界面编写的核心。

属性和字段都是类的命名成员，都具有相关的类型，访问字段和属性的语法也相同。

与字段不同的是属性不存储数据。属性本质上讲是两个函数/方法 (get 与 set)，也叫做访问器 (accessor)，get 代表读数据，set 代表写数据，访问器指定在它们的值被读取或写入时需执行的语句。

属性的声明与字段类似，不同的是属性声明以位于定界符 { 和 } 之间的一个 get 访问器和/或一个 set 访问器结束，不是以分号结束。

Point 类属性定义代码如下所示：

```
1 //在Point.cs文件中定义
2 public class Point : System.Object
3 {
4     private string name="";
5     public string Name
6     {
7         get{ return name;}
8         set{ name = value;}
9     }
10
11     private string code="";
12     public string Code
13     {
14         get{ return code;}
15         set{ code = value;}
16     }
17
18     private double x=0.0;
19     public double X
20     {
21         get{ return x;}
22         set{ x = value;}
23     }
24
25     private double y=0.0;
```

```
26     public double Y
27     {
28         get{ return y;}
29         set{ y = value;}
30     }
31
32     private double z=0.0;
33     public double Z
34     {
35         get{ return z;}
36         set{ z = value;}
37     }
38
39     //此处省略掉Point的构造函数，代码如前面所示
40 }
```

同时具有 get 访问器和 set 访问器的属性是读写属性 (read-write property)，只有 get 访问器的属性是只读属性 (read-only property)，只有 set 访问器的属性是只写属性 (write-only property)。

get 访问器相当于一个具有属性类型返回值的无形参方法。除了作为赋值的目标，当在表达式中引用属性时，将调用该属性的 get 访问器以计算该属性的值。

set 访问器相当于具有一个名为 value 的参数并且没有返回类型的方法。当某个属性作为赋值的目标被引用，或者作为 ++ 或 -- 的操作数被引用时，将调用 set 访问器，并传入提供新值的实参。

如果属性比较简单，可以使用 lambda 表达式进行简写：

```
1 private double x;
2 public double X
3 {
4     get => x;
5     set => x = value;
6 }
```

如果属性只有 get 部分，如 X 属性可以写成：

```
1 private double x;
2 public double X => x;
```

如果属性不使用字段的话，可以简写成 get/set 形式，如 X 属性可以写成：

```
1 //private double x;
2 public double X
3 {
4     get;
5     set;
6 }
```

属性的访问权限一般都定义为 public，用于对外交换数据。

3.1.9 类的实例方法 (method)

方法也称为函数，是类的一种行为成员，用于实现类的计算或操作。

- 方法的签名 (signature)

方法的签名在声明该方法的类中必须唯一。方法的签名由方法的名称、参数类型、参数个数、返回类型、修饰符组成。

方法可以有重载形式，但必须符合重载规则。

方法的参数 (parameter) 表示传递给该方法的值或变量，参数列表可以为空；

方法还有一个返回类型 (return type)，用于指定该方法的返回值类型。如果方法没有返回值，返回类型必须定义为 void，不能不定义或省略。

- 方法/函数的参数

参数用于向方法传数据，可以传值或传变量。有四类参数：值参数、引用参数、输出参数和参数数组。

- 值参数 (value parameter)

值参数用于传递值类型数据。一个值参数相当于一个局部变量，它的初始值来自于该形参传递的实参数据。值参数在传递时，实际上是传递值参数的一个拷贝，因此在函数内部对值参数的任何修改不会影响到传值的实参的。值参数在传值时可以直接传递数值。

- 引用参数 (reference parameter)

引用参数可用于传递输入和输出的数据。在方法执行期间，引用参数与实参变量表示同一存储位置，因此为引用参数传递的实参必须是变量，而且必须是具有初始值的变量。

引用参数使用 ref 修饰符声明。

下面的交换两个变量值的代码演示了 ref 参数的用法：

```
1 using System;
2 class Test
3 {
4     static void Swap(ref int x, ref int y) // 交换两个变量的值
5     {
6         int temp = x;
7         x = y;
8         y = temp;
9     }
10
11     static void Main()
12     {
13         int i = 1, j = 2; // 变量 i 与 j 必须初始化
14         Swap(ref i, ref j);
15         Console.WriteLine("{0} {1}", i, j); // 输出为 2 1
```

```
16     }
17 }
```

– 输出参数 (output parameter)

输出参数用于函数输出计算数据值。对于输出参数来说，调用方提供的实参的初始值并不重要。除此之外，输出参数与引用参数类似。输出参数用 `out` 修饰符声明的。

下面的代码演示 `out` 参数的用法：

```
1 using System;
2 class Test
3 {
4     static void Divide(int x, int y, out int result, out int remainder)
5     {
6         result = x / y;
7         remainder = x % y;
8     }
9
10    static void Main()
11    {
12        Divide(10, 3, out int res, out int rem);
13        Console.WriteLine("{0} {1}", res, rem); // Outputs "3 1"
14    }
15 }
```

第 12 行代码为较新式的 C# 语法。

– 参数数组 (parameter array)

参数数组允许向方法传递可变数量的实参。

参数数组使用 `params` 修饰符声明。只有方法的最后一个参数才可以是参数数组，并且参数数组的类型必须是一维数组类型。

`System.Console` 类的 `Write` 和 `WriteLine` 方法就是参数数组用法的很好示例，它们的声明如下：

```
1 public class Console
2 {
3     public static void Write(string fmt, params object[] args) {...}
4     public static void WriteLine(string fmt, params object[] args)
5     {...}
6 }
```

在使用参数数组的方法中，参数数组的行为完全就像常规的数组类型参数。但是，在具有参数数组的方法的调用中，既可以传递参数数组类型的单个实参，也可以传递参数数组的元素类型的任意数目的实参。在后一种情况下，将自动创建一个数组实例，并使用给定的实参对它进行初始化。

传递参数数组类型的单个实参代码示例如下：

```
1 Console.WriteLine("x={0} y={1} z={2}", x, y, z);
```

上面代码等价于以下语句：

```
1 string s = "x={0} y={1} z={2}";
2 object[] args = new object[3];
3 args[0] = x;
4 args[1] = y;
5 args[2] = z;
6 Console.WriteLine(s, args);
```

现在我们为 Point 类增加一个方法：计算两点的平面距离，定义如下：

```
1 //在Point.cs文件中定义
2 using System;
3 public class Point : System.Object
4 {
5     //其它省略，代码如前面所示
6     public double Distance(Point other)
7     {
8         double dx = this.x - other.x;
9         double dy = this.y - other.y;
10        return Math.Sqrt(dx*dx + dy*dy);
11    }
12 }
```

注意这个函数的定义，初学者可能认为这个函数需要两个参数，因为两个点嘛，实际上这是一个实例类，其本身就是一个点，程序中用 this 关键字表示的，因此只需参数传入另一个点 Other。

方法的访问权限与类字段相同。

3.1.10 类的静态成员

前边所讲的字段、属性、方法均为类的实例成员，它们为类的每个实例所有，在类中可以用关键字 this 进行应用。

如果一个类的成员不属于某个类实例，则应定义为类的静态成员，静态成员属于定义它的类，通过类名进行引用。

类的静态成员有静态字段，用于存储属于整个类的数据；有静态属性，用于读写类的静态字段；也有静态方法，用于操作类的静态数据；类中的常量也属于类的静态字段。

使用 static 修饰符声明的字段、属性、方法为类的静态成员。

试想我们要对前面我们定义的点 Point 类生成的实例对象进行计数，这个计数的字段最佳的定义方式就是定义为类的静态字段（实例对象的个数不应该存在于某个实例中，应属于整个点 Point 类）。

为了能够访问这个计数数字，我们也可为它定义一个静态只读属性，代码如下：

```
1 //在Point.cs文件中定义
2 public class Point
3 {
```

```
4     private static int count = 0;
5     public static int Count => count;
6
7     //此处省略掉Point的其它部分，代码如前面所示
8 }
```

现在我们要站在上帝的视角来实现两个点的距离计算，同样可以将函数定义为静态的，由于静态函数中没有实例对象了，所以参数也应该定义为两个点，实现的代码如下：

```
1 //在Point.cs文件中定义
2 using System;
3 public class Point
4 {
5     //其它省略，代码如前面所示
6     public static double Distance(Point from, Point to)
7     {
8         double dx = from.x - to.x;
9         double dy = from.y - to.y;
10        return Math.Sqrt(dx*dx + dy*dy);
11    }
12 }
```

静态方法中不能使用 this 关键字。

3.1.11 类的实例成员与静态成员的关系

一个类的静态方法或属性不能直接通过 this 关键字调用类的实例成员（如字段、属性、方法等），但可以通过类的实例对象调用类的实例成员。如上面静态计算两点距离的函数更好的实现方法为：

```
1 //在Point.cs文件中定义
2 using System;
3 public class Point
4 {
5     //其它省略，代码如前面所示
6     public static double Distance(Point from, Point to)
7     {
8         return from.Distance(to);
9     }
10 }
```

也就是通过类 Point 的实例对象 from 调用实例方法 Distance 实现计算，这样可以避免写相同或相似的计算平距的代码两份。

由于静态成员是属于整个类的，因此实例方法或属性是可以直接读写类的静态字段的，也是可以直接调用类的静态方法和属性的。

比如我们要在类 Point 添加一个 Id 属性，实现 Id 的自增，外部不能修改这个 Id 值，则可以在类 Point 的构造函数中调用 Point 的静态字段 count 实现自增并赋值给 id 实例字段，这样每个 Point 类实例都有自己的 Id 属性值了。完整的 Point 类代码如下：

```
1 //在Point.cs文件中定义
2 public class Point
3 {
4     private static int count = 0;
5     public static int Count => count;
6
7     private int id;
8     public int Id => id;
9
10    private string name="";
11    public string Name
12    {
13        get => name;
14        set => name = value;
15    }
16
17    private string code="";
18    public string Code
19    {
20        get => code;
21        set => code = value;
22    }
23
24    private double x=0.0;
25    public double X
26    {
27        get => x;
28        set => x = value;
29    }
30
31    private double y=0.0;
32    public double Y
33    {
34        get => y;
35        set => y = value;
36    }
37
38    private double z=0.0;
39    public double Z
40    {
41        get => z;
42        set => z = value;
43    }
44
45    public Point() //默认构造函数
46    {
```

```
47     this.name = null;
48     this.code = null;
49     this.x = 0;
50     this.y = 0;
51     this.z = 0;
52
53     id = ++count;
54 }
55
56 public Point(double x, double y)
57 {
58     this.name = null;
59     this.code = null;
60     this.x = x;
61     this.y = y;
62     this.z = 0;
63
64     id = ++count;
65 }
66
67 public Point(double x, double y, double z)
68 {
69     this.name = null;
70     this.code = null;
71     this.x = x;
72     this.y = y;
73     this.z = z;
74
75     id = ++count;
76 }
77
78 public Point(string name, double x, double y)
79 {
80     this.name = name;
81     this.code = null;
82     this.x = x;
83     this.y = y;
84     this.z = 0;
85
86     id = ++count;
87 }
88
89 public Point(string name, double x, double y, double z)
90 {
91     this.name = name;
92     this.code = null;
```

```
93         this.x = x;
94         this.y = y;
95         this.z = z;
96
97         id = ++count;
98     }
99
100     public Point(string name, string code, double x, double y, double z)
101     {
102         this.name = name;
103         this.code = code;
104         this.x = x;
105         this.y = y;
106         this.z = z;
107
108         id = ++count;
109     }
110
111     public override string ToString()
112     {
113         return $"Id={id}, Name={name}, Code={code}, X={x}, Y={y}, Z={z}";
114     }
115
116     public double Distance(Point other)
117     {
118         double dx = this.x - other.x;
119         double dy = this.y - other.y;
120         return Math.Sqrt(dx*dx + dy*dy);
121     }
122
123     public static double Distance(Point p1, Point p2)
124     {
125         return p1.Distance(p2);
126     }
127 }
```

以上代码的第 7、8 行定义了只读的 Id 属性，第 53、64、75、86、97、108 行为在类 Point 的所有构造函数中先将计数 count 加 1，赋值给 id 字段，从而实现 Id 属性的自我增长。

第 111 至 114 行重写了 ToString() 函数，这样就可以方便的输出 Point 的信息了，在 Main 函数中的测试代码如下：

```
1 //在Program.cs文件中Main()函数中使用
2 Point p1 = new Point();
3 Console.WriteLine(p1);
4
5 Point p2 = new Point(100.0, 100.0);
6 Console.WriteLine(p2);
```

```

7
8 Point p3 = new Point(300.0, 300.0, 430.0);
9 Console.WriteLine(p3);
10
11 Point p4 = new Point("p4", 400.0, 400.0);
12 Console.WriteLine(p4);
13
14 Point p5 = new Point("p5", 500.0, 500.0, 450.0);
15 Console.WriteLine(p5);
16
17 Point p6 = new Point("p6", "001", 600.0, 600.0, 460.0);
18 Console.WriteLine(p6);
19
20 Console.WriteLine($"已生成的总点数为: {Point.Count}");
21 Console.WriteLine($"p1--p2点的距离为: {p1.Distance(p2)}");
22 Console.WriteLine($"p3--p4点的距离为: {Point.Distance(p3, p4)}");

```

测试代码的输出为:

```

Id=1, Name=, Code=, X=0, Y=0, Z=0
Id=2, Name=, Code=, X=100, Y=100, Z=0
Id=3, Name=, Code=, X=300, Y=300, Z=430
Id=4, Name=p4, Code=, X=400, Y=400, Z=0
Id=5, Name=p5, Code=, X=500, Y=500, Z=450
Id=6, Name=p6, Code=001, X=600, Y=600, Z=460
已生成的总点数为: 6
p1--p2点的距离为: 141.4213562373095
p3--p4点的距离为: 141.4213562373095

```

3.1.12 类成员扩展阅读

C# 中类的成员不仅有字段、方法、属性、常量等，还有其它的一些类型成员，在本课程中不常用到，所以在此就不详讲了，如果在后续用到了我们再进行讲解。有兴趣的同学可以参照下表自行阅读：

C# 中类的成员如表 3.2 所示：

类中成员的类型也可以用泛型类型进行定义，则这个类就是泛型类了。

如下面的代码中，类 Pair 中就有两个类型参数 T1 和 T2：

```

1 public class Pair<T1,T2>
2 {
3     public T1 First;
4     public T2 Second;
5 }

```

泛型类在实际使用时必须类型化，如下所示：

表 3.2 类成员

成员	说明
常量	与类关联的常量值
字段	类的变量
方法	类可执行的计算和操作
属性	与读写类的命名属性相关联的操作
索引器	与以数组方式索引类的实例相关联的操作
事件	可由类生成的通知
运算符	类所支持的转换和表达式运算符
构造函数	初始化类的实例或类本身所需的操作
析构函数	在释放类的实例之前执行的操作
类型	类所声明的嵌套类型

```
1 Pair<int, string> pair = new Pair<int, string> { First = 1, Second = "two" };
2 int i = pair.First;      // TFirst is int
3 string s = pair.Second;  // TSecond is string
```

有关泛型的深入阅读请阅读相应的资料。

3.2 静态类与静态构造函数

3.2.1 静态类

如果一个类的所有数据成员、属性、方法均与类的单个实例没有关系，则这个类应定义为静态类。静态类中的所有成员均为静态成员。

类的静态成员通过类名进行引用。

静态方法和静态属性的定义是在访问修饰符后加 `static` 关键字，其它的与实例类中的定义相同。如我们在前面将排序算法时定义的 `Sort` 类，更应该以静态类的形式定义为如下形式：

```
1 //Sort.cs 文件中的内容
2 using System.Collections.Generic;
3
4 namespace ZXY
5 {
6     public static class Sort
7     {
8         public static int BubbleSort(List<int> arr)
9             ..... //此处省略的代码与前面的相同
10
11         public static int QuickSort(List<int> arr)
12             ..... //此处省略的代码与前面的相同
13     }
14 }
```

在调用时就不用再生成类 Sort 的示例了，直接用类名 Sort 进行引用调用，代码如下：

```
1 //Program.cs 文件中Main函数中的内容
2
3 //Sort sort = new Sort();//定义为静态的排序算法，此语句就没有必要
4 Stopwatch st = new Stopwatch();
5 st.Start();
6 //Sort.BubbleSort(list); //冒泡排序，通过类名引用
7 Sort.QuickSort(list);    //快速排序，通过类名引用
8 st.Stop();
9 Console.WriteLine($"运行时间：{st.Elapsed}");
```

3.2.2 静态构造函数

C# 中所有的类均有一个静态构造函数 (static constructor)，它是在第一次加载类本身时执行，用于实现类的初始化操作 (也是在类的所有构造函数执行之前执行)。

在构造函数声明前加入 static 修饰符，则显示声明了静态构造函数。

比如我们可以在前面的 Point 类中加入静态构造函数，如下所示：

```
1 namespace ZXY
2 {
3     public class Point
4     {
5         private static int count;
6
7         static Point() //静态构造函数
8         {
9             count = 0;
10            Console.WriteLine("这是Point类静态构造函数");
11        }
12
13        //其它代码省略
14    }
15 }
```

第 7 至 11 行为静态函数内容，第 9 行用于初始化静态字段 count 的值，第 10 行用于观察静态构造函数执行的时机，从输出结果可以观察到静态构造函数优先于 Point 类的其它函数执行。

静态函数没有重载形式，也不能被显示调用。在类被使用到时由系统自动调用。

3.3 用静态类与静态函数建立测量基础函数库

本节我们将利用已学的静态类与静态函数知识编写一些测量程序设计中常用的函数，以用于后续的测绘软件编写与设计。

3.3.1 C# 知识点

C# 是纯面向对象语言，所有的常量与方法都需要以类 *class* 为载体。

C# 中的类分为实例类与静态类。实例类需要用关键字 *new* 将类实例化，在一些与类的实例成员操作无关的环境中，对类进行实例化的操作显得冗余，而静态类与类的静态成员可以很好的简化这个问题。

我们分析 C# 系统中的 *System.Math* 类，会发现常用的一些数学函数都被设计成了静态函数。因此我们也模仿 *System.Math* 类设计，将我们常用的测绘算法与常量保存在 *SurMath.cs* 文件中，并命名类名为 *SurMath*(*Survey* + *Math*)，用 *SurMath.XXX* 形式引用。

我们新建一类库 (class library)，项目命名为 *SurApp.SurMath*。在项目中新建一文件 *SurMath.cs*，在文件 *SurMath.cs* 中编写静态类 *SurMath*，示例代码如下所示：

```
1 using System;
2
3 namespace ZXY;
4
5 public static class SurMath
6 {
7     public const double PI = Math.PI;
8     public const double TWOPI = 2 * Math.PI;
9     public const double HALFPI = 0.5 * Math.PI;
10    public const double TORADIANS = PI / 180.0;
11    public const double TODGREES = 180.0 / PI;
12    public const double TOSECONDS = 180.0 * 3600.0 / PI;
13
14    .....
15 }
```

以上示例代码为了与其他的函数或符号相区别，也为了与其他的代码一起配合使用，在此加入了自已的命名空间 *ZXY*（这是我用的名称空间，你当然也可以根据自己的习惯或爱好命名适当的名称空间）。

3.3.2 测绘常用算法设计

测绘算法中的常量

角度、距离与高差是测量工程师工作的基本对象，度分秒形式的角度与弧度之间的转换是我们进行测量数据处理的基础。为了方便的进行测量数据处理，我们首先定义一些常量，如上述示例代码所示，其中：

PI 表示 π ，*TWOPI* 表示 2π ，*HALFPI* 表示 $\pi/2$ ；*TODGREES* 表示 $*180/\pi$ ，用于将弧度化为度；*TORADIANS* 表示 $*\pi/180$ ，用于将度化为弧度；*TOSECONDS* 表示 $*180 * 3600/\pi$ ，用于将弧度转换为秒。

在静态类 *SurMath* 中我们采用常量 *const* 的形式定义了 π 、 2π 等常用的数值，在 C# 中 *const* 类型的数据总是静态的 (static)，而且是不需要 *static* 修饰符的。另外需注意，*const* 类型的数据在声明时必须进行初始化，且其值必须在编译时就能确定计算出。

六十进制度分秒化弧度函数

在测量工程中角度的常用习惯表示法是度分秒的形式，在计算程序中测量工程人员也常将度分秒形式的角度用格式为 xxx.xxxxx 的形式表示，即以小数点前的整数部分表示度，小数点后两位数表示分，从小数点后第三位起表示秒。在计算机编程时所用的角度要以弧度表示的，因此需要设计函数相互转换。

角度化弧度函数的逻辑非常简单，许多测量编程人员将其写成如下的形式：

```
1 public static double DmsToRadians(double dmsAngle)
2 {
3     int d = (int)dmsAngle;
4     dmsAngle = (dmsAngle - d) * 100.0;
5     int m = (int)dmsAngle;
6     double s = (dmsAngle - m) * 100.0;
7     return (d + m / 60.0 + s / 3600.0) * TORADIANS;
8 }
```

但由于计算机中浮点数的表示方法（IEEE754 标准）的原因，以上函数设计并不能精确的将度分秒形式的角度转换为弧度。

如角值为 1°40'00"，以 1.4000 浮点数输入，计算机将表示为 1.3999999999 的形式。这在计算机中并没有什么错误，但以上函数在提取角度的分秒时，提取到的 m 值为 39，提取到的 s 值为 99.9999999999，即我们提取到的角度为 1°39'100"，有 40" 的角度误差，显然这是测绘工程人员不能接受的误差。

有的软件设计人员在软件中发现这个问题后的处理的办法是让用户在角度后加减一秒，进行规避这种误差，显然，将软件人员的责任推给用户是极其不合适和不负责任的。还有许多的书籍中介绍了许多五花八门的处理方法，但奏效的小，不奏效的却很多，甚至有的将这么简单的算法逻辑变得逻辑十分复杂。

虽然浮点数的表达不够精确，但我们知道在计算机中，整数的表达与计算却是精确的，因此在角度的度分秒值提取中，我们采用先将度分秒角度值提取为整数，然后再提取度分秒的运算方式进行计算，相应代码如下。

```
1 public static double DmsToRadians(double dmsAngle)
2 {
3     dmsAngle *= 10000;
4     int angle = (int)dmsAngle;
5     int d = angle / 10000;
6     angle -= d * 10000;
7     int m = angle / 100;
8     double s = dmsAngle - d * 10000 - m * 100;
9     return (d + m / 60.0 + s / 3600.0) * TORAD;
10 }
```

比如表示 1°40'00" 的 1.4，首先将浮点形式的六十进制角度值乘以 10000（以秒为单位，秒本身就为浮点数），1.4 * 10000 的结果不会表达为 13999.999999999999 的形式，应是准确的 14000.000，再将其转换为整数 14000，利用整数整除的精确算法将度 (d) 与分 (m) 提取出来。

为了保持秒值的有效精度，用精度无损失的 dmsAngle 值减去 d 与 m 的值提取出秒值 (s)。这样既可以正确提取出度与分值，又可以保证秒值的有效精度。算法虽然简单，但确实保证了准确度与精度这两个方面的要求。

经分析，以上算法对于负的角度值的转换同样有效。

考察精度，如秒之后五位小数 $39^{\circ}52'0.71672''$ ，同样满足精度要求。

对 DmsToRadians 函数进行单元测试

DmsToRadians 函数是测量算法中的基础函数，必须保证其正确性，必须对其正确性进行测试。

测试算法最直接的方法就是目视观察比对，但随着算法的增加与优化、修改等，目视观察比对方法显然不能满足需求。我们需要引入单元测试 (UnitTest)，让算法的编写者首先对自己的算法进行测试，保证算法的正确性。

新建一测试项目，命名为 SurApp.TestSurMath，测试架构采用 xUnit，在项目中添加对项目 SurApp.SurMath 项目的引用。在项目中新建一文件 UnitTestSurMath.cs，在文件中编写测试代码。

```
1 using Moq;
2 using ZXY;
3
4 namespace UnitTestSurMath;
5
6 public class UnitTestSurMath
7 {
8     [Fact]
9     public void TestDmsToRadians()
10    {
11        var r = SurMath.DmsToRadians(101.02201);
12        Assert.Equal(1.76346176848151, r, 1e-14);
13
14        r = SurMath.DmsToRadians(-101.02201);
15        Assert.Equal(-1.76346176848151, r, 1e-14);
16
17        r = SurMath.DmsToRadians(1.4000);
18        Assert.Equal(0.0290888208665722, r, 1e-14);
19
20        r = SurMath.DmsToRadians(-1.4000);
21        Assert.Equal(-0.0290888208665722, r, 1e-14);
22
23        r = SurMath.DmsToRadians(1.40001);
24        Assert.Equal(0.02908930568025330, r, 1e-14);
25
26        r = SurMath.DmsToRadians(-1.40001);
27        Assert.Equal(-0.02908930568025330, r, 1e-14);
28    }
```

弧度函数化六十进制度分秒

同样的道理，以下函数将不能正确的将一些弧度值转换为度分秒形式的角度值，在某些情况下转换出的角度将出现 59'60" 或 59'59.9999" 的形式。

```
1 public static double RADtoDMS(double radAngle)
2 {
3     radAngle *= TODEG;
4     int d = (int)radAngle;
5     radAngle = (radAngle - d) * 60;
6     int m = (int)radAngle;
7     double s = (radAngle - m) * 60;
8     return (d + m / 100.0 + s / 10000.0);
9 }
```

我们需要利用整数的精确表达能力与计算能力来进行转换，正确的代码如下：

```
1 public static double RADtoDMS(double radAngle)
2 {
3     radAngle *= TOSECOND;
4     int angle = (int)radAngle;
5     int d = angle / 3600;
6     angle -= d * 3600;
7     int m = angle / 60;
8     double s = radAngle - d * 3600 - m * 60;
9     return (d + m / 100.0 + s / 10000.0);
10 }
```

首先我们将弧度转化为以秒为单位的 double 值，这个转换过程是精确的。其次，我们将其四舍五入转换为整秒值（虽然这个过程有精度损失，但我们仅用这个值提取度与分值）。再利用整数的精确计算能力提取度与分值，最后用无精度损失的 radAngle 值减去度与分值就可以提取出正确的度分秒值了，而且也可以保证秒值的有效精度。

在这个算法中，将弧度值首先转换为秒为单位的值是关键，保证秒值的精确有效位数也很重要。

同样这个算法对负值也适用。

角度规划函数

在导线计算中，我们推算出的方位角值常常超出 $0 - 2\pi$ 的范围，这时我们就需要将角度规划到 $0 - 2\pi$ 范围内。函数代码如下所示，rad 的单位为弧度。

```
1 public static double To0_2PI(double rad)
2 {
3     int f = rad >= 0 ? 0 : 1;
4     int n = (int)(rad / TWOPI);
5
6     return rad - n * TWOPI + f * TWOPI;
7 }
```

8

以上算法我们采用去整周角的算法, 先计算出角度中所包含的 2π 个数 n , 然后减去这 n 个 2π 值。

对于负的角度, 以上算出的角度为负值, 因此我们根据角度的符号性决定在角度为负值时再多加一个 2π 。

坐标方位角计算

在测量中, 常常需要根据两点的坐标反算其坐标方位角, 计算坐标方位角的关键是进行象限判断了。

已知 A 点与 B 点的坐标: A(x_A, y_A), B(x_B, y_B), 计算 A→B 的坐标方位角, 函数名称定义为 Azimuth, 函数返回值为两点的坐标方位角, 单位为弧度, 相应的代码如下:

```
1 public static double Azimuth(double xA, double yA, double xB, double yB)
2 {
3     double dx = xB - xA;
4     double dy = yB - yA;
5     return Math.Atan2(dy, dx) + (dy < 0 ? 1 : 0) * TWOPI;
6 }
```

在以上代码中, 我们没有用常用的 Math.Atan 函数, 该函数的取值范围为 $-\pi/2 - \pi/2$, 取值区间为 π , 与坐标方位角的取值区间 2π 不相符, 将其转换到坐标方位角的取值范围内十分麻烦, 而且还需要判断 dx 是否为 0 的情况。

查阅 MSDN 中对 Math.Atan2 函数的解释, 我们发现其取值范围为 $-\pi - \pi$, 刚好为 2π 区间, 与坐标方位角的定义一致。且第一、第二象限的计算值为 $0 - \pi$, 与测量上的方位角定义一致; 第三、第四象限的计算值为 $-\pi - 0$, 我们将其平移 2π 区间就可以将其转换到测量的第三、第四象限内, 使其与坐标方位角的定义一致。测量上第三、第四象限的判断条件为 $dy < 0$, 故只需要在 Math.Atan2 的计算值上用三目运算符 “? : ” 加上修正值 $(dy < 0 ? 1 : 0) * TWOPI$ 就可以了。这样可以最大程度的保持代码的简洁了。

计算 A、B 两点的平距函数设计如下:

```
1 public static double Distance(double xA, double yA, double xB, double yB)
2 {
3     double dx = xB - xA;
4     double dy = yB - yA;
5     return Math.Sqrt(dx * dx + dy * dy);
6 }
```

可以看出, Azimuth 函数与 Distance 函数十分相似, 为了提高效率, 也可以将这两个函数合并在一起, 代码如下所示:

```
1 public static double Azimuth(double xA, double yA, double xB, double yB, out
    double azimuth)
2 {
3     double dx = xB - xA;
4     double dy = yB - yA;
```

```

5 azimuth = Math.Atan2(dy, dx) + (dy < 0 ? 1 : 0) * TWOPI;
6 return Math.Sqrt(dx * dx + dy * dy);
7 }

```

在测量计算中，往往方位角的计算更加常用和重要，因此我们用 out 形式回带计算出的坐标方位角值，以函数返回值的形式返回两点的平距，当不需要平距时，不接收函数返回值就可以了。

算法的扩展

六十进制的 ddd.mmss 值十分利于角度数据的组织与输入，但不利于正式场景的角度展示。因此，在很多情况下我们需要将角度表示为 23°05′47.6324″ 或 -23°05′47.6324″ 这种形式。我们可以将前面的函数拷贝改写成如下形式的代码：

```

1 public static string DMStoString(double dmsAngle)
2 {
3     dmsAngle *= 10000;
4     int angle = (int)Math.Round(dmsAngle);
5     int d = angle / 10000;
6     angle -= d * 10000;
7     int m = angle / 100;
8     double s = dmsAngle - d * 10000 - m * 100;
9     return string.Format("{0}°{1:00}'{2:00.0####}"', d, m, s);
10 }

```

第 9 行语句利用 string 的 Format 函数将度分秒值组合为字符串形式，其中的分值我们保持为两位整数数据，不足两位的前面填 0，同样的秒值整数部分也做这样的处理。秒值的小数部分保留五位小数，如果后边为零的话自动去掉相应的小数位，但至少保持到 0.1”。

这样在我们的代码中就存在着两份实现提取度分秒功能的代码了。在程序设计中，本着相同或相似的代码应尽可能的只写一次的原则，我们应该将这部分功能代码独立出来，如下所示：

```

1 public static void DMStoDMS(double dmsAngle, out int d, out int m, out double s
2 )
3 {
4     dmsAngle *= 10000;
5     int angle = (int)Math.Round(dmsAngle);
6     d = angle / 10000;
7     angle -= d * 10000;
8     m = angle / 100;
9     s = dmsAngle - d * 10000 - m * 100;
10 }

```

然后在 DMStoRAD 函数和 DMStoString 函数中进行调用，保持功能代码的唯一性。

```

1 public static double DMStoRAD(double dmsAngle)
2 {
3     DMStoDMS(dmsAngle, out int d, out int m, out double s);
4     return (d + m / 60.0 + s / 3600.0) * TORAD;

```

```

5 }
6
7 public static string DMStoString(double dmsAngle)
8 {
9     DMStoDMS(dmsAngle, out int d, out int m, out double s);
10    return $"{d}°{m:00}'{s:00.0####}''";
11 }

```

第 10 行代码是 C# 6 的语法，相对于前面的 `string.Format` 函数的写法，更加简洁。

同样的道理，我们对 `RADtoDMS` 函数与 `RADtoString` 函数也应做相应的处理，其代码如下：

```

1 public static void RADtoDMS(double radAngle, out int d, out int m, out double s
  )
2 {
3     radAngle *= TOSECOND;
4     int angle = (int)Math.Round(radAngle);
5     d = angle / 3600;
6     angle -= d * 3600;
7     m = angle / 60;
8     s = radAngle - d * 3600 - m * 60;
9 }
10
11 public static double RADtoDMS(double radAngle)
12 {
13     RADtoDMS(radAngle, out int d, out int m, out double s);
14     return (d + m / 100.0 + s / 10000.0);
15 }
16
17 public static string RADtoString(double radAngle)
18 {
19     RADtoDMS(radAngle, out int d, out int m, out double s);
20     return $"{d}°{m:00}'{s:00.0####}''";
21 }

```

对算法进行单元测试

`SMath` 项目中的各个函数是我们的基础测量算法函数，必须保证它们的正确性。为此对其编写单元测试函数进行严格的测试。如果以后需要对这些算法进行优化，执行这些单元测试函数也可确保优化后的算法正确性。

点击展开 `UnitTestSMath` 项目，鼠标右键点击原来的文件 `UnitTest1.cs`，在弹出的菜单中选择 `Rename` 命令，将其改名为 `UnitTestSMath.cs`，在弹出的确认对话框中选择按钮‘是 (Y)’就可以将其中的类名改为 `UnitTestSMath` 了。

我们首先编写 `DMStoDMS` 函数的测试函数，其代码如下：

```

1 //UnitTestSMath.cs 文件内容

```

```
2 using System;
3 using Microsoft.VisualStudio.TestTools.UnitTesting;
4
5 namespace UnitTestSMath
6 {
7     [TestClass]
8     public class UnitTestSMath
9     {
10         [TestMethod]
11         public void TestDMStoDMS()
12         {
13             ZXY.SMath.DMStoDMS(1.4, out int d, out int m, out double s);
14             Assert.AreEqual(1, d);
15             Assert.AreEqual(40, m);
16             Assert.AreEqual(0, s, 1e-8);
17
18             ZXY.SMath.DMStoDMS(-1.4, out d, out m, out s);
19             Assert.AreEqual(-1, d);
20             Assert.AreEqual(-40, m);
21             Assert.AreEqual(0, s, 1e-8);
22
23             ZXY.SMath.DMStoDMS(235.07492345, out d, out m, out s);
24             Assert.AreEqual(235, d);
25             Assert.AreEqual(7, m);
26             Assert.AreEqual(49.2345, s, 1e-8);
27
28             ZXY.SMath.DMStoDMS(-235.07492345, out d, out m, out s);
29             Assert.AreEqual(-235, d);
30             Assert.AreEqual(-7, m);
31             Assert.AreEqual(-49.2345, s, 1e-8);
32         }
33     }
34 }
```

测试的原理非常简单，那就是已知一个值，让它经过一系列的运算得到期望的值。如果算法结果与期望值相符，则测试通过，否则测试就不通过。

在 Visual Studio 系统中，单元测试的主要函数在 Assert 类中，其静态重载函数 AreEqual 用于判断期望值 (expected) 与实际运算值 (actual) 是否相等。对于上述代码中 d 与 m，由于是整数，可以直接判断是否相等，而 s 是 double 类型，需给出一定的容错范围，不能直接判断是否相等。

在测试代码中，我们分别用 $1^{\circ}40'$ 、 $-1^{\circ}40'$ 、 $235^{\circ}07'49.2345''$ 、 $-235^{\circ}07'49.2345''$ 进行测试，确保秒后第 8 位小数的准确性。

依次点击菜单栏上的 Test -> Run -> All Tests 菜单项，在左侧的侧栏 Test Explorer 中就可以显示测试结果，如图3.1所示，图中左侧 Test Explorer 中测试项显示为绿色，表明测试通过。

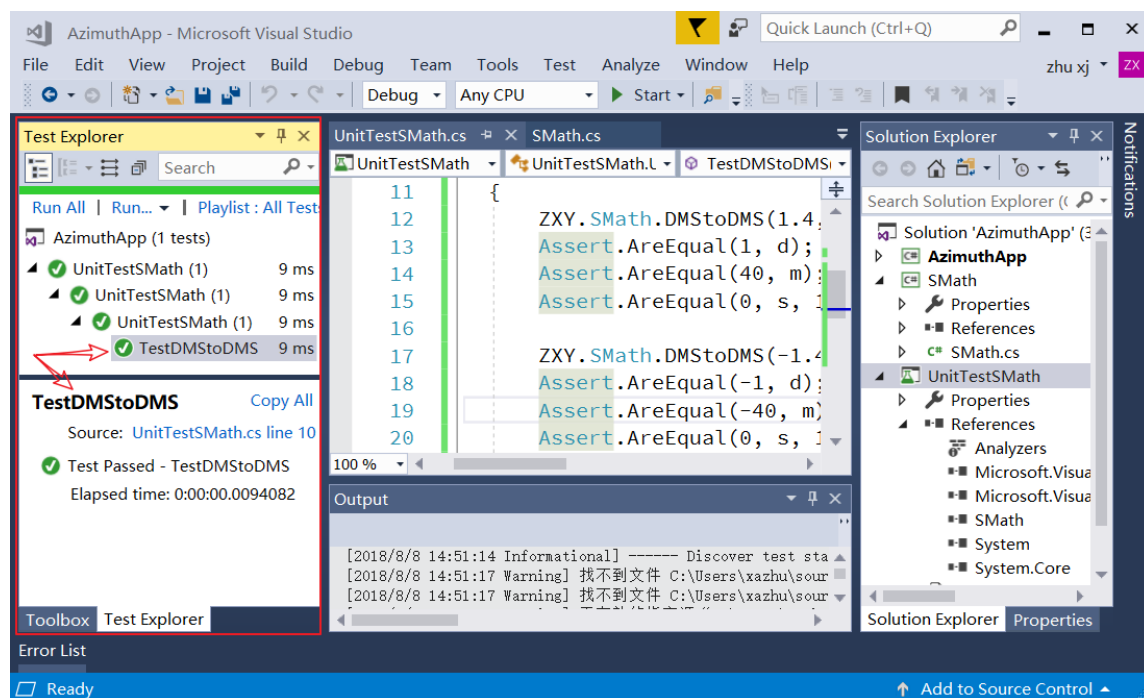


图 3.1 执行单元测试成功的示意图

如果我们将第 30 行代码中的-7 改为-8, 再执行测试命令, 执行结果如图3.2, 图中左侧 Test Explorer 中测试项显示为红色的“×”, 表明相应测试项未通过。

在 Test Explorer 下部显示了未通过测试的函数名称, 位于源代码的哪一行, 期望值是-8, 而实际值为-7。我们就可以根据这些信息去修正未通过测试的函数。

由图3.1与图3.2可看出, 测试通过, 相应函数测试项显示为绿色, 测试项中之一不能通过, 则相应函数测试项显示为红色。这就免去了我们人工再去比对的环节, 也利于机器的自动测试实施。

作业

1. 以上 DMStoString 函数与 RADtoString 函数可以正确的将角度值中的度分秒值提取出来并以更好阅读的字符串 $23^{\circ}05'47.6324''$ 形式表达。但对于负的角度会表达为 $-23^{\circ}-05'-47.6324''$ 形式, 对于追求完美的某些程序员来讲这似乎有些不可接受, 他们仍然想将负的角度表达为 $-23^{\circ}05'47.6324''$ 或 $-0^{\circ}05'47.6324''$ 形式。请试着改进以上的 DMStoString 与 RADtoString 函数。
2. 请用 Math.Atan 函数编写两点坐标反算坐标方位角的函数 Azimuth。
3. 请将单元测试项目 UnitTestSMath 中的测试函数 TestAzimuth 与 TestRADtoDMS 等补充编写完整。

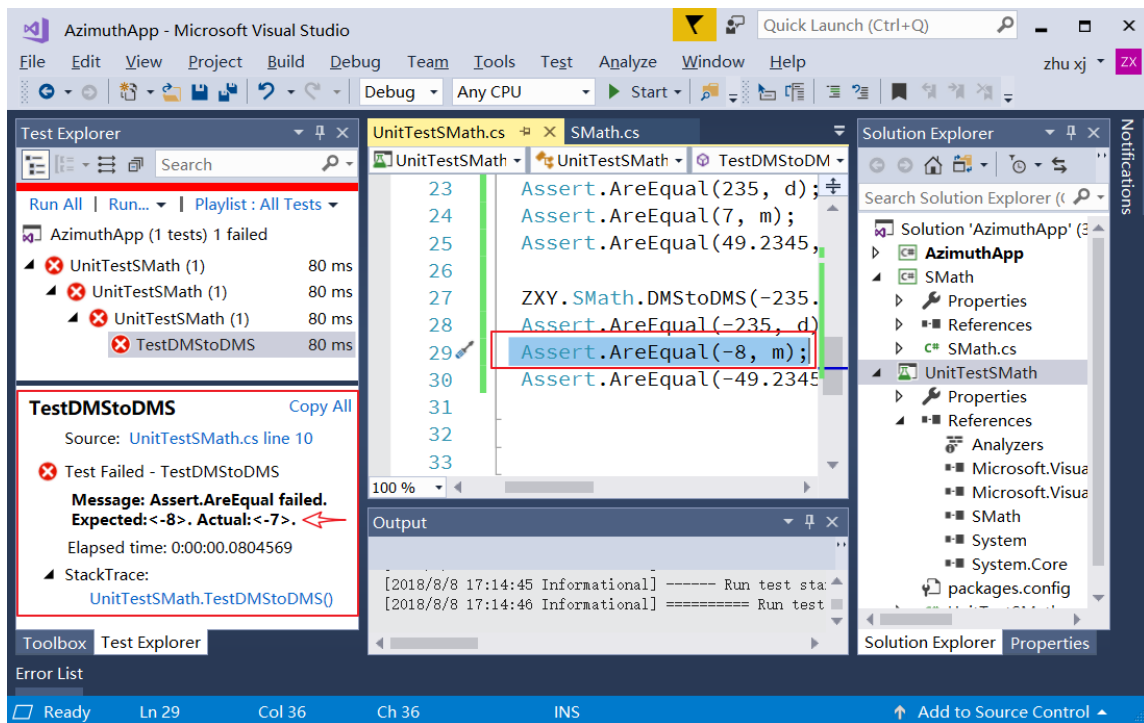


图 3.2 执行单元测试失败的示意图

第 4 章 C# 面向对象程序设计

类 (class) 是面向对象程序设计的基础与核心，封装、继承、多态 (polymorphism) 是面向对象程序设计的三大特征。

4.1 封装

封装把客观事物封装成抽象的类，在类中把自己的数据和方法只让可信的类或者对象操作，对不可信的类或对象进行信息隐藏。

也就是应尽可能的把类成员变量的访问权限定为 private 或 protected，通过属性 (Property) 或方法向外开放；类的方法/函数应尽可能的定义为 public；需向下继承的类成员，尽可能可以定义为 protected；项目内能访问的成员，尽可能定义为 internal。

在 C# 中类支持单继承 (inheritance)，通过类继承产生派生类 (derived class)，从而扩展和专用化基类 (base class)。

C# 中所有的类均从 System.Object 继承，如果一个类只从 System.Object 类继承，则可以将其省略。

继承 (Inheritance) 是指这样一种能力：它可以使用现有类的所有功能，并在无需重新编写原来的类的情况下对这些功能进行扩展。

继承符合开闭原则。

通过继承创建的新类称为“子类”或“派生类”，被继承的类称为“基类”、“父类”或“超类”。继承的过程，是从一般到特殊的过程。

多态 (polymorphism) 是指将子对象赋值给父对象，父对象就可以根据当前赋值给它的子对象的特性以不同的方式运作。

4.2 继承 (Inheritance)

继承是指这样一种能力：它可以使用现有类的所有功能，并在无需重新编写原来的类的情况下对这些功能进行扩展。

通过继承创建的新类称为“子类”或“派生类”。

被继承的类称为“基类” (base class)、“父类”或“超类” (super class)。

继承的过程，就是从一般到特殊的过程，也称为泛化。

在考虑使用继承时，有一点需要注意，那就是两个类之间的关系应该是“属于”关系 (is a)。

4.2.1 使用继承的目的

复用代码，减少类的冗余代码，减少开发工作量（符合开闭原则：对扩展开放，对修改关闭，简称为开闭原则）。

使得类与类之间产生关系，为多态的实现打下基础。

继承概念的实现方式有二种：类继承与接口继承（或称为接口实现）

4.2.2 C# 类继承

指使用基类的属性和方法而无需额外编码的能力，如：子窗体（类）使用基窗体（类）的外观和实现代码的能力。如在 WinForm 项目中，使用 System.Windows.Forms 生成我们自己的 Form1：

```
1 public partial class Form1 : System.Windows.Forms.Form
2 {
3     .....
4 }
```

如在 WPF 项目中，使用 System.Windows.Forms 生成我们自己的 WPF 窗体 MainWindow：

```
1 public partial class MainWindow : System.Windows.Window
2 {
3     .....
4 }
```

如此，微软构造了各种类库，我们只需简单的继承，就可以使用微软已经定义好的类库。

C# 中类的继承为单继承，不支持类的多继承。

C# 中类的继承形式为：派生类：基类

4.2.3 继承的写法

在类名后面添加一个冒号和基类的名称来指定一个基类，省略基类的类默认为从类 System.Object 派生。下面的代码：

```
1 class A : System.Object
2 {
3 }
```

可以写成：

```
1 class A
2 {
3 }
```

若 A 为基类，B 为派生类/子类，则写成：

```
1 class A {}
2
3 class B : A
```

```
4 {  
5 }
```

C# 中类的继承只支持单继承。

4.2.4 子类继承基类的成员

派生类继承了基类的所有成员，继承意味着派生类隐式地将它的基类的所有成员当作自己的成员，无论基类中的成员权限是 `private`，还是 `protected` 或 `public`。基类中的 `private` 不能直接访问，但可以通过 `protected` 或 `public` 方法间接访问。

```
1 class A  
2 {  
3     private int a1=1;  
4     protected int a2=2;  
5     public int a3=3;  
6  
7     public int Sum()  
8     {  
9         return a1+a2+a3;  
10    }  
11 }  
12  
13 class B : A  
14 {  
15     private int b1=10;  
16  
17     public int Sum1()  
18     {  
19         return a1 + a2 + a3 + b1;  
20     }  
21  
22     public int Sum2()  
23     {  
24         return Sum() + b1;  
25     }  
26 }
```

派生类 B 中的方法 Sum1 中的第 9 行会报错，因为 a1 为基类 A 中的 `private` 字段，不能直接使用。

派生类 B 中的方法 Sum2 通过基类 A 中 `public` 方法 Sum 调用了基类 A 的 `private` 字段 a1，不会报错。

4.2.5 派生类对基类方法的隐藏：使用关键字 new

如果将上面代码中的派生类 B 中的求和方法也定义为 Sum，则派生类 B 中的方法 Sum 隐藏了基类 A 中的方法 Sum，编译器会给出警告提示，如下面代码所示：

```
1 class A
2 {
3     private int a1=1;
4     protected int a2=2;
5     public int a3=3;
6
7     public int Sum()
8     {
9         return a1+a2+a3;
10    }
11 }
12
13 class B : A
14 {
15     private int b1=10;
16
17     public new int Sum()
18     {
19         return base.Sum() + b1;
20     }
21 }
```

为消除警告，需在 17 行添加 new 关键字，告诉编译器这是派生类 B 中的新方法，在 19 行为了引用基类中的 Sum 方法，需使用 base 关键字指明，否则会形成循环调用。

4.2.6 继承中的构造函数

- 基类与派生类的构造函数执行顺序

派生类中的一部分是来自基类中的，在构造派生类时，必须先构造基类，因此基类的构造函数肯定是先于派生类的构造函数执行的，可以通过如下代码验证：

```
1 class A
2 {
3     public A()
4     {
5         Console.WriteLine("A: A()");
6     }
7 }
8
9 class B : A
10 {
11     public B()
12     {
13         Console.WriteLine("B: B()");
14     }
15 }
```

如果在 Main 函数中运行如下代码：

```
B objB = new B();
```

则输出为：

```
A: A()
```

```
B: B()
```

验证了基类的构造函数会先于子类的构造函数执行的。

- 派生类调用基类的非默认构造函数的形式

以上在 new B() 时，系统会先调用默认的 A 类的构造函数 A()。如下代码所示，如果 A 类中不存在默认构造函数 A() 时，系统就会报错。

此时需在派生类 B 中的构造函数中用关键字 base 调用基类 A 的构造函数，如下代码第 24 行所示：

```
1 class A
2 {
3     private int a1;
4     protected int a2;
5     public int a3;
6
7     public A(int a1, int a2, int a3)
8     {
9         this.a1 = a1;
10        this.a2 = a2;
11        this.a3 = a3;
12    }
13
14    public int Sum()
15    {
16        return a1+a2+a3;
17    }
18 }
19
20 class B : A
21 {
22     private int b1;
23
24     public B(int a1, int a2, int a3, int b1) : base(a1, a2, a3)
25     {
26         this.b1 = b1;
27     }
28
29     public new int Sum()
30     {
```

```
31         return base.Sum() + b1;
32     }
33 }
```

4.2.7 小结

基类是不能直接调用子类的方法或函数，子类可以调用继承自基类的 `protected` 和 `public` 字段、属性与方法；

子类中的构造函数对基类构造函数的引用形式为 `base(..., ...)`；

如果在子类中要调用基类中的属性或方法时，用关键字 `base`；

对子类自己的调用用关键字 `this`。

4.3 多态与虚函数

在上节末尾的继承示例中，基类的变量可以指向派生类的实例，这是继承的一个重要概念，如下代码所示：

```
A a = new B(1, 2, 3, 10);
```

这与以前的知识：

```
A a = new A(1, 2, 3); 或 B b = new B(1, 2, 3, 10);
```

不同。在此情况下，如果我们执行 `a.Sum()` 语句，大家猜想其值为 6 还是 16？

这里的语境含义应该是：基类 A 的变量 a 指向基类 A 的实例时，希望执行的是基类 A 中的 `Sum()`，其值应为 6；基类 A 的变量 a 指向派生类 B 的实例时，希望执行的是派生类 B 中的 `Sum()`，其值应为 16，但上述语句执行的结果却是 6，也就是执行的是基类 A 中的 `Sum()` 函数，显然不符合期望。

所谓的多态 (polymorphism)，就是同样的消息 `a.Sum()`，所指向的实例不一样，则呈现出不同的行为姿态，而且这是在运行时呈现的。如此，在编程语言中，多态 (polymorphism) 就可以为不同数据类型的实体提供统一的接口，从而简化编码。

如何实现这种多态行为呢？在面向对象程序设计语言中的基本处理方法是将基类中需呈现多态的方法定义为虚函数 (virtual)，派生类中需表现出不同行为时，将其覆盖 (override)。如下代码所示：

```
1 class A
2 {
3     private int a1;
4     protected int a2;
5     public int a3;
6
7     public A(int a1, int a2, int a3)
8     {
9         this.a1 = a1;
10        this.a2 = a2;
11        this.a3 = a3;
```

```

12     }
13
14     public virtual int Sum()
15     {
16         return a1+a2+a3;
17     }
18 }
19
20 class B : A
21 {
22     private int b1;
23
24     public B(int a1, int a2, int a3, int b1) : base(a1, a2, a3)
25     {
26         this.b1 = b1;
27     }
28
29     public override int Sum()
30     {
31         return base.Sum() + b1;
32     }
33 }
34
35 //在Main()函数中运行代码，s的值为16，即执行派生类 B 中的 Sum() 方法
36 A a = new B(1, 2, 3, 10);
37 int s = a.Sum();

```

以上代码第 14 行需用关键字 virtual 定义，第 29 行需用关键字 override 进行覆盖。

为了更好地理解多态，下面我们以绘图程序中的几何图形为例进行演示虚函数实现多态的方法。设想有一绘图程序能够绘制点 Point、圆 Circle、多段线 Polyline、多边形 Polygon 等图形并能够计算图形的面积 Area 与长度 Length。面向对象的类图如 4.1 所示：

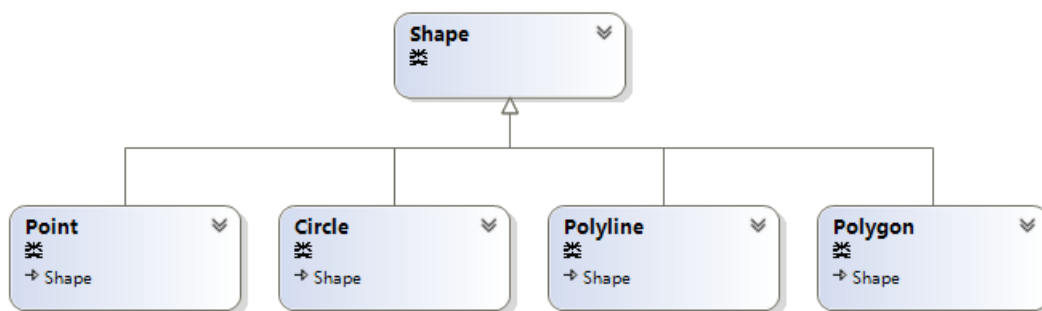


图 4.1 几何图形关系类图

请注意，类图4.1 中所有的类均用实线矩形表示。

我们首先将所有图形对象的公有属性面积 Area、长度 Length 和计算 Calculate、绘图 Draw 方法抽象到基类 Shape 中，代码如下：

```
1 public class Shape
2 {
3     protected double length = 0.0;
4     public double Length { get => length; }
5
6     protected double area = 0.0;
7     public double Area { get => area; }
8
9     public virtual void Calculate() { }
10    public virtual void Draw()
11    {
12        Console.WriteLine("Shape Draw .....");
13    }
14 }
```

由于各种图形计算长度与面积的方法各不相同，所以将 Calculate() 方法与 Draw() 方法定义为虚函数 virtual()，Calculate() 方法采用默认的空函数体方式实现；length 与 area 用于存储各种图形的计算值，要求外部对象不能访问与修改，定义为受保护权限 (protected)，通过只读属性向外部展现数据。

点类 Point 的实现如下：

```
1 public class Point : Shape
2 {
3     public double X { get; set; }
4     public double Y { get; set; }
5
6     public Point(double x, double y)
7     {
8         this.X = x;
9         this.Y = y;
10    }
11
12    public override void Draw()
13    {
14        Console.WriteLine("Point Draw .....");
15    }
16 }
```

点 Point 的面积与长度我们均默认为 0，Shape 类中的计算功能就够了，此处就不在覆盖 (override)；绘图需要修改为 “Point Draw ... ” 形式，故需覆盖 (override)。

圆类 Circle 的实现如下：

```
1 public class Circle : Shape
2 {
3     public Point Center { get; set; }
4
5     private double r;
```

```
6     public double R
7     {
8         get { return r; }
9         set
10        {
11            if(value != r && value >= 0)
12            {
13                r = value;
14                Calculate(); //r的值发生改变, 重新计算圆的面积
15            }
16        }
17    }
18
19
20    public Circle(double x, double y, double r)
21    {
22        Center = new Point(x, y);
23        this.R = r; //赋值给R而不是this.r, 确保计算圆的面积
24    }
25
26    public override void Calculate()
27    {
28        this.length = Math.PI * r * 2;
29        this.area = Math.PI * r * r;
30    }
31
32
33    public override void Draw()
34    {
35        Console.WriteLine("Circle Draw .....");
36    }
37 }
```

由于圆类的面积与周长计算方法与基类 Shape 不同, 需将 Calculate() 与 Draw() 方法覆盖 (override)。

多段线类 Polyline 的实现如下:

```
1 public class Polyline : Shape
2 {
3     private List<Point> points = new List<Point>();
4
5     private double CalLength()
6     {
7         if (points.Count < 2) return 0;
8         double sum = 0;
9         for (int i = 0; i < points.Count - 1; i++)
10        {
```

```
11         double dx = points[i + 1].X - points[i].X;
12         double dy = points[i + 1].Y - points[i].Y;
13         sum += Math.Sqrt(dx * dx + dy * dy);
14     }
15     return sum;
16 }
17
18 public void Add(double x, double y)
19 {
20     points.Add(new Point(x, y));
21 }
22
23 public Polyline()
24 {
25 }
26
27 public override void Calculate()
28 {
29     this.length = CallLength();
30     this.area = 0.0;
31 }
32
33
34 public override void Draw()
35 {
36     Console.WriteLine("Polyline Draw .....");
37 }
38 }
```

由于多段线类的面积与周长计算方法与基类 Shape 不同，需将 Calculate() 与 Draw() 方法覆盖 (override)。

多边形类 Polygon 的实现如下：

```
1 public class Polygon : Shape
2 {
3     private List<Point> points = new List<Point>();
4
5     public double CallLength()
6     {
7         if (points.Count < 2) return 0;
8         double sum = 0;
9         for (int i = 0; i < points.Count; i++)
10         {
11             int j = (i + 1) % points.Count;
12             double dx = points[j].X - points[i].X;
13             double dy = points[j].Y - points[i].Y;
14             sum += Math.Sqrt(dx * dx + dy * dy);
15         }
16     }
17 }
```

```
15     }
16     return sum;
17 }
18
19 public double CalArea()
20 {
21     if (points.Count < 3) return 0;
22     double s = 0;
23     for (int i = 0; i < points.Count; i++)
24     {
25         int j = i + 1;
26         if (j == points.Count) j = 0;
27         s += points[i].X * points[j].Y - points[j].X * points[i].Y;
28     }
29     return 0.5 * Math.Abs(s);
30 }
31
32 public void Add(double x, double y)
33 {
34     points.Add(new Point(x, y));
35 }
36
37 public override void Calculate()
38 {
39     this.length = CalLength();
40     this.area = CalArea();
41 }
42
43
44 public override void Draw()
45 {
46     Console.WriteLine("Polygon Draw .....");
47 }
48 }
```

由于多边形类的面积与周长计算方法与基类 Shape 不同，需将 Calculate() 与 Draw() 方法覆盖 (override)。

客户端调用的代码如下：

```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         Shape s = new Shape();
6         s.Calculate();
7         Console.WriteLine("Shape's Area={0}, Length={1}", s.Area, s.Length);
8         s.Draw();
9     }
10 }
```

```
9
10     Point p = new Point(100, 100);
11     p.Calculate();
12     Console.WriteLine("Point's Area={0}, Length={1}", p.Area, p.Length);
13     p.Draw();
14
15     Circle c = new Circle(100, 100, 80);
16     c.Calculate();
17     Console.WriteLine("Circle1's Area={0}, Length={1}", c.Area, c.Length);
18     c.Draw();
19
20     Polyline pl = new Polyline();
21     pl.Add(-1, 0);
22     pl.Add(2, 3);
23     pl.Add(4, 2);
24     pl.Add(4, 4);
25     pl.Add(6, 8);
26     pl.Add(-2, 5);
27     pl.Calculate();
28     Console.WriteLine("Polyline's Area={0}, Length={1}", pl.Area, pl.Length
29 );
30     pl.Draw();
31
32     Polygon pg = new Polygon();
33     pg.Add(-1, 0);
34     pg.Add(2, 3);
35     pg.Add(4, 2);
36     pg.Add(4, 4);
37     pg.Add(6, 8);
38     pg.Add(-2, 5);
39     pg.Calculate();
40     Console.WriteLine("Polygon's Area={0}, Length={1}", pg.Area, pg.Length)
41 ;
42     pg.Draw();
43
44     Console.ReadKey();
45 }
```

代码运行输出为:

Shape's Area=0, Length=0

Shape Draw

Point's Area=0, Length=0

Point Draw

Circle1's Area=20106.1929829747, Length=502.654824574367

Circle Draw

Polyline's Area=0, Length=21.4948483649362

Polyline Draw

Polygon's Area=28, Length=26.593867878529

Polygon Draw

很明显，客户端的调用代码有问题，如果增加一个图形实例的，调用代码就需要增加，代码会无限膨胀，对于绘图程序，也无法统一在屏幕上进行绘制与计算。更加合理的客户端调用的代码如下所示：

```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         List<Shape> shapeList = new List<Shape>();
6         shapeList.Add(new Shape());
7         shapeList.Add(new Point(100, 100));
8         shapeList.Add(new Circle(100, 100, 80));
9
10        Polyline pl = new Polyline();
11        pl.Add(-1, 0);
12        pl.Add(2, 3);
13        pl.Add(4, 2);
14        pl.Add(4, 4);
15        pl.Add(6, 8);
16        pl.Add(-2, 5);
17        shapeList.Add(pl);
18
19        Polygon pg = new Polygon();
20        pg.Add(-1, 0);
21        pg.Add(2, 3);
22        pg.Add(4, 2);
23        pg.Add(4, 4);
24        pg.Add(6, 8);
25        pg.Add(-2, 5);
26        shapeList.Add(pg);
27
28        CalculateAll(shapeList);
29        DrawAll(shapeList);
30
31
32        Console.ReadKey();
33    }
34
35    static void CalculateAll(List<Shape> shapeList)
36    {
```

```
37         foreach (var item in shapeList)
38         {
39             item.Calculate();
40             Console.WriteLine("Area={0}, Length={1}", item.Area, item.Length);
41         }
42     }
43
44     static void DrawAll(List<Shape> shapeList)
45     {
46         foreach (var item in shapeList)
47         {
48             item.Draw();
49         }
50     }
51 }
```

代码运行输出为：

```
Area=0, Length=0
Area=0, Length=0
Area=20106.1929829747, Length=502.654824574367
Area=0, Length=21.4948483649362
Area=28, Length=26.593867878529
Shape Draw .....
Point Draw .....
Circle Draw .....
Polyline Draw .....
Polygon Draw .....
```

这样写的好处是 28-50 行的代码基本稳定，图形实例的增加不会再去修改这部分代码。

可能有人认为这次的面积与长度输出看不到图形信息，其实对各个类的 ToString 函数覆盖 (override) 就可以达到与前面一样的效果，代码如下：

```
1 public class Shape
2 {
3     .....
4
5     public override string ToString()
6     {
7         return $"Shape's Area ={Area}, Length ={Length}";
8     }
9 }
10
11 public class Point : Shape
12 {
13     .....
```

```
14
15     public override string ToString()
16     {
17         return $"Point's Area ={Area}, Length ={Length}";
18     }
19 }
20
21 public class Circle : Shape
22 {
23     .....
24
25     public override string ToString()
26     {
27         return $"Circle's Area ={Area}, Length ={Length}";
28     }
29 }
30
31 public class Polyline : Shape
32 {
33     .....
34
35     public override string ToString()
36     {
37         return $"Polyline's Area ={Area}, Length ={Length}";
38     }
39 }
40
41 public class Polygon : Shape
42 {
43     .....
44
45     public override string ToString()
46     {
47         return $"Polygon's Area ={Area}, Length ={Length}";
48     }
49 }
```

ToString() 函数来自 System.Object 类, System.Object 类是所有的类的根, ToString() 函数被定义为 virtual。

CalculateAll 函数稍作修改:

```
1 static void CalculateAll(List<Shape> shapeList)
2 {
3     foreach (var item in shapeList)
4     {
5         item.Calculate();
6         Console.WriteLine(item);
7     }
8 }
```

```

7     }
8 }

```

则运行输出结果为：

```

Shape's Area =0, Length =0
Point's Area =0, Length =0
Circle's Area =20106.1929829747, Length =502.654824574367
Polyline's Area =0, Length =21.4948483649362
Polygon's Area =28, Length =26.593867878529
Shape Draw .....
Point Draw .....
Circle Draw .....
Polyline Draw .....
Polygon Draw .....

```

4.4 抽象方法与抽象类

在上节的内容中，Shape 类的虚函数 Calculate 函数体为空，什么内容都不做；在实际的绘图成图中，Draw 函数也一样。因此可以用关键字 abstract 将其定义为抽象方法，就可以不定义函数体了。

含有一个抽象方法的类就是抽象类，必须在关键字 class 前用 abstract 加以定义，抽象类 Shape 代码如下所示：

```

1 public abstract class Shape
2 {
3     protected double length = 0.0;
4     public double Length { get => length; }
5
6     protected double area = 0.0;
7     public double Area { get => area; }
8
9     public abstract void Calculate();
10    public abstract void Draw();
11 }

```

抽象类不同于普通实例类的特点是不能实例化，不能写成：`Shape s = new Shape()`，但可以写成 `Shape s = new Point()` 形式。

抽象方法不同于虚函数的地方在于派生类中必须覆盖 (override) 抽象方法，而虚函数则没有这样强制性的要求。因此前面的 Point 类由于没有覆盖 (override) 基类 Shape 中的抽象方法 Calculate，需做如下修改：

```

1 public sealed class Point : Shape
2 {
3     .....

```

```

4
5     public override void Draw()
6     {
7         Console.WriteLine("Point Draw .....");
8     }
9
10    .....
11 }

```

在类图中，抽象类是用虚线矩形表示的，如图 4.2 所示：

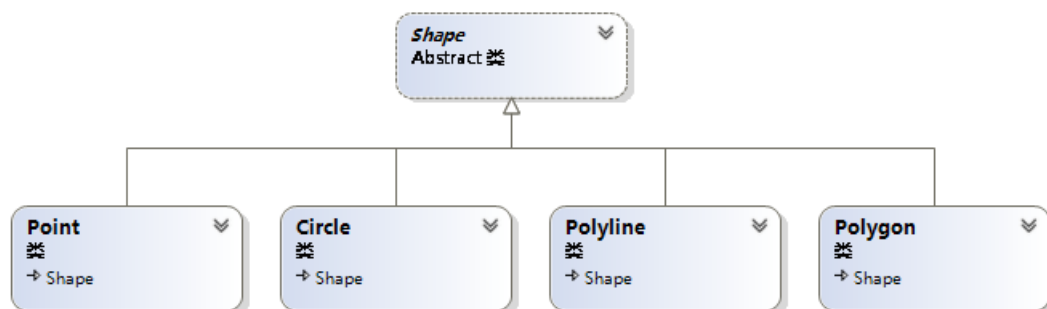


图 4.2 几何图形关系类图的抽象类

由于抽象类 Shape 不能实例化，前面的调用代码中生成 Shape 实例的代码需删除，如下代码所示：

```

1 class Program
2 {
3     static void Main(string[] args)
4     {
5         List<Shape> shapeList = new List<Shape>();
6         //shapeList.Add(new Shape()); //需删除
7         shapeList.Add(new Point(100, 100));
8         shapeList.Add(new Circle(100, 100, 80));
9
10        .....
11    }
12
13    .....
14 }

```

4.5 接口 (interface)

接口是指公开约定的属性或方法，接口是面向对象编程方法的重要概念。

接口的关键字是 interface，与抽象类不同的是接口中只能含有方法与属性，所有的方法与属性均不能有具体的实现。

如图 4.3 所示，我们设计图形接口 IShape 与绘制接口 ICalculate：

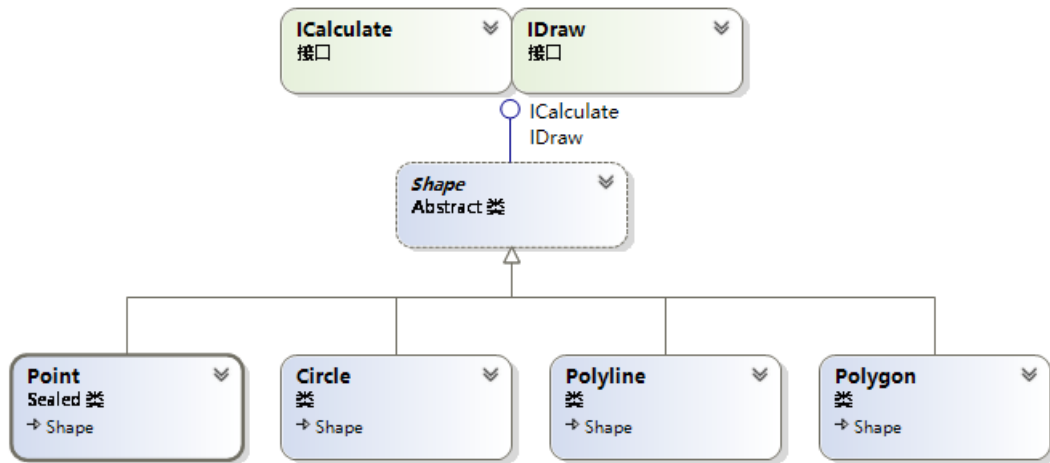


图 4.3 带有接口的几何图形关系类图

```

1 public interface ICalculate
2 {
3     void Calculate();
4 }
5
6 public interface IDraw
7 {
8     void Draw();
9 }

```

让抽象类 Shape 来实现这两个接口：

```

1 public abstract class Shape : ICalculate, IDraw
2 {
3     protected double length = 0.0;
4     public double Length { get => length; }
5
6     protected double area = 0.0;
7     public double Area { get => area; }
8
9     public abstract void Calculate();
10    public abstract void Draw();
11 }

```

实现接口也可以用一般的实例类，也可以用一般的方法，也可以用虚函数，也可以抽象方法，只要方法的签名与接口相符就行。

在我们的这个例子中，我们使用抽象类的抽象函数进行接口实现。

4.5.1 面向接口编程

接口是框架系统设计的关键技术，众多的商业软件为了易于扩展与修改，也为了多人协调开发，均是以架构形式出现，系统架构师的主要设计手段就是面向接口进行设计与编码。有兴趣的同学可以研读设计模式等相关资料。

在我们的例子中，如下代码中的 CalculateAll 与 DrawAll 函数分别是面向接口 ICalculate 与 IDraw 接口编写的：

```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         List<Shape> shapeList = new List<Shape>();
6         shapeList.Add(new Point(100, 100));
7         shapeList.Add(new Circle(100, 100, 80));
8
9         Polyline pl = new Polyline();
10        pl.Add(-1, 0);
11        pl.Add(2, 3);
12        pl.Add(4, 2);
13        pl.Add(4, 4);
14        pl.Add(6, 8);
15        pl.Add(-2, 5);
16        shapeList.Add(pl);
17
18        Polygon pg = new Polygon();
19        pg.Add(-1, 0);
20        pg.Add(2, 3);
21        pg.Add(4, 2);
22        pg.Add(4, 4);
23        pg.Add(6, 8);
24        pg.Add(-2, 5);
25        shapeList.Add(pg);
26
27        List<IDraw> drawList = new List<IDraw>();
28        foreach (var item in shapeList)
29        {
30            IDraw idraw = item;
31            drawList.Add(idraw);
32        }
33        DrawAll(drawList);
34
35        List<ICalculate> calculateList = new List<ICalculate>();
36        foreach (var item in calculateList)
37        {
38            ICalculate icalculate = item as ICalculate;
```

```
39         calculateList.Add(icalculate);
40     }
41     CalculateAll(calculateList);
42
43     foreach (var item in shapeList)
44     {
45         Console.WriteLine(item);
46     }
47
48     Console.ReadKey();
49 }
50
51 static void CalculateAll(List<ICalculate> calculateList)
52 {
53     foreach (var item in calculateList)
54     {
55         item.Calculate();
56     }
57 }
58
59 static void DrawAll(List<IDraw> shapeList)
60 {
61     foreach (var item in shapeList)
62     {
63         item.Draw();
64     }
65 }
66 }
```

代码中第 27-32 行是用接口 IDraw 指向各个 Shape 实例，以满足 DrawAll 的接口调用要求。

4.5.2 接口跳转

在某些软件设计中，有将某个类设计的功能特别多的情况，就可以用接口将这些功能分组，在使用的时候，某个接口功能使用完毕，就可以进行接口跳转，再去使用其他功能。

在上面的代码第 38 行：**ICalculate icalculate = item as ICalculate;**就是从接口 IDraw 跳转到 ICalculate。

这种跳转之所以能成功，是因为这些接口所指向的实例对象均实现了这些接口，否则，这种跳转将失败。

4.6 封闭类 (sealed class)

如果一个类不需要再通过继承扩展，就可以用关键字 sealed 将其定义为封闭类，比如点 Point 类：

```
1 public sealed class Point : Shape
2 {
3     public double X { get; set; }
4     public double Y { get; set; }
5
6     public Point(double x, double y)
7     {
8         this.X = x;
9         this.Y = y;
10    }
11
12    public override void Calculate()
13    {
14        this.area = 0.0;
15        this.length = 0.0;
16    }
17
18    public override void Draw()
19    {
20        Console.WriteLine("Point Draw .....");
21    }
22
23    public override string ToString()
24    {
25        return $"Point's Area ={Area}, Length ={Length}";
26    }
27 }
```

4.7 扩展阅读：面向对象程序设计原则

面向对象程序设计的核心是类与接口，设计良好的程序需要遵循类的七个如下设计原则：

- 开闭原则（OCP）
- 里氏代换原则
- 迪米特原则（最少知道原则）
- 单一职责原则
- 接口分隔原则
- 依赖倒置原则
- 组合/聚合复用原则

七大原则之间并不是相互孤立的，彼此之间是存在着一定关联，一个可以是另一个原则的加强或基础。违反其中的某一个原则，可能同时也会违反其余的原则。

开闭原则是面向对象的可复用设计的基石。其他设计原则是实现开闭原则的手段和工具。

一般地，可以把这七个原则分成了以下两个部分：

- 设计目标开闭原则、里氏代换原则、迪米特原则
- 设计方法单一职责原则、接口分隔原则、依赖倒置原则、组合/聚合复用原则

4.7.1 开闭原则 (OCP)

思想

软件实体（模块，类，方法等）应该对扩展开放，对修改关闭。

开闭原则英文为 The Open-Closed Principle，简称为 OCP，是指在进行面向对象设计中，设计类或其他程序单位时，应该遵循：

- 对扩展开放（open）

某模块的功能是可扩展的，则该模块是扩展开放的。软件系统的功能上的可扩展性要求模块是扩展开放的。

- 对修改关闭（closed）的设计原则

某模块被其他模块调用，如果该模块的源代码不允许修改，则该模块修改关闭的。软件系统的功能上的稳定性，持续性要求模块是修改关闭的。

开闭原则是判断面向对象设计是否正确的最基本的原理之一。

根据开闭原则，在设计一个软件系统模块（类，方法）的时候，应该可以在不修改原有的模块（修改关闭）的基础上，能扩展其功能（扩展开放）。

原因

- 稳定性

开闭原则要求扩展功能不修改原来的代码，这可以让软件系统在变化中保持稳定。

- 扩展性

开闭原则要求对扩展开放，通过扩展提供新的或改变原有的功能，让软件系统具有灵活的可扩展性。遵循开闭原则的系统设计，可以让软件系统可复用，并且易于维护。

实现方法

为了满足开闭原则的对修改关闭原则以及扩展开放原则，应该对软件系统中的不变的部分加以抽象，在面向对象的设计中，

可以把这些不变的部分加以抽象成不变的接口，这些不变的接口可以应对未来的扩展；

接口的最小功能设计原则。根据这个原则，原有的接口要么可以应对未来的扩展；不足的部分可以通过定义新的接口来实现；

模块之间的调用通过抽象接口进行，这样即使实现层发生变化，也无需修改调用方的代码。

接口可以被复用，但接口的实现却不一定能被复用。

接口是稳定的，关闭的，但接口的实现是可变的，开放的。

可以通过对接口的不同实现以及类的继承行为等为系统增加新的或改变系统原来的功能，实现软件系统的柔性扩展。

好处：提高系统的可复用性和可维护性。

简单地说，软件系统是否有良好的接口（抽象）设计是判断软件系统是否满足开闭原则的一种重要的判断基准。现在多把开闭原则等同于面向接口的软件设计。

代码示例

4.7.2 里氏替换原则 (Liskov Substitution Principle , LSP)

所有引用基类的地方必须能透明地使用其派生类的对象。

也就是说，只有满足以下 2 个条件的 OO 设计才可被认为是满足了 LSP 原则：

不应该在代码中出现 if/else 之类对派生类类型进行判断的条件。

派生类应当可以替换基类并出现在基类能够出现的任何地方，或者说如果我们把代码中使用基类的地方用它的派生类所代替，代码还能正常工作。

以下代码就违反了 LSP 定义。

里氏替换原则 (LSP) 是使代码符合开闭原则的一个重要保证。

同时 LSP 体现了：

类的继承原则：如果一个派生类的对象可能会在基类出现的地方出现运行错误，则该派生类不应该从该基类继承，或者说，应该重新设计它们之间的关系。

动作正确性保证：从另一个侧面上保证了符合 LSP 设计原则的类的扩展不会给已有的系统引入新的错误。示例：

里氏替换原则为我们是否应该使用继承提供了判断的依据，不再是简单地根据两者之间是否有相同之处来说使用继承。

里氏替换原则的引申意义：子类可以扩展父类的功能，但不能改变父类原有的功能。

具体来说：

子类可以实现父类的抽象方法，但不能覆盖父类的非抽象方法。子类中可以增加自己特有的方法。当子类的方法重载父类的方法时，方法的前置条件（即方法的输入/入参）要比父类方法的输入参数更宽松。当子类的方法实现父类的方法时（重载/重写或实现抽象方法）的后置条件（即方法的输出/返回值）要比父类更严格或相等。

4.7.3 迪米特原则（最少知道原则）(Law of Demeter , LoD)

迪米特原则 (Law of Demeter) 又叫最少知道原则 (Least Knowledge Principle)，可以简单说成：talk only to your immediate friends，只与你直接的朋友们通信，不要跟“陌生人”说话。

对于面向 OOD 来说，又被解释为下面两种方式：

- 1) 一个软件实体应当尽可能少地与其他实体发生相互作用。
- 2) 每一个软件单位对其他的单位都只有最少的知识，而且局限于那些与本单位密切相关的软件单位。

朋友圈的确定“朋友”条件：

当前对象本身 (this) 以参量形式传入到当前对象方法中的对象当前对象的实例变量直接引用的对象当前对象的实例变量如果是一个聚集，那么聚集中的元素也都是朋友当前对象所创建的对象任何一个对象，如果满足上面的条件之一，就是当前对象的“朋友”，否则就是“陌生人”。

迪米特原则的优缺点

迪米特原则的初衷在于降低类之间的耦合。由于每个类尽量减少对其他类的依赖，因此，很容易使得系统的功能模块功能独立，相互之间不存在（或很少有）依赖关系。

迪米特原则不希望类直接建立直接的接触。如果真的有需要建立联系，也希望能通过它的友元类来转达。因此，应用迪米特原则有可能造成的一个后果就是：系统中存在大量的中介类，这些类之所以存在完全是为了传递类之间的相互调用关系，这在一定程度上增加了系统的复杂度。

例如，购房者要购买楼盘 A、B、C 中的楼，他不必直接到楼盘去买楼，而是可以通过一个售楼处去了解情况，这样就减少了购房者与楼盘之间的耦合，如图所示。

4.7.4 单一职责原则

永远不要让一个类存在多个改变的理由。

换句话说，如果一个类需要改变，改变它的理由永远只有一个。如果存在多个改变它的理由，就需要重新设计该类。

单一职责原则原则的核心含意是：只能让一个类/接口/方法有且仅有一个职责。

为什么一个类不能有多于一个以上的职责？如果一个类具有一个以上的职责，那么就会有多个不同的原因引起该类变化，而这种变化将影响到该类不同职责的使用者（不同用户）：

一方面，如果一个职责使用了外部类库，则使用另外一个职责的用户却也不得不包含这个未被使用的外部类库。另一方面，某个用户由于某个原因需要修改其中一个职责，另外一个职责的用户也将受到影响，他将不得不重新编译和配置。这违反了设计的开闭原则，也不是我们所期望的。职责的划分既然一个类不能有多个职责，那么怎么划分职责呢？

Robert.C Martin 给出了一个著名的定义：所谓一个类的一个职责是指引起该类变化的一个原因。

如果你能想到一个类存在多个使其改变的原因，那么这个类就存在多个职责。

4.7.5 接口分隔原则 (Interface Segregation Principle , ISP)

不能强迫用户去依赖那些他们不使用的接口。

换句话说，使用多个专门的接口比使用单一的总接口总要好。

它包含了 2 层意思：

接口的设计原则：接口的设计应该遵循最小接口原则，不要把用户不使用的方法塞进同一个接口里。如果一个接口的方法没有被使用到，则说明该接口过胖，应该将其分割成几个功能专一的接口。

接口的依赖（继承）原则：如果一个接口 a 继承另一个接口 b，则接口 a 相当于继承了接口 b 的方法，那么继承了接口 b 后的接口 a 也应该遵循上述原则：不应该包含用户不使用的方法。反之，则说明接口 a 被 b 给污染了，应该重新设计它们的关系。

如果用户被迫依赖他们不使用的接口，当接口发生改变时，他们也不得不跟着改变。换言之，一个用户依赖了未使用但被其他用户使用的接口，当其他用户修改该接口时，依赖该接口的所有用户都将受到影响。这显然违反了开闭原则，也不是我们所期望的。

总而言之，接口分隔原则指导我们：

一个类对一个类的依赖应该建立在最小的接口上

建立单一接口，不要建立庞大臃肿的接口

尽量细化接口，接口中的方法尽量少

接口分隔原则的优点和适度原则接口分隔原则从对接口的使用上为我们对接口抽象的颗粒度建立了判断基准：在为系统设计接口的时候，使用多个专门的接口代替单一的胖接口。

符合高内聚低耦合的设计思想，从而使得类具有很好的可读性、可扩展性和可维护性。

注意适度原则，接口分隔要适度，避免产生大量的细小接口。

单一职责原则和接口分隔原则的区别单一职责强调的是接口、类、方法的职责是单一的，强调职责，方法可以多，针对程序中实现的细节；

接口分隔原则主要是约束接口，针对抽象、整体框架。

4.7.6 依赖倒置原则 (Dependency Inversion Principle , DIP)

A. 高层模块不应该依赖于低层模块，二者都应该依赖于抽象

B. 抽象不应该依赖于细节，细节应该依赖于抽象

C. 针对接口编程，不要针对实现编程。

依赖：在程序设计中，如果一个模块 a 使用/调用了另一个模块 b，我们称模块 a 依赖模块 b。

高层模块与低层模块：往往在一个应用程序中，我们有一些低层次的类，这些类实现了一些基本的或初级的操作，我们称之为低层模块；另外有一些高层次的类，这些类封装了某些复杂的逻辑，并且依赖于低层次的类，这些类我们称之为高层模块。

依赖倒置 (Dependency Inversion)：面向对象程序设计相对于面向过程（结构化）程序设计而言，依赖关系被倒置了。因为传统的结构化程序设计中，高层模块总是依赖于低层模块。

4.7.7 组合/聚合复用原则 (Composite/Aggregate Reuse Principle , CARP)

尽量使用组合/聚合，不要使用类继承。

即在一个新的对象里面使用一些已有的对象，使之成为新对象的一部分，新对象通过向这些对象的委派达到复用已有功能的目的。就是说要尽量地使用合成和聚合，而不是继承关系达到复用的目的。

组合和聚合都是关联的特殊种类。

聚合表示整体和部分的关系，表示“拥有”。组合则是一种更强的“拥有”，部分和整体的生命周期一样。

组合的新的对象完全支配其组成部分，包括它们的创建和湮灭等。一个组合关系的成分对象是不能与另一个组合关系共享的。

组合是值的聚合 (Aggregation by Value)，而一般说的聚合是引用的聚合 (Aggregation by Reference)。

在面向对象设计中，有两种基本的办法可以实现复用：第一种是通过组合/聚合，第二种就是通过继承。

什么时候才应该使用继承只有当以下的条件全部被满足时，才应当使用继承关系：

1) 派生类是基类的一个特殊种类，而不是基类的一个角色，也就是区分“Has-A”和“Is-A”。只有“Is-A”关系才符合继承关系，“Has-A”关系应当用聚合来描述。

2) 永远不会出现需要将派生类换成另外一个类的派生类的情况。如果不能肯定将来是否会变成另外一个派生类的话，就不要使用继承。

3) 派生类具有扩展基类的责任，而不是具有置换掉 (override) 或注销掉 (Nullify) 基类的责任。如果一个派生类需要大量的置换掉基类的行为，那么这个类就不应该是这个基类的派生类。

4) 只有在分类学角度上有意义时，才可以使用继承。

总的来说：

如果语义上存在着明确的“Is-A”关系，并且这种关系是稳定的、不变的，则考虑使用继承；如果没有“Is-A”关系，或者这种关系是可变的，使用组合。另外一个就是只有两个类满足里氏替换原则的时候，才可能是“Is-A”关系。也就是说，如果两个类是“Has-A”关系，但是设计成了继承，那么肯定违反里氏替换原则。

错误的使用继承而不是组合/聚合的一个常见原因是错误的把“Has-A”当成了“Is-A”。“Is-A”代表一个类是另外一个类的一种；“Has-A”代表一个类是另外一个类的一个角色，而不是另外一个类的特殊种类。

4.8 小结

多态的实现条件是这些类要满足继承关系，也就是多态只能发生在基类与派生类之间；

要实现多态，需将基类的方法定义为虚函数 (virtual)，派生类根据需要覆盖 (override) 相应的基类虚函数；

抽象方法不需要函数体，含有一个抽象方法的类必须定义为抽象类，抽象类不能实例化；

接口中不能含有字段，只能有方法与属性，接口可以继承接口，接口支持多继承，继承自接口的类必须实现接口。

第 5 章 WPF 界面编写基础

5.1 WPF 设计原则

5.1.1 概述

WPF 是 Windows Presentation Foundation 的简称,是一个与分辨率无关的 UI 框架。WPF 底层是通过 GPU 图形硬件加速,基于矢量的呈现引擎。WPF 提供一套完善的应用程序开发功能: Extensible Application Markup Language (XAML)、控件、数据绑定、布局、二维和三维图形、动画、样式、模板、文档、媒体、文本和版式。

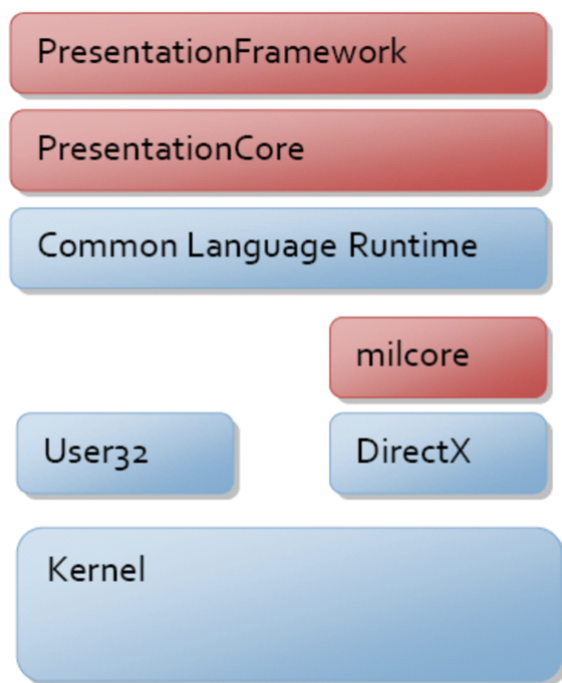


图 5.1 WPF 主要组件

WPF 的主要组件如图 5.1 所示,关系图的红色部分 (PresentationFramework、PresentationCore 和 milcore) 是 WPF 的主要代码部分。WPF 中的所有显示都是通过 DirectX 引擎完成的,因此硬件和软件呈现高效。

WPF 作为 .NET 类型的一个子集存在,大部分位于 System.Windows 命名空间中。WPF 设计思想是将界面标记描述与实现代码分离,从而实现界面设计 (美工) 与代码实现 (开发人

员) 并行工作, 提升开发效率。

界面标记描述使用 XAML, XAML 的英文全称为 Extensible Application Markup Language, 翻译为中文为“可扩展应用程序标记语言”, XAML 是派生自 XML 的可扩展应用程序标记语言, 以声明形式实现应用程序的外观。通常用它定义窗口、对话框、页面和用户控件, 并填充控件、形状和图形。

现代软件开发如 Android 等, 均采用 xml 等标记语言编写界面, 因此 WPF 的思想以及 XAML 的方法很值得我们学习。

5.1.2 WPF 核心设计原则

传统 GDI 方式与 WinForm 界面编写方式, 均以鼠标拖拽为基础布置界面。

WPF 界面设计的核心是:

1. 布局 Grid 化

摒弃鼠标拖拽这种过于简单方式布局界面, 一般的应用将整个 Window/Page 用 Grid 进行网格化, 再将各个控件按相应的网格行列号布置到相应位置上。

网格宽度与高度则可设置为自动、指定像素值、按比例设置三种模式, 从而控制在界面大小发生改变时, 仍然使界面保持合理布局。

2. 数据交换绑定化

传统的界面与数据交换方式是直接向诸如 Textbox 的 Text 属性取值, 并将其转换为相应数值, 提供给相应算法进行计算, 再将计算结果赋值到界面元素的相应属性中。

这种方式简单、直接。但在数据的容错处理上十分困难, 从而导致程序的健壮性、功能正确性上也受到影响。

WPF 为我们提供了数据绑定的方式进行数据交换。

3. 数据驱动界面化

WPF 以数据为中心, 数据驱动界面, 而不是传统的事件驱动。

WPF 是通过样式、控件模板、数据模板等方式实现了数据驱动界面。

5.2 XAML 基本语法

由于课时与篇幅的原因, 此处对 WPF 的设计原理不做过多的论述, 我们直接生成一个实例来分析 XAML 的语法。

5.2.1 WPF 项目结构

我们基于 .net 6 生成一个默认的 WPF 项目, 项目结构如图 5.2 所示:

项目中主要有两个文件 App.xaml 与 MainWindow.xaml。每个 xaml 文件都有一个相应的扩展名为 cs 的后台编码文件。

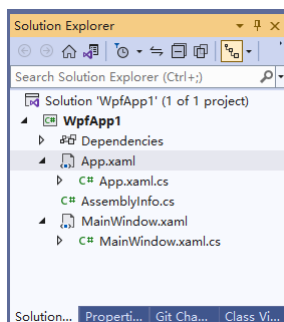


图 5.2 WPF 项目结构

5.2.2 项目中的 xaml 文件及含义

App.xaml 文件主要描述了项目的初始信息，代码如下：

```
1 <Application x:Class="WpfApp1.App"
2     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4     xmlns:local="clr-namespace:WpfApp1"
5     StartupUri="MainWindow.xaml">
6     <Application.Resources>
7
8     </Application.Resources>
9 </Application>
```

最主要的就是 StartupUri 属性指定了该 Application 的启动地址是“MainWindow.xaml”文件。

MainWindow.xaml 文件描述了启动的主窗体，代码如下：

```
1 <Window x:Class="WpfApp1.MainWindow"
2     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
5     xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
6     xmlns:local="clr-namespace:WpfApp1"
7     mc:Ignorable="d"
8     Title="MainWindow" Height="450" Width="800">
9     <Grid>
10
11     </Grid>
12 </Window>
```

文件中出现的 xmlns 为文件中引入的命名空间，xmlns 为 xml namespace 的缩写。

从这两个文件可以看出，每个 xaml 文件都是一颗 xml 的树，这棵树的根为 Application、Window、Page 等等。

5.3 WPF 常用控件

WPF 控件基本上可以分为六类：

1. 布局控件

可以容纳多个控件或嵌套其它布局控件，用于在 UI 上组织和排列控件。如 Grid、Stack-Panel、DockPanel 等。

2. 内容控件

只能容纳一个其它控件或布局控件作为它的内容。Window、Button 等均属于该类，是继承于类 ContentControl。

3. 带标题内容控件

相当于一个内容控件，但可以加一个标题 (Header)，如 GroupBox、TabItem 等，是继承于类 HeaderedContentControl。

4. 条目控件

可以显示一系列的数据，如 ListBox、ComboBox 等，它们继承于类 ItemsControl。

5. 带标题条目控件

相当于一个条目控件加上一个标题显示区，如 TreeViewItem、MenuItem 等，用于显示层级关系数据，节点显示在 Header 区域，子节点显示在条目控件区域。它们继承于类 HeaderedItemsControl。

6. 特殊内容控件

如 TextBox 容纳的字符串，TextBlock 容纳的可自由控制格式的文本，Image 容纳图片类型数据。

5.4 坐标方位角计算图形界面程序编写

前面我们编写了坐标方位角计算算法函数与角度弧度互相转换的函数。这节我们将借助于 WPF 界面编写技术完成一个较为完整的图形界面的坐标方位角程序的编写。

5.4.1 建立解决方案与项目

启动 Visual Studio (本书中所用版本为 2017 英文版)，依次点击菜单 File -> New -> Project...，在弹出的对话框中按图5.3所示输入新建项目名称、选择项目类型等。

建好后的项目如图5.4所示：

由图5.4可以看出，Visual Studio 是以 Solution、Project、File 的形式组织管理的。一个 Solution 可以包括一个至多个 Project，一个 Project 中包含多个文件，且一个 Project 编译为一个 exe 或 dll 文件，C# 中所有的对象与数据均以文件的形式组织。

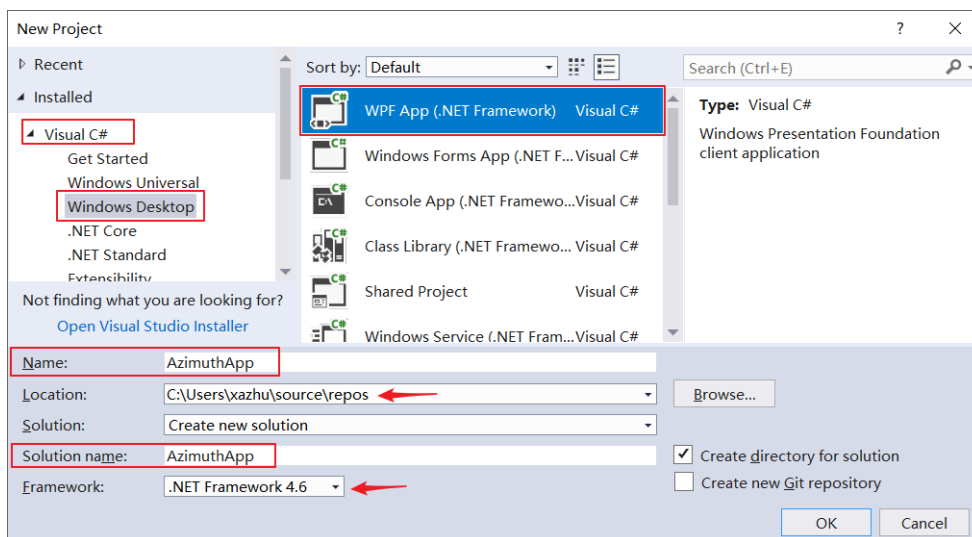


图 5.3 新建项目示意图

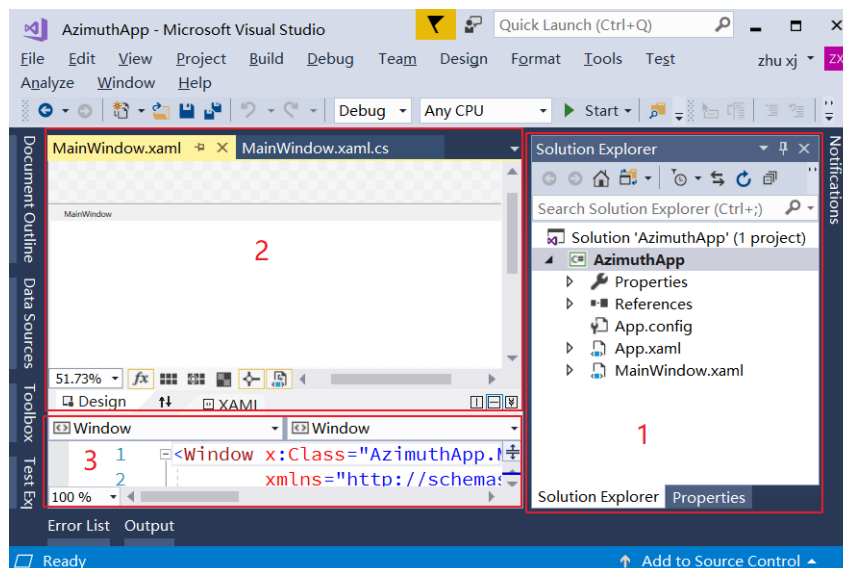


图 5.4 方位角计算项目示意图

图5.4中的 1 区为 Solution Explorer 区，可对 Project 及 File 进行各种操作。2、3 区为代码与界面编写区，文件类型不同，呈现的内容也会不一样。

现代软件开发应遵循界面与算法分离的原则，因此我们再新建一类库项目 SMath，用于组织我们的算法，如图5.5所示：

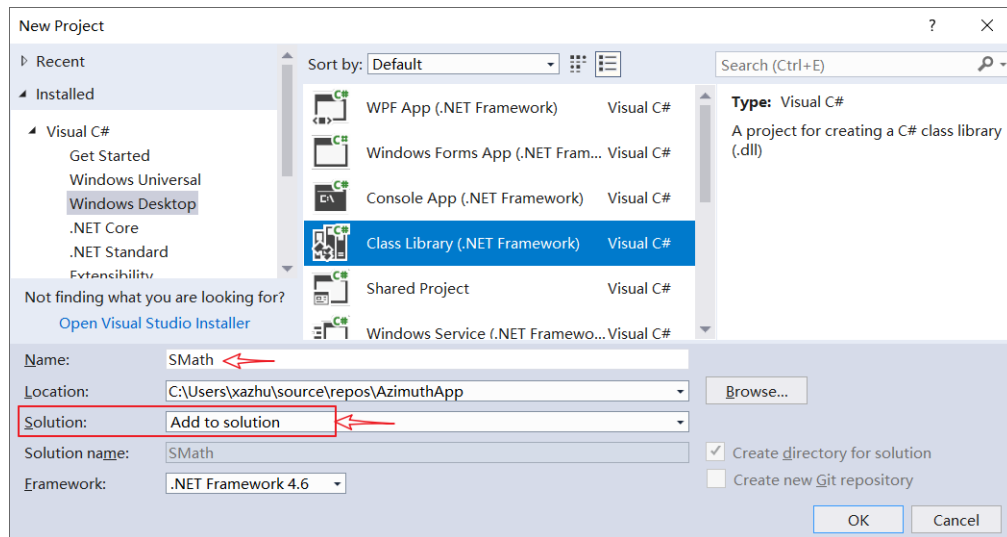


图 5.5 新建类库项目示意图

在该步操作中除了项目类型应选择 Class Library 之外，与图5.3 不同的是此处 Solution 项应选择 Add to solution 而不是原来的 Create new solution。

现代软件开发的另一个原则是保证代码的可测试性，在增加类库项目后，再增加一单元测试项目 UnitTestSMath。鼠标右击图5.4中的 Solution Explorer 区中的 Solution 'AzimuthApp' 项，在弹出的右键快捷菜单中选择 Add，在 Add 的下级菜单中选择 New Project...，将弹出如图5.6所示的添加单元测试项目的对话框。

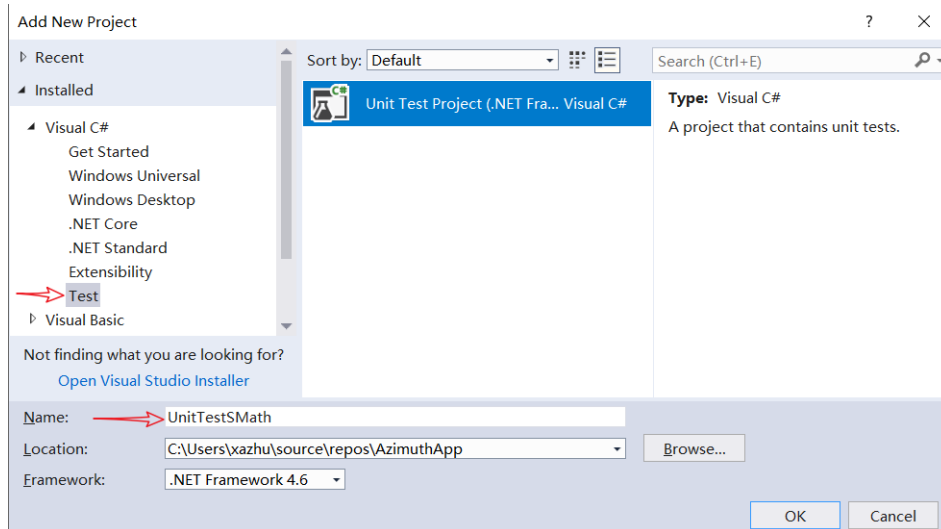


图 5.6 添加单元测试项目示意图

在图5.6的操作中,应在左侧选择 Test,在右边选择 Unit Test Project,

整个操作完成后,我们的解决方案 Solution 就拥有三个项目了,如图5.7所示。项目 AzimuthApp 为我们的界面编写项目,是启动项目 (StartUp Project)。项目 SMath 为我们的算法项目,项目 UnitTestSMath 为对算法项目 SMath 进行单元测试的项目。

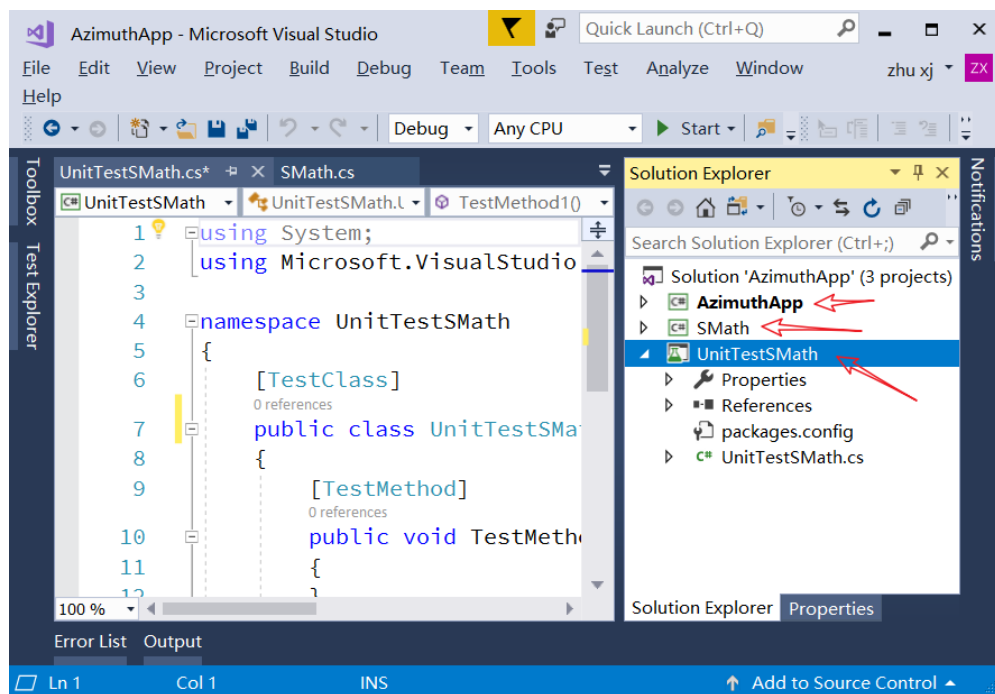


图 5.7 完整的 Solution 示意图

下面我们通过添加引用或参考 (Add Reference) 为三个项目建立关系。

SMath 项目为基础算法, AzimuthApp 需要用到其中的算法,因此应该向 AzimuthApp 项目添加对 SMath 项目的引用。点击 AzimuthApp 项目将其展开,鼠标右击项目中的 References 项,弹出如图5.8所示的快捷菜单:

点击 Add Reference... 项,弹出如图5.9所示的对话框。在对话框左侧确保选中 Projects,在右侧选中 SMath 项目,单击 OK 按钮就将引用添加完毕。同样的方法向 UnitTestSMath 项目添加对 SMath 项目的引用。

至此,我们的项目构建完成,整个程序的架构已经搭建好了。在 SMath 项目中将原来的 Class1.cs 文件删除,新添一 SMath.cs 文件,在 SMath.cs 文件中将我们前边讲解的 SMath 类加入其中,将各个函数组织进来,对该项目执行构建 (build) 操作生成 Dll 类库文件,在另两个项目中就可以使用了。

5.4.2 界面编写

在确保基本算法的正确性后,我们就可以编写界面程序了。我们编写一简单界面,如图5.10所示。

相应的 XAML 设计代码如下:

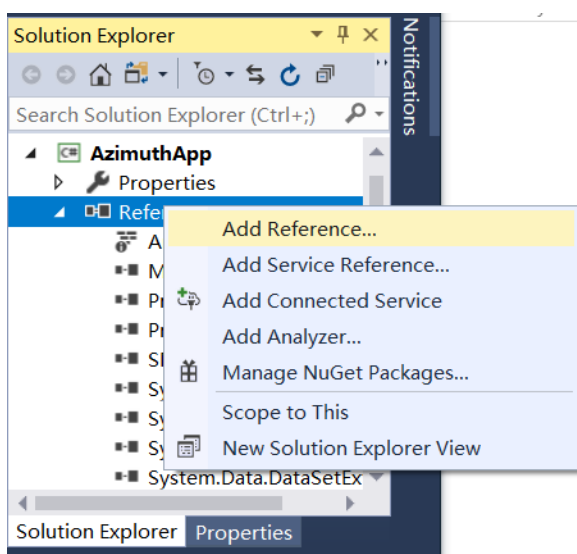


图 5.8 引用类库项目示意图

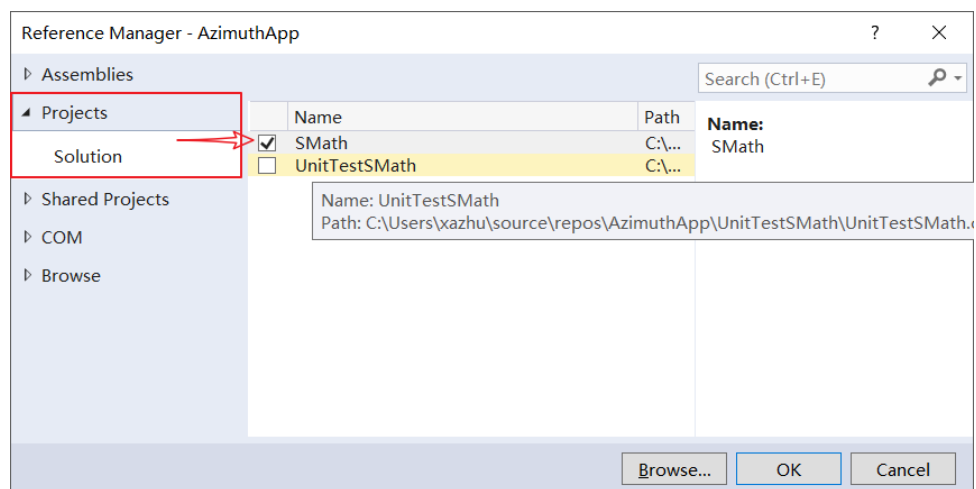


图 5.9 选择引用类库项目示意图

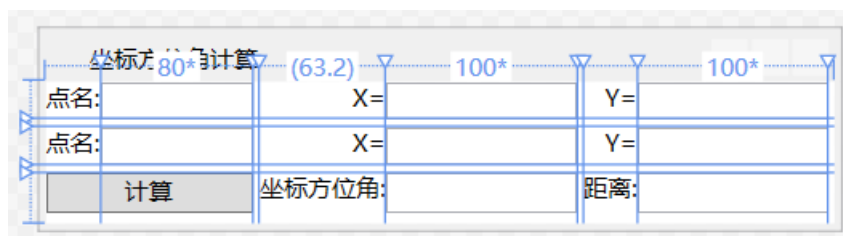


图 5.10 坐标方位角计算设计界面

```

1 <Window x:Class="AzimuthApp.MainWindow"
2     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
5     xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
6     xmlns:local="clr-namespace:AzimuthApp"
7     mc:Ignorable="d"
8     Title="坐标方位角计算" Height="100" Width="400">
9     <Grid>
10         <Grid.ColumnDefinitions>
11             <ColumnDefinition Width="Auto"/>
12             <ColumnDefinition Width="80*"/>
13             <ColumnDefinition Width="3"/>
14             <ColumnDefinition Width="Auto"/>
15             <ColumnDefinition Width="100*"/>
16             <ColumnDefinition Width="3"/>
17             <ColumnDefinition Width="Auto"/>
18             <ColumnDefinition Width="100*"/>
19             <ColumnDefinition Width="3"/>
20         </Grid.ColumnDefinitions>
21         <Grid.RowDefinitions>
22             <RowDefinition Height="Auto"/>
23             <RowDefinition Height="3"/>
24             <RowDefinition Height="Auto"/>
25             <RowDefinition Height="3"/>
26             <RowDefinition Height="Auto"/>
27         </Grid.RowDefinitions>
28         <TextBlock Text="点名:" Grid.Row="0" Grid.Column="0" />
29         <TextBox x:Name="textBoxAName" Grid.Row="0" Grid.Column="1" />
30         <TextBlock Text="X=" TextAlignment="Right" Grid.Row="0" Grid.Column="3"
31     />
32         <TextBox x:Name="textBoxAX" Grid.Row="0" Grid.Column="4"/>
33         <TextBlock Text="Y=" TextAlignment="Right" Grid.Row="0" Grid.Column="6"
34     />
35         <TextBox x:Name="textBoxAY" Grid.Row="0" Grid.Column="7" />
36
37         <TextBlock Text="点名:" Grid.Row="2" Grid.Column="0" />
38         <TextBox x:Name="textBoxBName" Grid.Row="2" Grid.Column="1" />
39         <TextBlock Text="X=" TextAlignment="Right" Grid.Row="2" Grid.Column="3"
40     />
41         <TextBox x:Name="textBoxBX" Grid.Row="2" Grid.Column="4"/>
42         <TextBlock Text="Y=" TextAlignment="Right" Grid.Row="2" Grid.Column="6"
43     />
44         <TextBox x:Name="textBoxBY" Grid.Row="2" Grid.Column="7" />
45
46         <Button x:Name="buttonAzimuth" Content="计算" Grid.Row="5"

```

```

43         Grid.Column="0" Grid.ColumnSpan="2" Click="buttonAzimuth_Click"/>
44         <TextBlock x:Name="textBlockAzimuth"
45             Text="坐标方位角:" TextAlignment="Right" Grid.Row="5" Grid.Column="
3"/>
46         <TextBox x:Name="textBoxAzimuth" Grid.Row="5" Grid.Column="4" />
47         <TextBlock Text="距离:" TextAlignment="Right" Grid.Row="5" Grid.Column="
6" />
48         <TextBox x:Name="textBoxDistance" Grid.Row="5" Grid.Column="7" />
49     </Grid>
50 </Window>

```

计算按钮的 Click 事件代码如下：

```

1 private void buttonAzimuth_Click(object sender, RoutedEventArgs e)
2 {
3     double.TryParse(textBoxAX.Text, out double xA);
4     double.TryParse(textBoxAY.Text, out double yA);
5     double.TryParse(textBoxBX.Text, out double xB);
6     double.TryParse(textBoxBY.Text, out double yB);
7
8     double distanceAB = ZXY.SMath.Azimuth(xA, yA, xB, yB, out double azimuthAB);
9
10    textBoxAzimuth.Text = ZXY.SMath.RADtoString(azimuthAB);
11    textBoxDistance.Text = distanceAB.ToString();
12    textBlockAzimuth.Text = $"{textBoxAName.Text}->{textBoxBName.Text}坐标方位
角:";
13 }

```

程序的运行结果如图5.11所示。

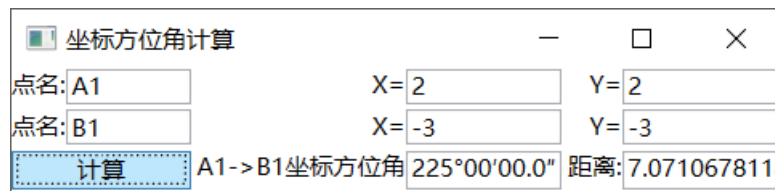


图 5.11 坐标方位角计算运行界面

5.4.3 界面程序的优化

WPF 技术的核心是绑定 (Binding)，以上程序虽然简单，但也涉及数据与界面的交互问题。我们对其界面稍加抽象，改写一下。

我们新建一个类 AzimuthUI，其代码如下：

```

1 using System.ComponentModel;
2

```

```
3 namespace AzimuthApp
4 {
5     class AzimuthUI : INotifyPropertyChanged
6     {
7         public event PropertyChangedEventHandler PropertyChanged;
8         public void RaisePropertyChanged(string propertyName)
9         {
10             if (PropertyChanged != null)
11             {
12                 PropertyChanged.Invoke(this, new PropertyChangedEventArgs(
13                     propertyName));
14             }
15
16             private string _AName = "";
17             public string AName
18             {
19                 get => _AName;
20                 set
21                 {
22                     _AName = value;
23                     RaisePropertyChanged("AName");
24                 }
25             }
26
27             private double _xA;
28             public double XA
29             {
30                 get => _xA;
31                 set
32                 {
33                     _xA = value;
34                     RaisePropertyChanged("XA");
35                 }
36             }
37
38
39             private double _yA;
40             public double YA
41             {
42                 get => _yA;
43                 set
44                 {
45                     _yA = value;
46                     RaisePropertyChanged("YA");
47                 }
48             }
49         }
50     }
```

```
48     }
49
50     private string _BName = "";
51     public string BName
52     {
53         get => _BName;
54         set
55         {
56             _BName = value;
57             RaisePropertyChanged("BName");
58         }
59     }
60
61     private double _xB;
62     public double XB
63     {
64         get => _xB;
65         set
66         {
67             _xB = value;
68             RaisePropertyChanged("XB");
69         }
70     }
71
72     private double _yB;
73     public double YB
74     {
75         get => _yB;
76         set
77         {
78             _yB = value;
79             RaisePropertyChanged("YB");
80         }
81     }
82
83     public string AzimuthName
84     {
85         get => $"{AName}->{BName}坐标方位角:";
86     }
87
88     private double _azimuthAB;
89     public string AzimuthAB
90     {
91         get => ZXY.SMath.RADtoString(_azimuthAB);
92     }
93
```

```

94     private double _distanceAB;
95     public double DistanceAB
96     {
97         get => _distanceAB;
98     }
99
100     public void CalAzimuthDistanceAB()
101     {
102         _distanceAB = ZXY.SMath.Azimuth(XA, YA, XB, YB, out _azimuthAB);
103         RaisePropertyChanged("AzimuthAB");
104         RaisePropertyChanged("DistanceAB");
105         RaisePropertyChanged("AzimuthName");
106     }
107 }
108 }

```

界面代码做如下修改:

```

1  <TextBlock Text="点名:" Grid.Row="0" Grid.Column="0" />
2  <TextBox x:Name="textBoxAName" Text="{Binding AName}"
3  Grid.Row="0" Grid.Column="1" />
4  <TextBlock Text="X=" TextAlignment="Right"
5  Grid.Row="0" Grid.Column="3"/>
6  <TextBox x:Name="textBoxAX" Text="{Binding XA}"
7  Grid.Row="0" Grid.Column="4"/>
8  <TextBlock Text="Y=" TextAlignment="Right"
9  Grid.Row="0" Grid.Column="6"/>
10 <TextBox x:Name="textBoxAY" Text="{Binding YA}"
11 Grid.Row="0" Grid.Column="7" />
12
13 <TextBlock Text="点名:" Grid.Row="2" Grid.Column="0" />
14 <TextBox x:Name="textBoxBName" Text="{Binding BName}"
15 Grid.Row="2" Grid.Column="1" />
16 <TextBlock Text="X=" TextAlignment="Right" Grid.Row="2" Grid.Column="3"/>
17 <TextBox x:Name="textBoxBX" Text="{Binding XB}"
18 Grid.Row="2" Grid.Column="4" />
19 <TextBlock Text="Y=" TextAlignment="Right" Grid.Row="2" Grid.Column="6"/>
20 <TextBox x:Name="textBoxBY" Text="{Binding YB}"
21 Grid.Row="2" Grid.Column="7" />
22
23 <Button x:Name="buttonAzimuth" Content="计算" Grid.Row="5"
24 Grid.Column="0" Grid.ColumnSpan="2"
25 Click="buttonAzimuth_Click"/>
26 <TextBlock x:Name="textBlockAzimuth"
27 Text="{Binding AzimuthName, Mode=OneWay}"
28 TextAlignment="Right" Grid.Row="5" Grid.Column="3"/>
29 <TextBox x:Name="textBoxAzimuth" Text="{Binding AzimuthAB, Mode=OneWay}"

```

```
30 Grid.Row="5" Grid.Column="4" />
31 <TextBlock Text="距离:" TextAlignment="Right" Grid.Row="5" Grid.Column="6"
32 />
33 <TextBox x:Name="textBoxDistance" Text="{Binding DistanceAB, Mode=OneWay}"
34 Grid.Row="5" Grid.Column="7" />
```

MainWindow 窗体的后代代码做如下修改：

```
1 namespace AzimuthApp
2 {
3     public partial class MainWindow : Window
4     {
5         AzimuthUI azimuthUI;
6         public MainWindow()
7         {
8             InitializeComponent();
9
10            azimuthUI = new AzimuthUI();
11            this.DataContext = azimuthUI;
12        }
13
14        private void buttonAzimuth_Click(object sender, RoutedEventArgs e)
15        {
16            azimuthUI.CalAzimuthDistanceAB();
17        }
18    }
19 }
```

第 6 章 通用高斯投影程序设计

高斯投影是为了解决球面与平面之间的坐标映射问题，即大地坐标 (B, L) 与高斯平面直角坐标 (x, y) 之间的相互换算，以及不同带之间的高斯坐标的换算问题。

高斯投影是我国 1:50 万以及更大比例尺地形图的数学基础，我国自 1953 年开始就采用，也是世界不少国家的地形图基础。高斯投影在低纬度地区变形较大，在纬度 30 以下投影带的边缘部分变形值超过 1/1000。对低纬度地区地形图不是很适宜，对高纬度地区的地形图较好。

通用横墨卡托 (Universal Transverse Mercator) 简称为 UTM 投影，是横割圆柱投影，投影方法与高斯投影基本相同。但每带中央子午线的长度比为 0.9996。对中纬度地区与低纬度地区来讲，UTM 投影优于高斯-克吕格投影。UTM 投影是目前美国、德国等 60 多个国家的国家基本地形图的数学基础。

本章将运用 C# 编程语言编写一个通用的高斯投影程序，用于 1954 北京坐标系、1980 西安坐标系、WGS84 坐标系以及 CGCS2000 大地坐标系或自定义参考椭球的高斯投影正反算与换带计算。

6.1 高斯投影的数学模型

为了编制正确而且高效的高斯投影与换带程序，我们首先需要分析高斯投影的数学模型，也就是我们常说的算法。

本章所引用的公式来自：孔祥元, 郭际明, 刘宗泉. 大地测量学基础-2 版. 武汉: 武汉大学出版社, 2010.5, 以下将这本书简称为大地测量学基础或大地测量学。

6.1.1 椭球基础

投影是基于参考椭球进行的，所以首先需要分析参考椭球的计算。

高斯投影是在椭球的几何参数 (长半轴 a、短半轴 b、扁率 α) 确定的条件下，根据给定的数学模型来进行计算的，我们首先分析这些计算公式与数学模型。

1. 基本公式

(a) 扁率：

$$\alpha = \frac{a - b}{a}$$

(b) 第一偏心率：

$$e = \sqrt{\frac{a^2 - b^2}{a^2}}$$

(c) 第二偏心率:

$$e' = \sqrt{\frac{a^2 - b^2}{b^2}}$$

(d) 子午圈曲率半径:

$$M = \frac{a(1 - e^2)}{(1 - e^2 \sin^2 B)^{\frac{3}{2}}}$$

(e) 卯酉圈曲率半径:

$$N = \frac{a}{\sqrt{1 - e^2 \sin^2 B}}$$

(f) 辅助符号:

$$t = \tan B \quad \eta = e' \cos B$$

2. 椭球面梯形图幅面积计算

$$P = \frac{b^2}{2}(L_2 - L_1) \left| \frac{\sin B}{1 - e^2 \sin^2 B} + \frac{1}{2e} \ln \frac{1 + e \sin B}{1 - e \sin B} \right|_{B_1}^{B_2}$$

公式引用自《大地测量学基础》第 121 页 (4-140)。

$$P = b^2(L_2 - L_1) \left| \sin B + \frac{2}{3}e^2 \sin^3 B + \frac{3}{5}e^4 \sin^5 B + \frac{4}{7}e^6 \sin^7 B \right|_{B_1}^{B_2}$$

公式引用自《大地测量学基础》第 121 页 (4-142), 该公式为 (4-140) 的展开式。

3. 子午线弧长

$$X = a_0 B - \frac{a_2}{2} \sin 2B + \frac{a_4}{4} \sin 4B - \frac{a_6}{6} \sin 6B + \frac{a_8}{8} \sin 8B \quad (6.1)$$

式中:

$$\left. \begin{aligned} a_0 &= m_0 + \frac{m_2}{2} + \frac{3}{8}m_4 + \frac{5}{16}m_6 + \frac{35}{128}m_8 \\ a_2 &= \frac{m_2}{2} + \frac{m_4}{2} + \frac{15}{32}m_6 + \frac{7}{16}m_8 \\ a_4 &= \frac{m_4}{8} + \frac{3}{16}m_6 + \frac{7}{32}m_8 \\ a_6 &= \frac{m_6}{32} + \frac{m_8}{16} \\ a_8 &= \frac{m_8}{128} \end{aligned} \right\} \quad (6.2)$$

式中 m_0, m_2, m_4, m_6, m_8 的值为:

$$\left. \begin{aligned} m_0 &= a(1 - e^2) \\ m_2 &= \frac{3}{2}e^2 m_0 \\ m_4 &= \frac{5}{4}e^2 m_2 = \frac{15}{8}e^4 m_0 \\ m_6 &= \frac{7}{6}e^2 m_4 = \frac{35}{16}e^6 m_0 \\ m_8 &= \frac{9}{8}e^2 m_6 = \frac{315}{128}e^8 m_0 \end{aligned} \right\} \quad (6.3)$$

以上公式引用自《大地测量学基础》第 115 页 (4-101)、(4-100) 与 (4-72)。

将公式 6.3 代入公式 6.2 可得到如下公式：

$$\left. \begin{aligned} a_0 &= (1 + \frac{3}{4}e^2 + \frac{45}{64}e^4 + \frac{175}{256}e^6 + \frac{11025}{16384}e^8)m_0 \\ a_2 &= (\frac{3}{4}e^2 + \frac{15}{16}e^4 + \frac{525}{512}e^6 + \frac{2205}{2048}e^8)m_0 \\ a_4 &= (\frac{15}{64}e^4 + \frac{105}{256}e^6 + \frac{2205}{4096}e^8)m_0 \\ a_6 &= (\frac{35}{512}e^6 + \frac{315}{2048}e^8)m_0 \\ a_8 &= \frac{315}{16384}e^8m_0 \end{aligned} \right\} \quad (6.4)$$

若令：

$$\left. \begin{aligned} A_0 &= a_0 = (1 + \frac{3}{4}e^2 + \frac{45}{64}e^4 + \frac{175}{256}e^6 + \frac{11025}{16384}e^8)m_0 \\ A_2 &= -\frac{a_2}{2} = -\frac{1}{2}(\frac{3}{4}e^2 + \frac{15}{16}e^4 + \frac{525}{512}e^6 + \frac{2205}{2048}e^8)m_0 \\ A_4 &= \frac{a_4}{4} = \frac{1}{4}(\frac{15}{64}e^4 + \frac{105}{256}e^6 + \frac{2205}{4096}e^8)m_0 \\ A_6 &= -\frac{a_6}{6} = -\frac{1}{6}(\frac{35}{512}e^6 + \frac{315}{2048}e^8)m_0 \\ A_8 &= \frac{a_8}{8} = \frac{1}{8} \times \frac{315}{16384}e^8m_0 \end{aligned} \right\} \quad (6.5)$$

则子午线弧长公式 6.1 可以演变为如下公式：

$$X = A_0B + A_2 \sin 2B + A_4 \sin 4B + A_6 \sin 6B + A_8 \sin 8B \quad (6.6)$$

公式 6.6 将作为我们进行程序设计时进行子午线弧长与底点纬度计算的基本公式。

由公式 6.5 看出，其值只与椭球的定义有关，一旦椭球的参数确定，这些值也就被确定。

6.1.2 高斯投影正算

高斯投影正算就是将球面上一点的大地坐标 (B, L) 投影为高斯平面直角坐标 (x, y) 。

4. 高斯投影正算

$$\left. \begin{aligned} x &= X + \frac{l^2 N}{2} \sin B \cos B + \frac{l^4 N}{24} \sin B \cos^3 B (5 - t^2 + 9\eta^2 + 4\eta^4) \\ &\quad + \frac{N}{720} \sin B \cos^5 B (61 - 58t^2 + t^4) l^6 \\ y &= lN \cos B + \frac{l^3 N}{6} \cos^3 B (1 - t^2 + \eta^2) \\ &\quad + \frac{l^5 N}{120} \cos^5 B (5 - 18t^2 + t^4 + 14\eta^2 - 58\eta^2 t^2) \end{aligned} \right\} \quad (6.7)$$

公式引用自《大地测量学基础》第 169 页 (4-367)。

5. 高斯投影反算

$$\left. \begin{aligned} B &= B_f - \frac{t_f}{2M_f N_f} y^2 + \frac{t_f}{24M_f N_f^3} (5 + 3t_f^2 + \eta_f^2 - 9\eta_f^2 t_f^2) y^4 \\ &\quad - \frac{t_f}{720M_f N_f^5} (61 + 90t_f^2 + 45t_f^4) y^6 \\ l &= \frac{1}{N_f \cos B_f} y - \frac{1}{6N_f^3 \cos B_f} (1 + 2t_f^2 + \eta_f^2) y^3 \\ &\quad + \frac{1}{120N_f^5 \cos B_f} (5 + 28t_f^2 + 24t_f^4 + 6\eta_f^2 + 8\eta_f^2 t_f^2) y^5 \end{aligned} \right\} \quad (6.8)$$

公式引用自《大地测量学基础》第 171 页 (4-383)。

6. 平面子午线收敛角计算

利用 (B, l) 计算公式如下:

$$\gamma = l \cdot \sin B + \frac{l^3}{3} \sin B \cos^2 B \cdot (1 + 3\eta^2 + 2\eta^4) + \frac{l^5}{15} \sin B \cos^4 B \cdot (2 - t^2)$$

公式引用自《大地测量学基础》第 181 页 (4-408)。

利用 (x, y) 计算公式如下:

$$\gamma = \frac{1}{N_f} t_f y - \frac{1 + t_f^2 - \eta_f^2}{3N_f^3} t_f y^3 + \frac{2 + 5t_f^2 + 3t_f^4}{15N_f^5} t_f y^5$$

公式引用自《大地测量学基础》第 182 页 (4-410)。

7. 长度比计算

利用 (B, l) 计算公式如下:

$$m = 1 + \frac{l^2}{2} \cos^2 B (1 + \eta^2) + \frac{l^4}{24} \cos^4 B (5 - 4t^2)$$

公式引用自《大地测量学基础》第 189 页 (4-447)。

利用 (x, y) 计算公式如下:

$$m = 1 + \frac{y^2}{2R^2} + \frac{y^4}{24R^4}$$

式中 $R = \sqrt{MN}$, 公式引用自《大地测量学基础》第 189 页 (4-451)。

6.2 程序功能分析与设计

6.2.1 高斯投影的主要内容

1. 坐标正算: 将点的大地坐标转换成高斯投影平面直角坐标。
2. 坐标反算: 将点的高斯投影平面直角坐标转换成大地坐标。

3. 换带计算：将某带的点的高斯投影平面直角坐标转换成邻带或某中央子午线经度的高斯投影平面直角坐标。
4. 其他计算：计算子午线收敛角、长度比等。

6.2.2 参考椭球类的设计

分析以上各个计算公式发现，如果椭球长半轴与扁率确定，参考椭球的第一偏心率 e 、第二偏心率 e' ，子午线弧长计算的辅助计算参数 $(m_0, m_2, m_4, m_6, m_8)$ 与 $(a_0, a_2, a_4, a_6, a_8)$ 也就确定了，其中的 $(m_0, m_2, m_4, m_6, m_8)$ 作为计算 $(a_0, a_2, a_4, a_6, a_8)$ 的值使用过一次。也就是说这些参数对于某一种确定的参考椭球是常数。而 (M, N, t, η) 则是纬度 B 的函数。

我们新建一 WPF 项目，命名为 GaussProj，程序中我们将应用 WPF 技术编写图形界面。

在高斯投影中由于要处理角度与点等数据，我们可将前面的角度处理函数 DMS2RAD、RAD2DMS 与 SPoint 点类拷贝到当前项目中来，也可以将其打包成一 Class Library(.NET Framework) 即类库文件引用到当前项目中，在当前项目中尽量不修改前面的各个功能代码。

基于上一小节的分析，我们新建一椭球类 (Spheroid)，在其中我们定义以上与椭球类型相关的元素。代码如下所示：

```

1 namespace GaussProj
2 {
3     /// <summary>
4     /// 参考椭球
5     /// </summary>
6     public class Spheroid
7     {
8         /// <summary>
9         /// 长半轴
10        /// </summary>
11        public double a { get; set; }
12
13        /// <summary>
14        /// 短半轴
15        /// </summary>
16        public double b { get; set; }
17
18        /// <summary>
19        /// 扁率分母
20        /// </summary>
21        public double f { get; set; } //(a-b)/a = 1/f
22
23        /// <summary>
24        /// 第一偏心率的平方
25        /// </summary>
26        public double e2 { get; set; }
27

```

```

28     /// <summary>
29     /// 第二偏心率的平方
30     /// </summary>
31     public double eT2 { get; set; }
32
33     //计算子午线弧长时的各个系数项
34     private double a0;
35     private double a2;
36     private double a4;
37     private double a6;
38     private double a8;
39 }
40 }

```

在各项计算中，第一偏心率与第二偏心率的直接使用较少，其平方值用的较多，因此在 Spheroid 类我们直接用其平方值。

在 Spheroid 类中我们可以如下直接定义构造函数用于初始化其各个字段 (Field):

```

1  /// <summary>
2  /// 构造函数
3  /// </summary>
4  private Spheroid(double semimajor_axis, double inverse_flattening)
5  {
6      this.a = semimajor;
7      this.f = inverse_flattening;
8      .....
9  }

```

但这样我们可能无法为已知的一些参考椭球直接提供参数，所以我们将构造函数定义为 private，让用户无法通过 new 直接创建实例化对象。我们设计类的静态函数 Create**** 来创建类 Spheroid 的实例化对象，其代码如下：

```

1  /// <summary>
2  /// 构造函数
3  /// </summary>
4  private Spheroid(){}
5
6  /// <summary>
7  /// 初始化椭球参数的内部函数
8  /// </summary>
9  private void Init(double semimajor_axis, double inverse_flattening)
10 {
11     this.a = semimajor_axis; this.f = inverse_flattening;
12     b = a * (1 - 1 / f);
13
14     e2 = 1 - b / a * b / a;
15     eT2 = a / b * a / b - 1;
16

```

```
17     double m0 = a * (1 - e2);
18     double m2 = 3.0 / 2.0 * e2 * m0;
19     double m4 = 5.0 / 4.0 * e2 * m2;
20     double m6 = 7.0 / 6.0 * e2 * m4;
21     double m8 = 9.0 / 8.0 * e2 * m6;
22
23     a0 = m0 + m2 / 2.0 + 3.0 / 8.0 * m4 + 5.0 / 16.0 * m6
24         + 35.0 / 128.0 * m8;
25     a2 = m2 / 2.0 + m4 / 2.0 + 15.0 / 32.0 * m6 + 7.0 / 16.0 * m8;
26     a4 = m4 / 8.0 + 3.0 / 16.0 * m6 + 7.0 / 32.0 * m8;
27     a6 = m6 / 32.0 + m8 / 16.0;
28     a8 = m8 / 128.0;
29 }
30
31 public static Spheroid CreateBeiJing1954()
32 {
33     Spheroid spheroid = new Spheroid();
34     spheroid.Init(6378245, 298.3);
35     return spheroid;
36 }
37
38 public static Spheroid CreateXian1980()
39 {
40     Spheroid spheroid = new Spheroid();
41     spheroid.Init(6378140, 298.257);
42     return spheroid;
43 }
44
45 public static Spheroid CreateWGS1984()
46 {
47     Spheroid spheroid = new Spheroid();
48     spheroid.Init(6378137, 298.257223563);
49     return spheroid;
50 }
51
52 public static Spheroid CreateCGCS2000()
53 {
54     Spheroid spheroid = new Spheroid();
55     spheroid.Init(6378137, 298.257222101);
56     return spheroid;
57 }
58
59 public static Spheroid CreateCoordinateSystem(double semimajor_axis,
60     double inverse_flattening)
61 {
62     Spheroid spheroid = new Spheroid();
```

```

63     spheroid.Init(semimajor_axis, inverse_flattening);
64     return spheroid;
65 }

```

这种将构造函数设为 private，通过静态函数来创建实例化对象的技术在软件设计中会经常用到。同时在设计类的方法时，应注意访问权限的设置，如上面代码中 Init 函数，在类中用于初始化椭球的各项几何参数，并不需要在类外来调用它，所以我们将其设置为 private 权限。

6.2.3 高斯投影正算功能的实现

有了类 Spheroid 的设计，我们先来完成高斯投影正算功能。为了避免在计算中频繁的进行度分秒与弧度之间的转换问题，我们设定除了特别声明之外，在类 Spheroid 中所用的角度均为弧度。

我们将高斯投影正算的函数名称定义为 BLtoXY，其设计如下：

```

1  /// <summary>
2  /// 高斯投影正算
3  /// </summary>
4  /// <param name="B">纬度,单位:弧度</param>
5  /// <param name="L">经度,单位:弧度</param>
6  /// <param name="L0">中央子午线经度,单位:弧度</param>
7  /// <param name="x">高斯平面x坐标</param>
8  /// <param name="y">高斯平面y坐标</param>
9  public void BLtoXY(double B, double L, double L0,
10     out double x, out double y)
11  {
12     .....
13 }

```

由于函数的返回值为两个值 x 与 y，无法以函数的返回值 return 的形式返回计算结果，所以我们用函数参数 out 的形式将计算结果返回。

由前面的高斯投影正算公式分析可知，高斯投影正算的计算较为简单，没有复杂的逻辑，先计算经差，然后计算子午线弧长后就可以直接写计算坐标 x,y 的算法了。但公式较为复杂，容易写错，公式中具有大量的平方、四次方等变量。因此在编程时应将这些变量命名为与其相似的形式并提前计算。相关代码如下：

```

1  /// <summary>
2  /// 计算子午线弧长
3  /// </summary>
4  /// <param name="B">纬度(单位:弧度)</param>
5  /// <returns>子午线弧长</returns>
6  private double funX(double B)
7  {
8     return a0 * B - a2 / 2.0 * Math.Sin(2 * B)
9         + a4 / 4.0 * Math.Sin(4 * B)
10        - a6 / 6.0 * Math.Sin(6 * B)
11        + a8 / 8.0 * Math.Sin(8 * B);

```

```
12 }
13
14 private double funN(double sinB)
15 {
16     return a / Math.Sqrt(1 - e2 * sinB * sinB);
17 }
18
19 /// <summary>
20 /// 高斯投影正算
21 /// </summary>
22 /// <param name="B">纬度,单位:弧度</param>
23 /// <param name="L">经度,单位:弧度</param>
24 /// <param name="L0">中央子午线经度,单位:弧度</param>
25 /// <param name="x">高斯平面x坐标</param>
26 /// <param name="y">高斯平面y坐标</param>
27 public void BLtoXY(double B, double L, double L0,
28     out double x, out double y)
29 {
30     double l = L - L0; //计算经差
31
32     double sinB = Math.Sin(B);
33     double cosB = Math.Cos(B);
34     double cosB2 = cosB * cosB;
35     double cosB4 = cosB2 * cosB2;
36     double t = Math.Tan(B);
37     double t2 = t * t;
38     double t4 = t2 * t2;
39     double g2 = e2 * cosB * cosB;
40     double g4 = g2 * g2;
41     double l2 = l * l;
42     double l4 = l2 * l2;
43
44     double X = funX(B); //计算子午线弧长
45     double N = funN(sinB);
46
47     x = X + 0.5 * N * sinB * cosB * l2 * (1
48         + cosB2 / 12.0 * (5 - t2 + 9 * g2 + 4 * g4) * l2
49         + cosB4 / 360.0 * (61 - 58 * t2 + t4) * l4);
50
51
52     y = N * cosB * l * (1
53         + cosB2 * (1 - t2 + g2) * l2 / 6.0
54         + cosB4 * (5 - 18 * t2 + t4 + 14 * g2 - 58 * g2 * t2)
55         * l4 / 120.0);
56 }
```

利用 BLtoXY 函数就可以进行高斯投影正算了，其算法流程为：

```

1 //创建克拉索夫斯基参考椭球
2 Spheroid spheroid = Spheroid.CreateBeiJing1954();
3
4 //传入点的纬度B、经度与中央子午线经度，单位为弧度
5 double B = ZXY.SMath.DMS2RAD(21.58470845);
6 double L = ZXY.SMath.DMS2RAD(113.25314880);
7 double L0 = ZXY.SMath.DMS2RAD(111);
8 double x, y;
9
10 spheroid.BLtoXY(B, L, L0, out x, out y);

```

点的纬度、经度为： $B = 21^{\circ}58'47.0845''$, $L = 113^{\circ}25'31.4880''$ ，中央子午线经度为： $L0 = 111^{\circ}$ ，计算出的坐标为： $x = 2433586.692$, $y = 250547.403$ ，坐标 y 为点的自然坐标，未加常数 500km 与带号。

6.2.4 高斯投影反算功能的实现

由高斯投影反算公式分析，在反算时需要首先计算底点纬度。底点纬度需要由子午线弧长公式进行反算，由该公式可以看出，已知 $X=x$ 时，这个函数在计算 B 值时它并不是一个线型函数，不能直接计算。解决这类问题的计算方法就是迭代计算，我们将公式进行变换，如下所示：

$$B = (X + \frac{a_2}{2} \sin 2B - \frac{a_4}{4} \sin 4B + \frac{a_6}{6} \sin 6B - \frac{a_8}{8} \sin 8B) / a_0$$

在该公式中，两边都有 B ，我们将公式右边的 B 赋初始值 $B_0 = X/a_0$ 代入可以计算出新的 B 值，循环进行计算，由于该迭代收敛，两值之差 $B - B_0$ 在一定范围内时我们认为其值 B 即为我们的解。

因此底点纬度计算函数设计为：

```

1 /// <summary>
2 /// 计算底点纬度
3 /// </summary>
4 /// <param name="x">高斯平面坐标x</param>
5 /// <returns>底点纬度</returns>
6 private double funBf(double x)
7 {
8     double sinB, sin2B, sin4B, sin6B, sin8B;
9     double Bf0 = x / a0, Bf = 0; //子午线弧长的初值
10    int i = 0;
11    while (i < 10000) //设定最大迭代次数
12    {
13        sinB = Math.Sin(Bf0);
14        sin2B = Math.Sin(2 * Bf0);
15        sin4B = Math.Sin(4 * Bf0);
16        sin6B = Math.Sin(6 * Bf0);

```

```

17     sin8B = Math.Sin(8 * Bf0);
18     Bf = (x
19         + a2 * sin2B / 2
20         - a4 * sin4B / 4
21         + a6 * sin6B / 6
22         - a8 * sin8B / 8) / a0;
23
24     if (Math.Abs(Bf - Bf0) < 1e-10) //计算精度
25         return Bf;
26     else
27     {
28         Bf0 = Bf;
29         i++;
30     }
31 }
32 return -1e12;
33 }

```

在底点纬度计算出以后，高斯投影反算计算就没有难度了。我们将函数名命名为 XYtoBL，其函数设计为：

```

1 public void XYtoBL(double x, double y, double L0,
2     out double B, out double L)
3 {
4     double Bf = funBf(x);
5
6     double cosBf = Math.Cos(Bf);
7     double gf2 = eT2 * cosBf * cosBf;
8     double gf4 = gf2 * gf2;
9     double tf = Math.Tan(Bf);
10    double tf2 = tf * tf;
11    double tf4 = tf2 * tf2;
12    double sinB = Math.Sin(Bf);
13    double Nf = funN(sinB);
14    double Mf = funM(sinB);
15    double Nf2 = Nf * Nf;
16    double Nf4 = Nf2 * Nf2;
17    double y2 = y * y;
18    double y4 = y2 * y2;
19
20    B = Bf + tf * y2 / Mf / Nf * 0.5 * (
21        -1.0
22        + y2 * (5.0 + 3.0 * tf2 + gf2 - 9.0 * gf2 * tf2) / 12.0 / Nf2
23        - y4 * (61.0 + 90.0 * tf2 + 45.0 * tf4) / 360.0 / Nf4);
24
25    double l = y / Nf / cosBf * (
26        1.0

```

```

27     - y2 / 6.0 / Nf2 * (1.0 + 2.0 * tf2 + gf2)
28     + y4 / 120.0 / Nf4
29     * (5.0 + 28.0 * tf2 + 24.0 * tf4 + 6.0 * gf2 + 8.0 * gf2 * tf2));
30
31     L = L0 + l;
32 }

```

该函数中还有 Nf 与 Mf 需要计算, Nf 的计算同前面的 funN 函数, Mf 的计算函数为

```

1 private double funM(double sinB)
2 {
3     return a * (1 - e2) * Math.Pow(1 - e2 * sinB * sinB, -1.5);
4 }

```

有了高斯投影反算函数 XYtoBL, 就可以比较容易的写出其反算示例了, 如以下代码所示:

```

1 //创建克拉索夫斯基参考椭球
2 Spheroid spheroid = Spheroid.CreateBeiJing1954();
3
4 //传入点的X、Y坐标与中央子午线经度(单位为弧度)
5 double x=2433586.692, y=250547.403;
6 double L0 = ZXY.SMath.DMS2RAD(111);
7
8 double B, L;
9 spheroid.XYtoBL(x, y, L0, out B, out L);
10 //B= ZXY.SMath.RAD2DMS(B);
11 //L= ZXY.SMath.RAD2DMS(L);

```

传入的 y 坐标应为真实坐标值, 应不包括 500km 与带号等。如果计算的经纬度需向界面展示, 还应像上述代码后两行所示将其值转换为度分秒形式。

计算出的点的纬度与经度为: $B = 21^{\circ}58'47.0845''$, $L = 113^{\circ}25'31.4880''$

计算点的子午线收敛角与计算某点处的长度变形值均比较简单, 可以在正算或反算时将其同时计算出, 大家可以自己练习完成, 在此就不再讲述了。

6.2.5 换带计算

换带计算其实质就是变换坐标系的中央子午线位置。因此首先应根据点的坐标反算出点的经纬度, 然后再根据新的坐标系中央子午线位置计算出点在新的坐标系中的高斯平面坐标。

也就是先反算, 再正算, 注意此处的反算与正算的中央子午线经度值是不一样的。

该函数我们命名为 XYtoXY, 其设计为如下代码:

```

1 /// <summary>
2 /// 高斯投影换带
3 /// </summary>
4 /// <param name="ox">点在源坐标系的x坐标</param>
5 /// <param name="oy">点在源坐标系的y坐标</param>
6 /// <param name="oL0">源坐标系的中央子午线经度, 单位:弧度</param>
7 /// <param name="nL0">目标坐标系的中央子午线经度, 单位:弧度</param>

```

```

8 /// <param name="nx">点在目标坐标系的x坐标</param>
9 /// <param name="ny">点在目标坐标系的y坐标</param>
10 public void XYtoXY(double ox, double oy,
11     double oL0, double nL0,
12     out double nx, out double ny)
13 {
14     double B, L;
15     XYtoBL(ox, oy, oL0, out B, out L); //高斯投影反算
16     BLtoXY(B, L, nL0, out nx, out ny); //高斯投影正算
17 }

```

高斯投影换带的外部调用可以按如下方式写：

```

1 double oldX = 3275110.535, oldY = 235437.233;
2
3 double oldL0 = ZXY.SMath.DMS2RAD(117);
4 double newL0 = ZXY.SMath.DMS2RAD(120);
5
6 double newX, newY;
7 Spheroid spheroid = Spheroid.CreateBeiJing1954();
8 spheroid.XYtoXY(oldX, oldY, oldL0, newL0, out newX, out newY);

```

计算出的点在新坐标系下的坐标为 (3272782.315, -55299.545)。

至此，我们已经完成了高斯投影的全部计算功能了。需要注意的是以上的函数调用中的角度均使用了弧度的形式，在外部调用时可以利用前面所讲的度分秒化弧度和弧度化度分秒函数先行转换。

6.3 图形界面程序编写

以上我们已经将主要的算法编写完毕，下面我们将用 C# 的 WPF 技术编写图形界面，让我们的程序变得更加实用与易用。

6.3.1 单点高斯投影正反算图形界面编写

我们先设计一简单的界面进行高斯投影正反算计算，以验证程序功能。程序中我们默认的坐标系为 1954 北京坐标系，界面设计如图6.1所示，界面我们采用 Grid 布局，将其划分成三行三列，将我们的控件布置在相应的单元格中，Textblock 控件用来描述文字类的标题，TextBox 用来输入经纬度与坐标数据，中间用 Button 控件响应事件。

相应的主要界面代码如下：

```

1 <TextBlock Text="大地坐标" HorizontalAlignment="Center"
2     VerticalAlignment="Center"/>
3 <TextBlock Text="B=" Grid.Row="1" Margin="5"/>
4 <TextBox x:Name="textBox_B" Grid.Row="1"
5     Text=""
6     VerticalAlignment="Center" Margin="25,0,0,0"/>

```

图 6.1 简单的高斯投影正反算界面

```

7 <TextBlock Text="L=" Margin="5" Grid.Row="2" />
8 <TextBox x:Name="textBox_L"
9     Text=""
10    VerticalAlignment="Center" Margin="25,0,0,0" Grid.Row="2" />
11
12 <TextBlock Text="L0=" Grid.Column="1"
13     Margin="5,0,0,0" VerticalAlignment="Center"/>
14 <TextBox x:Name="textBox_L0"
15     Text=""
16     Grid.Column="1"
17     Margin="30,0,3,0" VerticalAlignment="Center"/>
18 <Button Content="(B,L)->(x,y)" Grid.Column="1" Grid.Row="1"
19     Click="BLtoXYButton_Click" Margin="3,3"/>
20 <Button Content="(B,L)<-(x,y)" Grid.Column="1" Grid.Row="2"
21     Click="XYtoBLButton_Click" Margin="3,3"/>
22
23 <TextBlock Text="高斯投影坐标" Grid.Column="2"
24     HorizontalAlignment="Center" VerticalAlignment="Center"/>
25 <TextBlock Text="x=" Margin="5" Grid.Row="1" Grid.Column="2"/>
26 <TextBox x:Name="textBox_x" Grid.Row="1" Grid.Column="2"
27     Text=""
28     Margin="25,0,0,0" VerticalAlignment="Center" />
29 <TextBlock Text="y=" Grid.Row="2" Grid.Column="2" Margin="5,0,0,0"/>
30 <TextBox x:Name="textBox_y"
31     Text=""
32     Grid.Row="2" Grid.Column="2"
33     Margin="25,0,0,0" VerticalAlignment="Center" />

```

请注意，在这段 xaml 界面代码中，由于我们的算法需要与界面上的 TextBox 控件进行数据交换，每个 TextBox 都需要命名（如上的每个 TextBox 控件都有一个 x:Name 属性）。

相应的正算与反算按钮响应事件代码如下：

```

1 private void BLtoXYButton_Click(object sender, RoutedEventArgs e)
2 {
3     double B, L, L0, x, y;
4     double.TryParse(this.textBox_B.Text, out B);
5     double.TryParse(this.textBox_L.Text, out L);
6     double.TryParse(this.textBox_L0.Text, out L0);

```

```

7
8     Spheroid proj = Spheroid.CreateBeiJing1954();
9     proj.BLtoXY(
10         ZXY.SMath.DMS2RAD(B),
11         ZXY.SMath.DMS2RAD(L),
12         ZXY.SMath.DMS2RAD(L0),
13         out x, out y);
14     this.textBox_x.Text = x.ToString();
15     this.textBox_y.Text = y.ToString();
16 }
17
18 private void XYtoBLButton_Click(object sender, RoutedEventArgs e)
19 {
20     double B, L, L0, x, y;
21     double.TryParse(this.textBox_x.Text, out x);
22     double.TryParse(this.textBox_y.Text, out y);
23     double.TryParse(this.textBox_L0.Text, out L0);
24
25     Spheroid proj = Spheroid.CreateBeiJing1954();
26     proj.XYtoBL(x, y,
27         ZXY.SMath.DMS2RAD(L0),
28         out B, out L);
29     this.textBox_B.Text = ZXY.SMath.RAD2DMS(B).ToString();
30     this.textBox_L.Text = ZXY.SMath.RAD2DMS(L).ToString();
31 }

```

在这个两个响应事件中，首先需要直接从界面上的 TextBox 控件中取值，由于 TextBox 的 Text 属性是文本（string 类型），需要用 double.TryParse 函数将其转换为 double 类型。

其次我们默认参考椭球为 1954 北京坐标系的参考椭球，需要将其创建，此处我们用类的静态函数可以方便创建，不必记忆其椭球的几何参数值。

然后我们就可以调用我们前边写的正算与反算函数进行高斯投影正反算了，算完后将值再赋值给相应的 TextBox 控件的 Text 属性就可以了。注意传入函数的值如果是度分秒形式的角度，应该先调用我们前边的写的函数将其转换为弧度，如果算出的值也是角度，也应该调用我们前面所写的函数将其转换为度分秒形式。

6.3.2 界面程序的优化

上面的简单界面程序存在着一个很大的问题，就是从界面取数据时无法判断数据的有效性等，也无法发挥 WPF 界面技术。WPF 界面技术里的数据绑定功能（binding）可以很好的简化这一过程。

我们仔细分析前边的界面，这个程序的实质就是一个点的两种形式的坐标之间的转换，因此我们可以定义一个点类 GeoPoint，其定义如下：

```

1 public class GeoPoint
2 {

```

```

3     public string Name { get; set; } //点名
4     public double B { get; set; } //纬度, 单位: 度分秒
5     public double L { get; set; } //经度, 单位: 度分秒
6     public double L0 { get; set; } //中央子午线经度, 单位: 度分秒
7     public double X { get; set; } //X坐标
8     public double Y { get; set; } //Y坐标
9 }

```

则在界面代码中可对 TextBox 的 Text 做如下绑定:

```

1 <TextBlock Text="大地坐标" HorizontalAlignment="Center"
2     VerticalAlignment="Center"/>
3 <TextBlock Text="B=" Grid.Row="1" Margin="5"/>
4 <TextBox x:Name="textBox_B" Grid.Row="1"
5     Text="{Binding B}"
6     VerticalAlignment="Center" Margin="25,0,0,0"/>
7 <TextBlock Text="L=" Margin="5" Grid.Row="2" />
8 <TextBox x:Name="textBox_L"
9     Text="{Binding L}"
10    VerticalAlignment="Center" Margin="25,0,0,0" Grid.Row="2" />
11
12 <TextBlock Text="L0=" Grid.Column="1"
13     Margin="5,0,0,0" VerticalAlignment="Center"/>
14 <TextBox x:Name="textBox_L0"
15     Text="{Binding L0}"
16     Grid.Column="1"
17     Margin="30,0,3,0" VerticalAlignment="Center"/>
18 <Button Content="(B,L)->(x,y)" Grid.Column="1" Grid.Row="1"
19     Click="BLtoXYButton_Click" Margin="3,3"/>
20 <Button Content="(B,L)&lt;-(x,y)" Grid.Column="1" Grid.Row="2"
21     Click="XYtoBLButton_Click" Margin="3,3"/>
22
23 <TextBlock Text="高斯投影坐标" Grid.Column="2"
24     HorizontalAlignment="Center" VerticalAlignment="Center"/>
25 <TextBlock Text="x=" Margin="5" Grid.Row="1" Grid.Column="2"/>
26 <TextBox x:Name="textBox_x" Grid.Row="1" Grid.Column="2"
27     Text="{Binding X}"
28     Margin="25,0,0,0" VerticalAlignment="Center" />
29 <TextBlock Text="y=" Grid.Row="2" Grid.Column="2" Margin="5,0,0,0"/>
30 <TextBox x:Name="textBox_y"
31     Text="{Binding Y}"
32     Grid.Row="2" Grid.Column="2"
33     Margin="25,0,0,0" VerticalAlignment="Center" />

```

以上代码中的 TextBox 控件中的 x:Name 属性甚至都可以省略。由于这些控件都是以这个窗体 (Window) 作为容器的, 他们的数据源都可用这个窗体的 DataContext 一次性设置, 让系统以冒泡的形式自动为属性绑定寻找数据源。窗体为之设定数据源的代码如下:

```

1 public partial class MainWindow : Window
2 {
3     private GeoPoint geoPoint;
4     private Spheroid proj = Spheroid.CreateBeiJing1954();
5
6     public MainWindow()
7     {
8         InitializeComponent();
9         geoPoint = new GeoPoint(){ B= 21.58470845, L= 113.25314880 };
10        this.DataContext = geoPoint;
11    }
12    //.....省略了其他代码.....
13 }

```

程序中由于正反算都是基于相同的椭球基准，所以在类 MainWindow 中定义了 geoPoint 实例字段与 proj 实例字段。在构造函数中为其赋了初值以简化每次在界面输入数据，为这个窗体的 DataContext 指定点的各项属性绑定的数据源。

相应的正反算按钮的响应事件修改为：

```

1 private void BLtoXYButton_Click(object sender, RoutedEventArgs e)
2 {
3     double x, y;
4     proj.BLtoXY(
5         ZXY.SMath.DMS2RAD(geoPoint.B),
6         ZXY.SMath.DMS2RAD(geoPoint.L),
7         ZXY.SMath.DMS2RAD(geoPoint.L0),
8         out x, out y);
9     geoPoint.X = x; geoPoint.Y = y;
10 }
11
12 private void XYtoBLButton_Click(object sender, RoutedEventArgs e)
13 {
14     double B, L;
15     proj.XYtoBL(geoPoint.X, geoPoint.Y,
16         ZXY.SMath.DMS2RAD(geoPoint.L0),
17         out B, out L);
18     geoPoint.B = ZXY.SMath.RAD2DMS(B);
19     geoPoint.L = ZXY.SMath.RAD2DMS(L);
20 }

```

从响应事件可以看出，代码简洁了很多。运行程序时，TextBox 框中都有默认数值，而且非数值数据也输入不进去了，也不需要将在文本框中的 Text 属性转换为 double 类型了。一切看似都好，但你发现在点击正算或反算按钮时，界面上的数据没有变化，好像功能没有实现一样，问题出现在什么地方呢？

我们回过头再看 GeoPoint 的定义，发现其属性定义过于简单。根据 WPF 知识可知，在对象属性发生改变时（如我们的计算中正算改变了 X 与 Y，反算改变了 B 与 L），还需要一种

机制通知系统需要刷新界面，这就需要类 GeoPoint 从接口 INotifyPropertyChanged 继承并实现它（该接口所在的命名空间为 System.ComponentModel）。修改后的 GeoPoint 类如下：

```
1 public class GeoPoint : INotifyPropertyChanged
2 {
3     public event PropertyChangedEventHandler PropertyChanged;
4
5     public void RaisePropertyChange(string propertyName)
6     {
7         if (this.PropertyChanged != null)
8         {
9             this.PropertyChanged.Invoke(this,
10                 new PropertyChangedEventArgs(propertyName));
11         }
12     }
13
14     private string _name;
15     public string Name //点名
16     {
17         get { return _name; }
18         set
19         {
20             _name = value;
21             RaisePropertyChange("Name");
22         }
23     }
24
25     private double _B;
26     public double B //纬度，单位：度分秒
27     {
28         get { return _B; }
29         set
30         {
31             _B = value;
32             RaisePropertyChange("B");
33         }
34     }
35
36     private double _L;
37     public double L //经度，单位：度分秒
38     {
39         get { return _L; }
40         set
41         {
42             _L = value;
43             RaisePropertyChange("L");
```

```
44     }
45 }
46
47 private double _L0;
48 public double L0 //中央子午线经度, 单位: 度分秒
49 {
50     get { return _L0; }
51     set
52     {
53         _L0 = value;
54         RaisePropertyChanged("L0");
55     }
56 }
57
58 private double _X;
59 public double X //X坐标
60 {
61     get { return _X; }
62     set
63     {
64         _X = value;
65         RaisePropertyChanged("X");
66     }
67 }
68
69 private double _Y;
70 public double Y //Y坐标
71 {
72     get { return _Y; }
73     set
74     {
75         _Y = value;
76         RaisePropertyChanged("Y");
77     }
78 }
79 }
```

与前面的 GeoPoint 类相比较, 现在的这个类从 INotifyPropertyChanged 继承并实现了接口成员 PropertyChanged, 在属性值发生改变时利用该接口成员通知界面属性值发生了变化。

运行程序, 功能一切正常。

6.3.3 点类的进一步优化

我们再次审阅界面程序后的正反算代码, 发现事实上的正反算都是基于点类的, 也就是说只与 GeoPoint 类相关, 因此我们将正反算功能移到 GeoPoint 类中, 以进一步简化界面的方法调用。其代码如下:

```

1 public class GeoPoint : INotifyPropertyChanged
2 {
3     //.....其它代码.....
4     public void BLtoXY(Spheroid spheroid)
5     {
6         double x, y;
7         spheroid.BLtoXY(
8             ZXY.SMath.DMS2RAD(this.B),
9             ZXY.SMath.DMS2RAD(this.L),
10            ZXY.SMath.DMS2RAD(this.L0),
11            out x, out y);
12         this.X = x; this.Y = y;
13     }
14
15     public void XYtoBL(Spheroid spheroid)
16     {
17         double B, L;
18         spheroid.XYtoBL(X, Y, ZXY.SMath.DMS2RAD(this.L0),
19             out B, out L);
20         this.B = ZXY.SMath.RAD2DMS(B);
21         this.L = ZXY.SMath.RAD2DMS(L);
22     }
23 }

```

界面正反算代码可以进一步简化为如下形式：

```

1 private void BLtoXYButton_Click(object sender, RoutedEventArgs e)
2 {
3     geoPoint.BLtoXY(proj);
4 }
5
6 private void XYtoBLButton_Click(object sender, RoutedEventArgs e)
7 {
8     geoPoint.XYtoBL(proj) ;
9 }

```

至此，我们的算法与界面几乎是完全分离了，这符合我们第一章所讲的界面与算法相分离的原则，也为我们的算法进行单元测试和进一步优化迭代打下了基础。

6.4 更加实用的多点计算图形程序

上面的程序从编程的角度讲比较完美了，但从实用的角度来说还有很多缺点，比如不能选择椭球基准，不能进行多点的正反算，不能读取和写出文本文件数据。这一节我们将从算法和界面两个方面来构造这个更加实用的多点高斯投影计算的图形程序。

6.4.1 程序的功能

实用的高斯投影程序功能如下：

- 1. 椭球基准的选择：能够自由的选择参考椭球基准，或者自定义参考椭球；
- 2. 能实现高斯投影正反算与换带功能；
- 3. 高斯投影坐标的定义：能自动去除或添加点的 Y 坐标前的常数 500km 和带号；
- 4. 数据的界面录入：能利用程序界面组织输入数据；
- 5. 能导入导出文本数据：能将外部文本数据导入到程序中，能将程序中的数据导出为文本文件；
- 6. 能实现多个点的批量计算。

软件运行时的界面如图6.2所示，该界面基本能满足以上功能。



图 6.2 实用高斯投影程序界面

6.4.2 程序的面向对象分析与实现

从界面上我们可以看出，程序中应该包含三部分内容：椭球基准、坐标系、点集。为了满足以上功能，我们需要对我们的程序进行重构。

分析我们前边的 GeoPoint 类会发现，一个点有中央子午线经度 L0 这个属性会很奇怪，而在实际应用中点集也是基于坐标系的点集，一个点集的 y 坐标甚至 x 坐标前的加常数也是基本相同。因此坐标系应该有中央子午线经度 L0 属性，带号 N 与 Y 坐标前的加常数 YKM，由于坐标系是依赖于椭球基准的，也应有椭球基准属性 ProjSpheroid。坐标系类的定义如下代码所示：

```
1 using System.Collections.ObjectModel;
2
3 /// <summary>
4 /// 坐标系
5 /// </summary>
6 public class CoordinateSystem : INotifyPropertyChanged
7 {
8     //....省略与接口INotifyPropertyChanged有关的代码....
9
10    /// <summary>
11    /// 中央子午线经度
12    /// </summary>
13    private double _L0;
14
15    /// <summary>
16    /// 中央子午线经度
17    /// </summary>
18    public double L0
19    {
20        get { return _L0; }
21        set
22        {
23            _L0 = value;
24            RaisePropertyChange("L0");
25        }
26    }
27
28    /// <summary>
29    /// 带号
30    /// </summary>
31    private int _N;
32
33    /// <summary>
34    /// 带号
35    /// </summary>
36    public int N
37    {
38        get { return _N; }
39        set
40        {
```

```
41         _N = value;
42         RaisePropertyChange("N");
43     }
44 }
45
46 /// <summary>
47 /// Y坐标的加常数
48 /// </summary>
49 private double _YKM;
50
51 /// <summary>
52 /// Y坐标的加常数
53 /// </summary>
54 public double YKM
55 {
56     get { return _YKM; }
57     set
58     {
59         _YKM = value;
60         RaisePropertyChange("YKM");
61     }
62 }
63
64 /// <summary>
65 /// 坐标点集
66 /// </summary>
67 private ObservableCollection<GeoPoint> geoPointList =
68     new ObservableCollection<GeoPoint>();
69
70 /// <summary>
71 /// 坐标点集
72 /// </summary>
73 public ObservableCollection<GeoPoint> GeoPointList
74 {
75     get { return geoPointList; }
76 }
77
78 /// <summary>
79 /// 投影椭球基准
80 /// </summary>
81 private Spheroid spheroid = Spheroid.CreateBeiJing1954();
82
83 /// <summary>
84 /// 投影椭球基准
85 /// </summary>
86 public Spheroid ProjSpheroid
```

```

87     {
88         get { return spheroid; }
89         set
90         {
91             spheroid = value;
92             RaisePropertyChanged("ProjSpheroid");
93         }
94     }
95
96     public CoordinateSystem() { }
97 }

```

在类 GeoPoint 中将属性 L0 的定义删除。在 CoordinateSystem 类中, 由于 N, L0, YKM 需与界面交互, 故须从接口 INotifyPropertyChanged 继承。

为与 WPF 界面中的 DataGrid 控件交互, 点集需要用 ObservableCollection<GPoint> 表达, 不能用 List<GPoint>, 而且默认状态下就应该生成其实例对象。注意二者的命名空间也不一样, 前者为 System.Collections.ObjectModel, 用于界面交互较多, 后者为 System.Collections.Generic, 用于不需要界面的算法较多。

为了与界面的初始状态一致, 对投影的参考椭球我们默认生成北京 54 坐标系的参考椭球。

在我们的程序中 GeoPoint 类与 CoordinateSystem 类为了处理与界面交互的问题, 都需要实现接口 INotifyPropertyChanged, 实现接口的代码重复。本着相同或相似的代码在程序中只写一次的原则, 我们将这部分的代码独立到类 NotificationObject 中, 让 GeoPoint 类与 CoordinateSystem 类从 NotificationObject 类继承。相应的实现代码如下:

```

1 using System.ComponentModel;
2
3 namespace GaussProj
4 {
5     public class NotificationObject : INotifyPropertyChanged
6     {
7         public event PropertyChangedEventHandler PropertyChanged;
8
9         public void RaisePropertyChanged(string propertyName)
10        {
11            if (this.PropertyChanged != null)
12            {
13                this.PropertyChanged.Invoke(this, new PropertyChangedEventArgs(
14                    propertyName));
15            }
16        }
17    }
18
19    public class GeoPoint : NotificationObject
20    {
21        //删除与NotificationObject类中相同的代码

```

```

22     //省略GeoPoint类中的内容
23 }
24
25 public class CoordinateSystem : NotificationObject
26 {
27     //删除与NotificationObject类中相同的代码
28     //省略CoordinateSystem类中的内容
29 }

```

6.4.3 多点的高斯投影计算

如此，在类 `CoordinateSystem` 中就有了用于高斯投影的椭球基准，有了坐标系的中央子午线经度，有了带号及 `y` 坐标前的加常数与点集，多点的高斯投影计算就万事俱备，只欠实现了。其实现代码如下：

```

1 public class CoordinateSystem : NotificationObject
2 {
3     //省略其他代码
4
5     /// <summary>
6     /// 多点高斯投影正算
7     /// </summary>
8     public void BLtoXY()
9     {
10         foreach (var pnt in this.geoPointList)
11         {
12             pnt.BLtoXY(spheroid, this);
13         }
14     }
15
16     /// <summary>
17     /// 多点高斯投影反算
18     /// </summary>
19     public void XYtoBL()
20     {
21         foreach (var pnt in this.geoPointList)
22         {
23             pnt.XYtoBL(spheroid, this);
24         }
25     }
26 }

```

从代码中可以看出，在类 `CoordinateSystem` 中并没有真正的进行高斯投影正算与反算，而是通过循环将其委托给每个点的实例了。

点类 `GeoPoint` 中的高斯投影正反算实现如下：

```

1 public class GeoPoint : NotificationObject

```

```

2 {
3     //省略其他代码
4     /// <summary>
5     /// 高斯投影正算
6     /// </summary>
7     /// <param name="spheroid">投影椭球</param>
8     /// <param name="cs">坐标系</param>
9     public void BLtoXY(Spheroid spheroid, CoordinateSystem cs)
10    {
11        double x, y;
12        spheroid.BLtoXY(
13            ZXY.SMath.DMS2RAD(this.B),
14            ZXY.SMath.DMS2RAD(this.L),
15            ZXY.SMath.DMS2RAD(cs.L0),
16            out x, out y);
17        this.X = x; this.Y = y + cs.YKM * 1000 + cs.N * 1000000;
18    }
19    /// <summary>
20    /// 高斯投影反算
21    /// </summary>
22    /// <param name="spheroid">投影椭球</param>
23    /// <param name="cs">坐标系</param>
24    public void XYtoBL(Spheroid spheroid, CoordinateSystem cs)
25    {
26        double tB, tL;
27        double y = Y - cs.YKM * 1000 - cs.N * 1000000;
28        spheroid.XYtoBL(X, y, ZXY.SMath.DMS2RAD(cs.L0),
29            out tB, out tL);
30        this.B = ZXY.SMath.RAD2DMS(tB);
31        this.L = ZXY.SMath.RAD2DMS(tL);
32    }
33 }

```

从如上的代码中可以看出，真正的高斯投影正反算还是在我们前面写的 Spheroid 类中。请注意在 Spheroid 类中，所有的与角度有关的单位是弧度，y 坐标也是点的真实坐标，在此我们需要根据坐标系中的信息对其做相应的预处理。

还应注意，在 BLtoXY 中，为了与界面交互，要赋值给 this.X 与 this.Y，而不是赋值给其变量 this._X 与 this._Y。在 XYtoBL 中也同样如此，当然还需要将计算出的弧度值转换为度分秒形式。

6.4.4 点坐标数据的读入与写出

利用我们的界面可以手工输入点的坐标数据，但导入与导出文本数据对于一个程序来讲是必不可少的功能。

为了避免我们的教学程序过于复杂，我们对数据文件的格式进行适当简化。

在高斯投影正算时，所需的数据应该是：点名，纬度 B，经度 L，设计我们的文本文件内容如下：

```
#点名,    B,    L
GP01,34.2154,109.112
GP02,34.215,109.1119
GP03,34.2145,109.1119
GP04,34.2147,109.1114
GP05,34.2151,109.1114
```

文件中的每一行第一个字符以 # 开头的我们视为注释行，予以忽略。

在高斯投影反算时，所需的数据应该是：点名，X，Y，设计我们的文本文件内容如下：

```
#点名,      X,      Y,      H
GP01, 3805709.2106, 19333388.3123, 466.419
GP02, 3805595.1034, 19333360.1973, 470.94
GP03, 3805440.0738, 19333360.1727, 478.728
GP04, 3805481.9494, 19333237.0999, 475.975
GP05, 3805598.4201, 19333235.7343, 469.738
```

文件中的数据项可以多余三项，我们只读取第 1、2、3 项，其余忽略。

在写出数据时，我们将点的五项数据：点名，B，L，X，Y 全部写出，如下所示：

```
#点名,    B,    L,    X,    Y
GP01, 34.2154, 109.112 , 3804863.095, 36609369.106
GP02, 34.215,   109.1119, 3804748.229, 36609344.381
GP03, 34.2145, 109.1119, 3804593.296, 36609348.937
GP04, 34.2147, 109.1114, 3804631.509, 36609224.705
GP05, 34.2151, 109.1114, 3804747.866, 36609219.899
```

用户在使用时可以将其数据拷贝到 Word 中按分隔符“,”生成表格进行编辑处理，或按分隔符“,”导入到 Excel 中进行编辑排版处理。

读入的点坐标数据应存储在我们的程序中，很显然应该在类 CoordinateSystem 中实现读入文本文件数据功能，写出数据的功能也如此，其实现代码如下：

```
1 public class CoordinateSystem : NotificationObject
2 {
3     //省略其他代码
4
5     /// <summary>
6     /// 读入点集坐标数据
7     /// </summary>
8     /// <param name="fileName">文件名</param>
9     /// <param name="format">点的坐标格式：BL-Name, B, L
```

```

10      /// XY-Name, X, Y
11      /// </param>
12      public void ReadGeoPointData(string fileName, string format)
13      {
14          using (System.IO.StreamReader sr = new System.IO.StreamReader(fileName)
15          )
16          {
17              string buffer;
18              //读入点的坐标数据
19              this.GeoPointList.Clear();
20              while (true)
21              {
22                  buffer = sr.ReadLine();
23                  if (string.IsNullOrEmpty(buffer)) break; //文件末尾或空行退出
24                  if (buffer[0] == '#') continue;
25                  string[] its = buffer.Split(new char[1] { ',' });
26                  if (its.Length < 3) continue; //少于三项数据, 不是点的坐标数
27                  //数据, 忽略
28                  GeoPoint pnt = new GeoPoint();
29                  pnt.Name = its[0].Trim();
30                  if (format == "XY")
31                  {
32                      pnt.X = double.Parse(its[1]);
33                      pnt.Y = double.Parse(its[2]);
34                      pnt.B = 0; pnt.L = 0;
35                  }
36                  else if (format == "BL")
37                  {
38                      pnt.B = double.Parse(its[1]);
39                      pnt.L = double.Parse(its[2]);
40                      pnt.X = 0; pnt.Y = 0;
41                  }
42                  this.GeoPointList.Add(pnt);
43              }
44          }
45
46      /// <summary>
47      /// 写出点集坐标数据
48      /// </summary>
49      /// <param name="fileName">文件名</param>
50      public void WriteGeoPointData(string fileName)
51      {
52          using (System.IO.StreamWriter sr = new System.IO.StreamWriter(fileName)
53          )

```

```

53     {
54         sr.WriteLine("#点名,  B,  L,  X,  Y");
55         foreach (var pnt in this.geoPointList)
56         {
57             sr.WriteLine( pnt );
58         }
59     }
60 }
61 }

```

写数据比较简单, 按要求写出即可, 需要注意的是第 57 行, 在函数 WriteLine 中我们直接写出了 GeoPoint 的实例对象 pnt, 这需要在 GeoPoint 中将 ToString() 进行 override, 实现代码如下:

```

1 public class GeoPoint : NotificationObject
2 {
3     public override string ToString()
4     {
5         return string.Format("{0},{1:0.0000},{2:0.0000},{3:0.000},{4:0.000}",
6             Name, B, L, X, Y);
7     }
8 }

```

在占位符中我们加入了输出浮点数据的格式控制, 保证输出的角度小数位数不超过四位, 输出的坐标不超过三位。

读入数据相对于写出数据要由于需要解码数据, 所以要复杂一些。小于三个数据项的行我们直接略过, 同时我们能加入格式控制, 如果格式控制 BL, 意味着数据文件是经纬度数据, 其他属性相应置零。

读入数据的这段代码我们实现的方式较为粗略, 只是通过逗号分隔的数据项个数进行了判断, 这在真正的程序开发中是不可靠的, 可以通过正则表达式对每行数据进行检验, 对符合要求的文本数据加以处理, 以此来提高读取文本数据功能的容错能力。

清除 (X, Y) 与清除 (B, L) 功能实质上是将点的这些属性置零, 比较简单, 相应代码如下:

```

1 public class CoordinateSystem : NotificationObject
2 {
3     //省略其他代码
4
5     public void ClearXY()
6     {
7         foreach (var pnt in this.geoPointList)
8         {
9             pnt.X = pnt.Y = 0;
10        }
11    }
12    public void ClearBL()
13    {
14        foreach (var pnt in this.geoPointList)

```

```

15     {
16         pnt.B = pnt.L = 0;
17     }
18 }
19 }

```

6.4.5 界面设计与实现

按图6.2设计的界面代码如下：

```

1 <Window x:Class="GaussProj.GaussProjWin"
2     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
5     xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
6     xmlns:local="clr-namespace:GaussProj"
7     mc:Ignorable="d"
8     Title="高斯投影换带" Height="360" Width="540">
9     <Grid>
10         <Grid.RowDefinitions>
11             <RowDefinition Height="45"/>
12             <RowDefinition Height="5"/>
13             <RowDefinition Height="45"/>
14             <RowDefinition Height="5"/>
15             <RowDefinition Height="Auto"/>
16         </Grid.RowDefinitions>
17         <GroupBox x:Name="groupBox_spheroid" Header="参考椭球定义"
18             DataContext="ProjSpheroid" Margin="1">
19             <Grid>
20                 <Grid.ColumnDefinitions>
21                     <ColumnDefinition Width="200*"/>
22                     <ColumnDefinition Width="70"/>
23                     <ColumnDefinition Width="110*"/>
24                     <ColumnDefinition Width="60"/>
25                     <ColumnDefinition Width="110*"/>
26                 </Grid.ColumnDefinitions>
27                 <ComboBox Grid.Column="0" x:Name="comboBox_Spheroid"
28                     SelectionChanged="comboBox_Spheroid_SelectionChanged"
29                 >
30                     <ComboBoxItem Content="1954北京坐标系" Tag="BJ1954"/>
31                     <ComboBoxItem Content="1980西安坐标系" Tag="XA1980"/>
32                     <ComboBoxItem Content="CGCS2000大地坐标系" Tag="CGCS2000"/>
33                     <ComboBoxItem Content="自定义参考椭球" Tag="CS0000"/>
34                 </ComboBox>
35                 <TextBlock Text="长半轴: a=" Grid.Column="1"
36                     VerticalAlignment="Center" HorizontalAlignment="Right"/>

```

```

36         <TextBox x:Name="textBox_a" Grid.Column="2" Text="{Binding a}"/
37     >
38         <TextBlock Text="扁率: 1/" Grid.Column="3"
39             VerticalAlignment="Center" HorizontalAlignment="Right"/>
40         <TextBox x:Name="textBox_f" Grid.Column="4" Text="{Binding f}"/
41     >
42     </Grid>
43 </GroupBox>
44
45 <Border Grid.Row="2" BorderBrush="Yellow" BorderThickness="2">
46     <GroupBox x:Name="groupBox_CoordinateSystem" Header="坐标系定义">
47         <StackPanel Orientation="Horizontal">
48             <TextBlock Text="中央子午线经度L0=" />
49             <TextBox x:Name="textBox_L0" Text="{Binding L0}" Width="50"/>
50             <TextBlock Text="Y坐标加常数: " Margin="5,0,0,0" />
51             <TextBox x:Name="textBox_YKM" Text="{Binding YKM}" Width="50"/>
52             <TextBlock Text="km" />
53             <TextBlock Text="带号: " Margin="5,0,0,0"/>
54             <TextBox x:Name="textBox_N" Text="{Binding N}" Width="50"/>
55         </StackPanel>
56     </GroupBox>
57 </Border>
58
59 <Border Grid.Row="4" BorderBrush="Yellow" BorderThickness="2">
60     <Grid>
61         <Grid.ColumnDefinitions>
62             <ColumnDefinition Width="200*"/>
63             <ColumnDefinition Width="90"/>
64         </Grid.ColumnDefinitions>
65         <DataGrid x:Name="dataGrid_ctrPnt"
66             AutoGenerateColumns="False" Margin="2"
67             ItemsSource="{Binding GeoPointList}" >
68             <DataGrid.Columns>
69                 <DataGridTextColumn Header="点名"
70                     Binding="{Binding Name}"
71                     MinWidth="40"/>
72                 <DataGridTextColumn Header="B"
73                     Binding="{Binding B , StringFormat={}{0:##0.####}}"
74                     MinWidth="60"/>
75                 <DataGridTextColumn Header="L"
76                     Binding="{Binding L , StringFormat={}{0:##0.####}}"
77                     MinWidth="60"/>
78                 <DataGridTextColumn Header="X"
79                     Binding="{Binding X , StringFormat={}{0:##0.####}}"
80                     MinWidth="60"/>
81                 <DataGridTextColumn Header="Y"

```

```

80         Binding="{Binding Y, StringFormat={}{0:##0.###}}"
```

```
81         MinWidth="60" />
```

```
82     </DataGrid.Columns>
```

```
83 </DataGrid>
```

```
84
```

```
85     <StackPanel Grid.Column="1" Orientation="Vertical">
```

```
86         <RadioButton x:Name="radioButton_BLtoXY" Content="正算"
```

```
87             Height="25" Width="80" Margin="5"/>
```

```
88         <RadioButton x:Name="radioButton_XYtoBL" Content="反算"
```

```
89             IsChecked="True" Height="25" Width="80" Margin="5" />
```

```
90
```

```
91         <Button x:Name="Button_ReadGaussProjData" Content="读入数据"
```

```
92             Height="25" Width="80" Margin="5"
```

```
93             Click="Button_ReadGaussProjData_Click"/>
```

```
94         <Button x:Name="Button_WriteGaussProjData" Content="写出数
```

```
95             据"
```

```
96             Height="25" Width="80" Margin="5"
```

```
97             Click="Button_WriteGaussProjData_Click"/>
```

```
98
```

```
99         <Button x:Name="Button_ClearXY" Content="清除(X,Y)"
```

```
100             Height="25" Width="80" Margin="5"
```

```
101             Click="Button_ClearXY_Click"/>
```

```
102         <Button x:Name="Button_ClearBL" Content="清除(B, L)"
```

```
103             Height="25" Width="80" Margin="5"
```

```
104             Click="Button_ClearBL_Click"/>
```

```
105
```

```
106         <Button x:Name="Button_CalGaussProj" Content="计算"
```

```
107             Height="25" Width="80" Margin="5"
```

```
108             Click="Button_CalGaussProj_Click"/>
```

```
109         <Button x:Name="Button_Close" Content="关闭"
```

```
110             Height="25" Width="80" Margin="5"
```

```
111             Click="Button_Close_Click"/>
```

```
112     </StackPanel>
```

```
113 </Grid>
```

```
114 </Border>
```

```
115 </Grid>
```

```
116 </Window>
```

在这段 xaml 代码中，除了功能按钮区外，实际上分为了三部分：

第一部分为参考椭球定义，如第 18 行代码所示，我们将其布局到 groupBox_spheroid 中，数据绑定 a 与 f，数据源设置在 groupBox_spheroid 中，采用冒泡搜寻类 CoordinateSystem 中的 ProjSpheroid 属性。

为了简化示例程序的编写，ComboBox 中的参考椭球类型采用硬编码方式直接写在其中了。通过 SelectionChange 事件对不同的椭球类型选择进行响应。

第二部分为坐标系定义，同样采用数据绑定的形式与界面交互数据，数据源来自类 `CoordinateSystem`。

第三部分为点集部分，数据源为类 `CoordinateSystem` 中的 `GeoPointList` 属性，如第 65 行代码所示。

界面后台代码如下：

```

1 public partial class GaussProjWin : Window
2 {
3     private CoordinateSystem myCoordinateSystem;
4
5     public GaussProjWin()
6     {
7         InitializeComponent();
8
9         myCoordinateSystem = new CoordinateSystem() {
10             L0 = 111, N = 19, YKM = 500};
11         this.DataContext = myCoordinateSystem;
12
13         this.comboBox_Spheroid.SelectedIndex = 0;
14     }
15
16     //省略其他代码
17 }
```

在窗体后台代码中，我们设置了窗体类的类实例变量 `myCoordinateSystem` 完成各项功能。在其窗体类的构造函数中实例化并对其属性赋了初始值，同时将窗体的 `DataContext` 属性设为 `myCoordinateSystem`，完成数据绑定的数据源设置。同时为了维持与界面属性的一致性，将 `comboBox_Spheroid` 的 `SelectedIndex` 设为 0。

界面的其它功能按钮的实现比较简单，实现代码如下：

```

1 public partial class GaussProjWin : Window
2 {
3     //省略其他代码
4
5     private void comboBox_Spheroid_SelectionChanged(object sender,
6         SelectionChangedEventArgs e)
7     {
8         if (comboBox_Spheroid.SelectedIndex == 0)
9             myCoordinateSystem.ProjSpheroid = Spheroid.CreateBeiJing1954();
10        else if (comboBox_Spheroid.SelectedIndex == 1)
11            myCoordinateSystem.ProjSpheroid = Spheroid.CreateXian1980();
12        else if (comboBox_Spheroid.SelectedIndex == 2)
13            myCoordinateSystem.ProjSpheroid = Spheroid.CreateCGCS2000();
14        else if (comboBox_Spheroid.SelectedIndex == 3)
15            myCoordinateSystem.ProjSpheroid =
16                Spheroid.CreateCoordinateSystem(6378137, 298.257222101);
17        this.groupBox_spheroid.DataContext = myCoordinateSystem.ProjSpheroid;
18    }
19 }
```

```
18     }
19
20     private void Button_ReadGaussProjData_Click(object sender,
21         RoutedEventArgs e)
22     {
23         OpenFileDialog dlg = new OpenFileDialog();
24         dlg.DefaultExt = ".txt";
25         dlg.Filter = "高斯投影坐标数据|*.txt|All File(*)|*.*";
26         if (dlg.ShowDialog() != true) return;
27
28         if (this.radioButton_BLtoXY.IsChecked == true)
29             myCoordinateSystem.ReadGeoPointData(dlg.FileName, "BL");
30         else if (this.radioButton_XYtoBL.IsChecked == true)
31             myCoordinateSystem.ReadGeoPointData(dlg.FileName, "XY");
32     }
33
34     private void Button_WriteGaussProjData_Click(object sender,
35         RoutedEventArgs e)
36     {
37         SaveFileDialog dlg = new SaveFileDialog();
38         dlg.DefaultExt = ".txt";
39         dlg.Filter = "高斯投影坐标数据|*.txt|All File(*)|*.*";
40         if (dlg.ShowDialog() != true) return;
41
42         myCoordinateSystem.WriteGeoPointData(dlg.FileName);
43     }
44
45     private void Button_ClearXY_Click(object sender, RoutedEventArgs e)
46     {
47         myCoordinateSystem.ClearXY();
48     }
49
50     private void Button_ClearBL_Click(object sender, RoutedEventArgs e)
51     {
52         myCoordinateSystem.ClearBL();
53     }
54
55     private void Button_CalGaussProj_Click(object sender, RoutedEventArgs e)
56     {
57         if (radioButton_BLtoXY.IsChecked == true)
58             myCoordinateSystem.BLtoXY();
59         else if (radioButton_XYtoBL.IsChecked == true)
60             myCoordinateSystem.XYtoBL();
61     }
62
63     private void Button_Close_Click(object sender, RoutedEventArgs e)
```

```
64 {
65     this.Close();
66 }
67 }
```

6.4.6 换带功能的实现

在我们以上的设计中好像没有换带计算，虽然没有直接实现，但确实是实现了。在前边的论述中，我们讲过，换带计算的实质是变换坐标系的中央子午线位置，过程为：先进行坐标反算，然后变换中央子午线经度，再反算新的坐标即可。



图 6.3 6° 带第 19 带数据界面

启动程序，在反算的模式下读入数据，如图6.3所示，这是一 6 度带第 19 带的北京 54 坐标:

点击计算，反算各点的大地坐标经纬度，如图6.4所示。

现在我们准备将其换带到 3 度带的 38 带坐标系去，设置中央子午线经度 L0 为 108，带号设为 36，选择正算，点击清除 (x,y) 按钮将 X, Y 坐标设置为 0，如图6.5所示：

点击计算按钮，计算出新坐标系下各点的坐标，如图6.6所示：

到此就完成了换带功能，可以点击写出数据按钮将计算成果输出。

6.5 扩展

6.5.1 UTM 投影

北半球: $x = x_r + 0$

高斯投影换带

参考椭球定义
1954北京坐标系 长半轴: a=6378245 扁率: 1/298.3

坐标系定义
中央子午线经度L0=111 Y坐标加常数: 500 km 带号: 19

点名	B	L	X	Y
GP01	34.2154	109.112	3805709.211	19333388.312
GP02	34.215	109.1119	3805595.103	19333360.197
GP03	34.2145	109.1119	3805440.074	19333360.173
GP04	34.2147	109.1114	3805481.949	19333237.1
GP05	34.2151	109.1114	3805598.42	19333235.734
GP06	34.2153	109.1114	3805684.438	19333237.48
GP07	34.2146	109.1109	3805471.453	19333102.265
GP08	34.2146	109.1107	3805466.169	19333046.388
GP09	34.2147	109.1104	3805493.924	19332974.303
GP10	34.2151	109.1104	3805627.993	19332976.46
GP11	34.2153	109.1104	3805691.585	19332978.197
GP12	34.2157	109.1104	3805802.191	19332980.243
GP13	34.22	109.1104	3805905.385	19332981.768
GP14	34.2204	109.1104	3806031.178	19332985.179

☐ 正算
☒ 反算

读入数据
 写出数据
 清除(X,Y)
 清除(B, L)
 计算
 关闭

图 6.4 6° 带第 19 带反算数据界面

高斯投影换带

参考椭球定义
1954北京坐标系 长半轴: a=6378245 扁率: 1/298.3

坐标系定义
中央子午线经度L0=108 Y坐标加常数: 500 km 带号: 36

点名	B	L	X	Y
GP01	34.2154	109.112	0	0
GP02	34.215	109.1119	0	0
GP03	34.2145	109.1119	0	0
GP04	34.2147	109.1114	0	0
GP05	34.2151	109.1114	0	0
GP06	34.2153	109.1114	0	0
GP07	34.2146	109.1109	0	0
GP08	34.2146	109.1107	0	0
GP09	34.2147	109.1104	0	0
GP10	34.2151	109.1104	0	0
GP11	34.2153	109.1104	0	0
GP12	34.2157	109.1104	0	0
GP13	34.22	109.1104	0	0
GP14	34.2204	109.1104	0	0

☒ 正算
☐ 反算

读入数据
 写出数据
 清除(X,Y)
 清除(B, L)
 计算
 关闭

图 6.5 设置 3° 带第 36 带数据界面

高斯投影换带

参考椭球定义

1954北京坐标系

长半轴: a=6378245

扁率: 1/298.3

坐标系定义

中央子午线经度L0=108

Y坐标加常数: 500

km 带号: 36

点名	B	L	X	Y
GP01	34.2154	109.112	3804863.095	36609369.106
GP02	34.215	109.1119	3804748.229	36609344.381
GP03	34.2145	109.1119	3804593.296	36609348.937
GP04	34.2147	109.1114	3804631.509	36609224.705
GP05	34.2151	109.1114	3804747.866	36609219.899
GP06	34.2153	109.1114	3804833.881	36609219.101
GP07	34.2146	109.1109	3804617.034	36609090.265
GP08	34.2146	109.1107	3804610.103	36609034.58
GP09	34.2147	109.1104	3804635.711	36608961.72
GP10	34.2151	109.1104	3804769.759	36608959.915
GP11	34.2153	109.1104	3804833.362	36608959.772
GP12	34.2157	109.1104	3804943.958	36608958.548
GP13	34.22	109.1104	3805047.131	36608957.022
GP14	34.2204	109.1104	3805172.945	36608956.714

☒ 正算

☐ 反算

读入数据

写出数据

清除(X,Y)

清除(B, L)

计算

关闭

图 6.6 3° 带第 36 带正算数据界面

南半球: $x = x_r + 10,000,000$, 加一万公里

Y 坐标: $y = y_r + 500,000$, 加五百公里

需要注意的是:

UTM 投影的分带与国际 1:100 万地图的划分一致, 从 180° 起向东每 6° 为一带。因此, 高斯-克吕格投影的第一带 (0° – 6° E) 为 UTM 投影的第 31 带, UTM 投影的第一带 (180° – 174° W) 为高斯-克吕格投影的第 31 带。

6.5.2 注意事项

以上过程也可以看成是我们这个软件的教程。但应注意, 换带计算只限于相同椭球基准的情况才能应用, 不能在不同椭球基准间应该该功能。

以上换带功能也可以设计一个界面, 在新的界面上将新旧坐标系的中央子午线经度都设置出, 从而完成一键式换带计算功能。

6.5.3 空间直角坐标系与大地坐标之间的转换

随着我国对 CGCS2000 大地坐标系的推广应用与北斗导航系统的日益成熟, 将大地坐标 (B, L, H) 与空间直角坐标 (X, Y, Z) 进行相互转换的应用也会越来越多。如果能实现大地坐标 (B, L, H) 与空间直角坐标 (X, Y, Z) 的相互转换, 也就可以实现空间直角坐标 (X, Y, Z) 到高斯平面坐标的投影计算了, 进而在桌面端电脑或移动设备上编程实现更多的功能应用。

大地坐标 (B, L, H) 与空间直角坐标 (X, Y, Z) 的相互转换公式如下:

已知大地坐标计算空间直角坐标的公式:

$$\left. \begin{aligned} X &= (N + H) \cos B \cos L \\ Y &= (N + H) \cos B \sin L \\ Z &= [N(1 - e^2) + H] \sin B \end{aligned} \right\} \quad (6.9)$$

已知空间直角坐标 (X, Y, Z) 计算大地坐标 (B, L, H) 的公式如下:

$$\left. \begin{aligned} L &= \arctan \frac{Y}{X} \\ \tan B &= \frac{Z + Ne^2 \sin B}{\sqrt{X^2 + Y^2}} \\ H &= \frac{Z}{\sin B} - N(1 - e^2) \\ \text{or} \\ H &= \frac{\sqrt{X^2 + Y^2}}{\cos B} - N \end{aligned} \right\} \quad (6.10)$$

由以上公式可以看出, L 可以直接由 X, Y 算出, B 则需要迭代计算。

可对上面公式的 B 直接进行迭代计算, 其过程如下:

先取 B 的初始值为: $B_0 = \arctan(Z/\sqrt{X^2 + Y^2})$ 计算出 N 与 $\sin B_0$, 代入上式, 计算出新的 B 值 B_1 。然后将新的 B_1 代入再计算, 直到两次的 B 值满足所要求的精度。

或将:

$$N = \frac{c}{\sqrt{1 + e'^2 \cos^2 B}}$$

与

$$\frac{1}{\cos^2 B} = 1 + \tan^2 B$$

代入到上式中, 得到:

$$\tan B = \frac{Z}{\sqrt{X^2 + Y^2}} + \frac{ce^2 \cdot \tan B}{\sqrt{X^2 + Y^2} \cdot \sqrt{1 + e'^2 + \tan^2 B}} \quad (6.11)$$

则可令 $tB = \tan B$, 对公式两边的 tB 进行迭代计算, 当满足精度要求时, 再求出 B 值 $B = \arctan(tB)$ 即可。

本处只是列出了基本的计算公式与计算思路, 大家可以查找资料寻找更加高效的计算方法予以实现。

6.5.4 同基准下的椭球膨胀法

我们经常在一些资料中见到椭球膨胀法建立独立坐标系或施工坐标系的方法, 椭球膨胀法是依据某种给定的参考椭球, 根据测区的平均高程和平距纬度将椭球面扩大而维持扁率不变, 也可以应用换带的方法将椭球膨胀后的坐标与原坐标系的坐标进行相互转换。但应注意椭球膨胀后, 各个点的纬度值也会发生变化, 在这种换带过程中需要修正各个点的纬度值, 大家可以参看这方面的资料。

平距曲率半径:

$$R_m = \sqrt{MN} = a \frac{\sqrt{1-e^2}}{1-e^2 \sin^2 B_m}$$

子午曲率半径

$$\left. \begin{aligned} M &= \frac{a \cdot (1-e^2)}{\sqrt{(1-e^2 \sin^2 B)^3}} \\ N &= \frac{a}{\sqrt{1-e^2 \sin^2 B}} \end{aligned} \right\}$$

用 R_m 求 a

$$a = \frac{1-e^2 \sin^2 B_m}{\sqrt{1-e^2}} R_m$$

新椭球的平均曲率半径 R'_m

$$R'_m = R_m + H_m = a \frac{\sqrt{1-e^2}}{1-e^2 \sin^2 B_m} + H_m$$

则有：

$$da = \frac{1-e^2 \sin^2 B_m}{\sqrt{1-e^2}} \cdot H_m$$

由于是同基准的椭球膨胀，所以公式中的平移量 $\Delta X_0, \Delta Y_0, \Delta Z_0$ 均为 0，旋转量 $\varepsilon_X, \varepsilon_Y, \varepsilon_Z$ 均为 0，尺度变化参数 $m = 0$ ，扁率变化参数 $\Delta\alpha = 0$ 。公式 (2-77) 可以简化为：

$$\left. \begin{aligned} dL &= 0 \\ dB &= \frac{N}{(M+H) \cdot a} e^2 \sin B \cos B \cdot \Delta a \\ dH &= -\frac{N}{a} (1-e^2 \sin^2 B) \cdot \Delta a \end{aligned} \right\}$$

小结

我们从较为简单的单点高斯投影正反算程序开始，运用 **C#** 知识与 WPF 界面技术实现了一个较为实用的高斯投影正反算与换带程序。

在程序过程中，我们遵循了界面与算法相分离的原则，运用了 WPF 的事件绑定等技术。

这个程序从功能上讲还是有许多不足的，从易用性上也还有许多值得改进的地方。希望随着我们的 **C#** 知识与 WPF 技术的积累，能在以后将它优化得更好！

第 7 章 线路要素计算程序设计

7.1 圆曲线程序设计

7.1.1 圆曲线的数学模型与算法分析

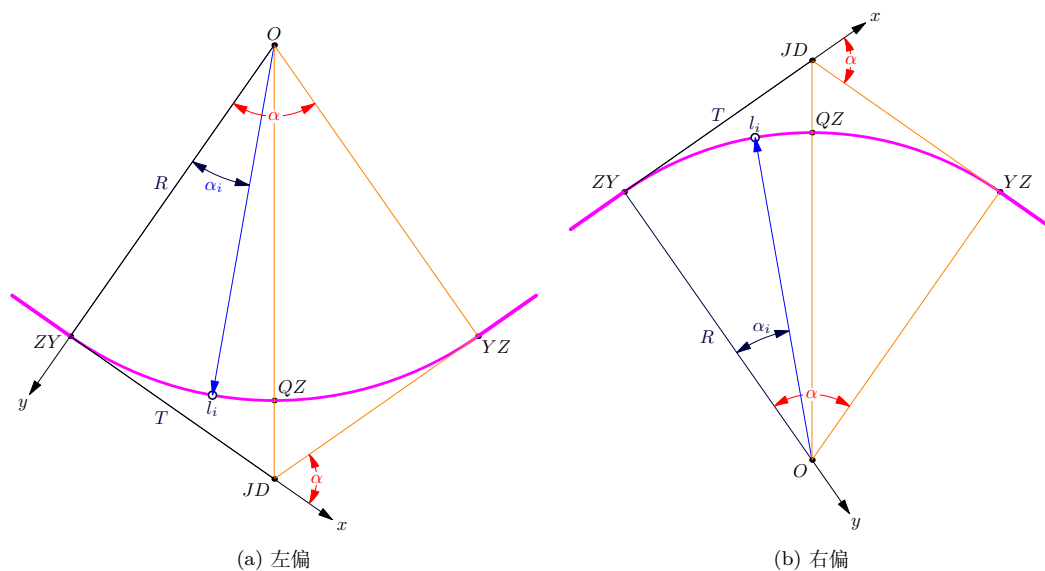


图 7.1 圆曲线要素图

圆曲线的各个要素计算算法如下：

切线长： $T = R \cdot \tan(\alpha/2)$

曲线长： $L = R \cdot \alpha$

外矢距： $E = R \cdot (\sec(\alpha/2) - 1)$

切曲差： $q = 2T - L$

7.1.2 圆曲线上点的坐标计算算法分析

全站仪的坐标放样模式与 GPS RTK 可以十分方便地进行圆曲线上点的坐标放样。

尽管圆曲线的计算方法有很多，也比较简单，但为了与后边的缓和曲线计算方法相一致，我们在此采用圆心角加半径再加坐标系转换法进行圆曲线上点的坐标计算。

如图7.1所示,以 ZY 点为原点,以 ZY 至 JD 切线方向为 x 轴,以 ZY 至 O 点方向为 y 轴建立 ZY 切线测量坐标系。从 (a) 与 (b) 两图可以看出无论圆曲线是左偏还是右偏的,其坐标系是一致的。在 ZY 切线坐标系中用极坐标法按如下公式可以计算出圆曲线上任意一点 l_i 的坐标。

圆曲线偏左:

$$\left. \begin{aligned} x_i &= R \sin \alpha_i \\ y_i &= -R(1 - \cos \alpha_i) \end{aligned} \right\} \quad (7.1)$$

圆曲线偏右:

$$\left. \begin{aligned} x_i &= R \sin \alpha_i \\ y_i &= R(1 - \cos \alpha_i) \end{aligned} \right\} \quad (7.2)$$

式中: $\alpha_i = l_i/R$, $\alpha_i \leq \alpha$, l_i 可用圆曲线上任意一点的里程桩号减去 ZY 点的里程桩号。

如果我们以偏右为正、偏左为负,则可以将以上两公式统一,将偏左的 Y 坐标乘以 -1 即可。

已知 JD 的坐标与里程桩号,根据圆曲线的结合几何要素 R, α 即可计算出圆曲线上特征点的里程桩号:

$$KN_{OZY} = KN_{OJD} - T$$

$$KN_{OQZ} = KN_{OZY} + T/2$$

$$KN_{OYZ} = KN_{OQZ} + T/2$$

在 ZY 切线坐标系中计算出圆曲线上各点的坐标之后,还需将其转换为测量坐标系(或更正式的称为大地坐标系或独立施工坐标系)。在前一章我们已经做过坐标系的转换了,在这个线路转换中,我们将 ZY-JD 边定义为 x 轴,因此两坐标系的夹角即为 ZY-JD 边的坐标方位角,其转换关系如图7.2所示:

转换公式如下:

$$\left. \begin{aligned} x_P &= x_{ZY} + x'_P \cos \alpha - y'_P \sin \alpha \\ y_P &= y_{ZY} + x'_P \sin \alpha + y'_P \cos \alpha \end{aligned} \right\} \quad (7.3)$$

7.1.3 圆曲线坐标计算中的类设计

由以上分析可知,线路计算中至少应该有点类 GPoint 和线路类 CircleRoute。

1. 程序中的点类 GPoint

线路上的 GPoint 类应包含序号、里程桩号、X 坐标、Y 坐标、备注信息等内容。我们设计如下:

```
1 using System;
2 namespace Route.models
3 {
4     /// <summary>
5     /// 点类
```

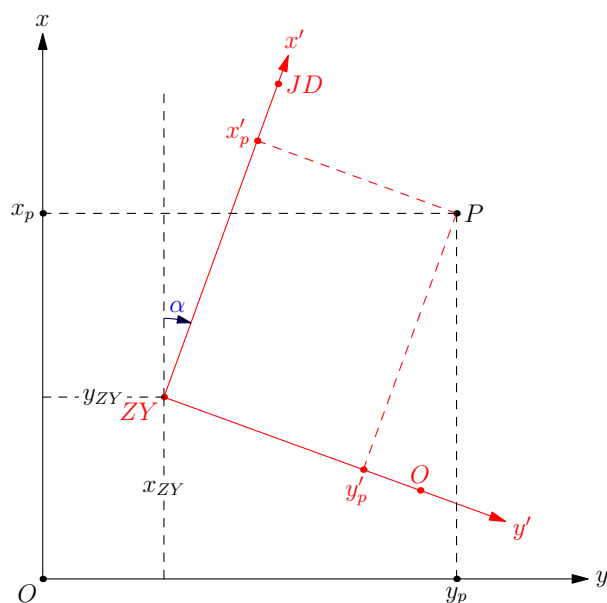


图 7.2 ZY 坐标系转测量坐标系

```

6  /// </summary>
7  public class GPoint : NotificationObject
8  {
9      private int _no;
10     /// <summary>
11     /// 序号
12     /// </summary>
13     public int No
14     {
15         get { return _no; }
16         set
17         {
18             _no = value;
19             RaisePropertyChange("No");
20         }
21     }
22
23     private double _kno;
24     /// <summary>
25     /// 里程桩号
26     /// </summary>
27     public double KNo
28     {
29         get { return _kno; }
30         set
31         {
32             _kno = value;

```

```
33         RaisePropertyChange("KNo");
34     }
35 }
36
37 private double _x;
38 /// <summary>
39 /// X坐标
40 /// </summary>
41 public double X
42 {
43     get { return _x; }
44     set
45     {
46         _x = value;
47         RaisePropertyChange("X");
48     }
49 }
50
51 private double _y;
52 /// <summary>
53 /// Y坐标
54 /// </summary>
55 public double Y
56 {
57     get { return _y; }
58     set
59     {
60         _y = value;
61         RaisePropertyChange("Y");
62     }
63 }
64
65 private string _note;
66 /// <summary>
67 /// 备注
68 /// </summary>
69 public string Note
70 {
71     get { return _note; }
72     set
73     {
74         _note = value;
75         RaisePropertyChange("Note");
76     }
77 }
78
```

```

79     public GPoint()
80     {
81         _no = 0;
82         _kno = _x = _y = 0;
83         _note = "";
84     }
85 }
86 }

```

在上面 GPoint 类中,由于点的里程桩号是 double 类型,而我们常用的里程桩号是“K5+200.00”这样的形式,因此我们在类 GPoint 中设计函数 OutKNoInfo() 将其转换输出,其代码如下所示:

```

1 public class GPoint : NotificationObject
2 {
3     //.....类中其它代码.....
4
5     /// <summary>
6     /// 将double类型的kno分解为K5+200.00形式的里程桩号
7     /// </summary>
8     /// <returns>里程桩号</returns>
9     public string OutKNoInfo()
10    {
11        int k = (int)(_kno / 1000);
12        double klength = _kno - k * 1000;
13        return string.Format("K{0}+{1:0.000}", k, klength);
14    }
15 }

```

以上代码的逻辑非常简单, double 类型的里程除以 1000, 取出公里数, 然后将不足整公里数的部分再取出, 然后组合成形如“K5+200.00”这样的字符串输出。

由于以上代码只涉及类中的 _kno 字段, 因此也可以写成如下的只读属性字段, 调用会更加方便。

```

1 public class GPoint : NotificationObject
2 {
3     //.....类中其它代码.....
4
5     /// <summary>
6     /// 将double类型的kno分解为K5+200.00形式的里程桩号
7     /// </summary>
8     public string KNoInfo
9     {
10        get {
11            int k = (int)(_kno / 1000);
12            double klength = _kno - k * 1000;
13            return string.Format("K{0}+{1:0.000}", k, klength);

```

```

14     }
15 }
16 }

```

2. 程序中的圆曲线类 CircleRoute

圆曲线类中应包含偏转角 α 、曲率半径 R 、切线长 T 等基本属性，还应包括 JD 点、ZY 点、QZ 点、YZ 点等属性，设计代码如下：

```

1 public class CircleRoute : NotificationObject
2 {
3     private double _alpha;
4     /// <summary>
5     /// 偏转角，单位：度分秒
6     /// </summary>
7     public double alpha
8     {
9         get { return _alpha; }
10        set
11        {
12            _alpha = value;
13            RaisePropertyChange("alpha");
14        }
15    }
16
17    protected double _R;
18    /// <summary>
19    /// 圆曲线半径
20    /// </summary>
21    public double R
22    {
23        get { return _R; }
24        set
25        {
26            _R = value;
27            RaisePropertyChange("R");
28        }
29    }
30
31    double _T;
32    /// <summary>
33    /// 切线长
34    /// </summary>
35    public double T
36    {
37        get { return _T; }
38        set

```

```
39     {
40         _T = value;
41         RaisePropertyChanged("T");
42     }
43 }
44
45 double _L;
46 /// <summary>
47 /// 曲线长
48 /// </summary>
49 public double L
50 {
51     get { return _L; }
52     set
53     {
54         _L = value;
55         RaisePropertyChanged("L");
56     }
57 }
58
59 double _E;
60 /// <summary>
61 /// 外矢距
62 /// </summary>
63 public double E
64 {
65     get { return _E; }
66     set
67     {
68         _E = value;
69         RaisePropertyChanged("E");
70     }
71 }
72
73 double _q;
74 /// <summary>
75 /// 切曲差
76 /// </summary>
77 public double q
78 {
79     get { return _q; }
80     set
81     {
82         _q = value;
83         RaisePropertyChanged("q");
84     }
85 }
```

```
85     }
86
87     GPoint _JD = new GPoint();
88     /// <summary>
89     /// 交点
90     /// </summary>
91     public GPoint JD
92     {
93         get { return _JD; }
94         set
95         {
96             _JD = value;
97             RaisePropertyChange("JD");
98         }
99     }
100
101     GPoint _QZ = new GPoint();
102     /// <summary>
103     /// 曲中点
104     /// </summary>
105     public GPoint QZ
106     {
107         get { return _QZ; }
108         set
109         {
110             _QZ = value;
111             RaisePropertyChange("QZ");
112         }
113     }
114
115     GPoint _ZY = new GPoint();
116     /// <summary>
117     /// 直圆点
118     /// </summary>
119     public GPoint ZY
120     {
121         get { return _ZY; }
122         set
123         {
124             _ZY = value;
125             RaisePropertyChange("ZY");
126         }
127     }
128
129     GPoint _YZ = new GPoint();
130     /// <summary>
```

```
131     /// 圆直点
132     /// </summary>
133     public GPoint YZ
134     {
135         get { return _YZ; }
136         set
137         {
138             _YZ = value;
139             RaisePropertyChange("YZ");
140         }
141     }
142
143     private int _isLeftRight = 1;
144     /// <summary>
145     /// 左偏: -1 或 右偏: 1
146     /// </summary>
147     public int IsLeftRight
148     {
149         get { return _isLeftRight; }
150         set
151         {
152             _isLeftRight = value;
153             RaisePropertyChange("IsLeftRight");
154         }
155     }
156
157     /// <summary>
158     /// 线路点集
159     /// </summary>
160     protected ObservableCollection<GPoint> _pointList =
161         new ObservableCollection<GPoint>();
162
163     /// <summary>
164     /// 线路点集
165     /// </summary>
166     public ObservableCollection<GPoint> PointList
167     {
168         get { return _pointList; }
169     }
170 }
```

在圆曲线计算时，由于左偏与右偏的计算是不完全相同的，在此设计一属性 IsLeftRight 用来标记。

为了存储计算后的点坐标，我们设计了一点集属性 PointList。

7.1.4 任意桩号点的坐标计算

给定任意一桩号，计算圆曲线上点的坐标，函数设计如下：

```

1  /// <summary>
2  /// 计算任意里程桩号的点坐标
3  /// </summary>
4  /// <param name="kno">里程桩号</param>
5  public void CalSinglePoint(double kno)
6  {
7      double radAlpha = ZXY.SMath.DMS2RAD(alpha);
8      T = R * Math.Tan(radAlpha * 0.5);
9      L = R * radAlpha;
10     E = R * (1 / Math.Cos(radAlpha * 0.5) - 1);
11
12     ZY.KNo = JD.KNo - T;
13     QZ.KNo = ZY.KNo + L/2;
14     YZ.KNo = QZ.KNo + L/2;
15
16     double li = kno - ZY.KNo;
17     if( li<0 || li>L) return; //不是圆曲线上有效范围
18     double alphai = li / R;
19
20     GPoint pt = new GPoint();
21     pt.KNo = kno;
22
23     pt.X = R * Math.Sin(alphai);
24     pt.Y = IsLeftRight * R * (1 - Math.Cos(alphai));
25
26     double A = ZXY.SMath.Azimuth(ZY.X, ZY.Y, JD.X, JD.Y);
27     pt.TransformXY(ZY.X, ZY.Y, A); //转换到大地坐标
28
29     PointList.Add(pt); //将计算结果存入PointList
30 }

```

在该算法中，第 7 行将度分秒形式的偏转角转换为弧度，再根据前边的算法计算圆曲线的几何要素 T、L、E 等。

一般情况下 JD 的里程桩号与坐标是已知的，直圆点 ZY 的坐标也会已知，但里程桩号未知，所以需要首先计算出 ZY 点的里程桩号。同样的道理也可以计算出曲中点 QZ 与圆直点 YZ 点的里程桩号。

第 16 行根据给定的里程桩号与 ZY 点的里程桩号计算圆曲线的弧长 l_i ，第 17 行做有效性判断： $0 \leq l_i \leq L$ ，第 18 行根据弧长计算该点所在位置的圆心角 α_i 。

第 23、24 行根据公式 7.2 计算圆曲线上各点在 ZY 坐标系中的各点坐标。如果圆曲线偏左，根据公式 7.1 将其 Y 坐标乘以 -1 即 IsLeftRight 的属性值即可。

第 26 行计算出 ZY 点至 JD 点的坐标方位角，第 27 行根据公式 7.3 将其转换到大地坐标系。

第 29 行将其加入到 PointList 列表中，用于界面显示。

7.1.5 圆曲线算例与测试代码

圆曲线的曲率半径为 $R = 120m$ ，偏转角为 $\alpha = 40^{\circ}20'$ ，交点 JD 的里程桩号为“K3+135.12”，坐标为 (6848.320, 5634.240)；直圆点 ZY 的坐标为 (6821.3545599.381)。计算出的圆曲线上各点坐标如表7.1所示。

表 7.1 圆曲线计算示例数据表

序号	里程	X	Y	备注
1	K3+091.05	6821.35	5999.38	ZY
2	K3+100	6826.56	5606.66	
3	K3+120	6836.15	5624.18	
4	K3+133.29	6840.85	5636.60	QZ
5	K3+140	6842.69	5643.06	
6	K3+160	6846.02	5662.76	
7	K3+175.52	6846.31	5678.27	YZ

测试代码为：

```
1 models.CircleRoute route = new models.CircleRoute();
2 route.R = 120;
3 route.alpha = 40.20;
4 route.IsLeftRight = 1;
5
6 route.JD.X = 6848.320;
7 route.JD.Y = 5634.240;
8 route.JD.KNo = 3135.12;
9
10 route.ZY.X = 6821.354;
11 route.ZY.Y = 5599.381;
12
13 route.CalSinglePoint(3175.52);
```

7.1.6 线路上的点按间距批量计算

除了能进行任意里程桩号的计算外，线路计算程序还应该支持按指定间隔进行批量计算的功能。在批量计算中，还应该将线路特征点的坐标计算并显示出来，并按照里程桩号的顺序进行排列。

7.1.7 界面的实现

程序的界面编写我们仍采用 WPF 技术，布局如图... 所示：

7.2 有缓和曲线的圆曲线

有缓和曲线的圆曲线的数学模型与算法相对于圆曲线来说就复杂多了。下面我们将从算法到程序实现逐一分析。

7.2.1 缓和曲线的数学模型

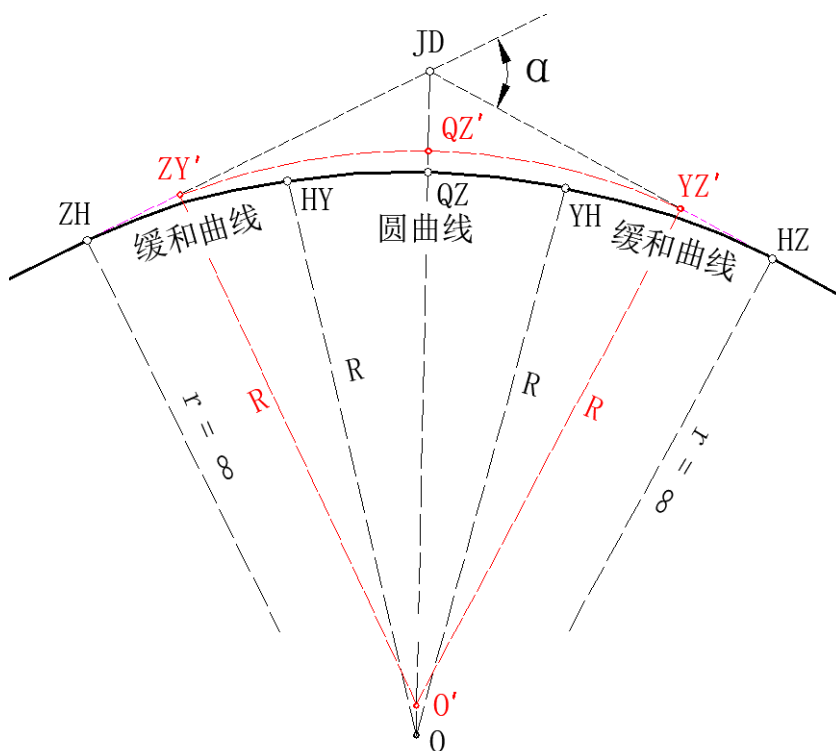


图 7.3 缓和曲线的定义

缓和曲线参数计算公式为：

$$\beta_0 = \frac{l_0}{2R}$$

切垂距：

$$m = \frac{l_0}{2} - \frac{l_0^3}{240R^2}$$

内移距：

$$p = \frac{l_0^2}{24R}$$

切线长：

$$T = m + (R + p) \cdot \tan \frac{\alpha}{2}$$

曲线全长：

$$L = R \cdot (\alpha - 2\beta_0) + 2l_0$$

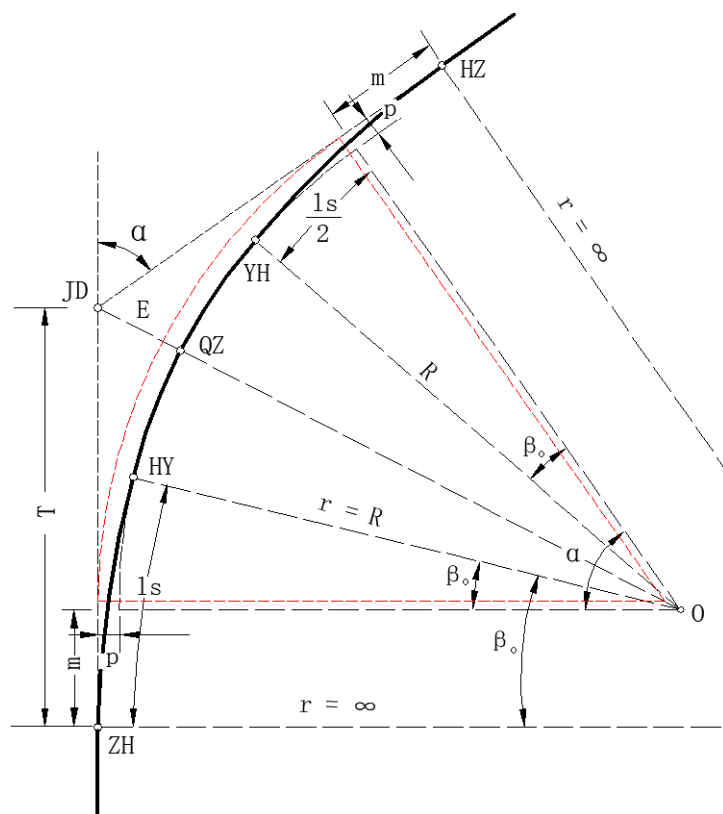


图 7.4 缓和曲线的内移距和切线增长

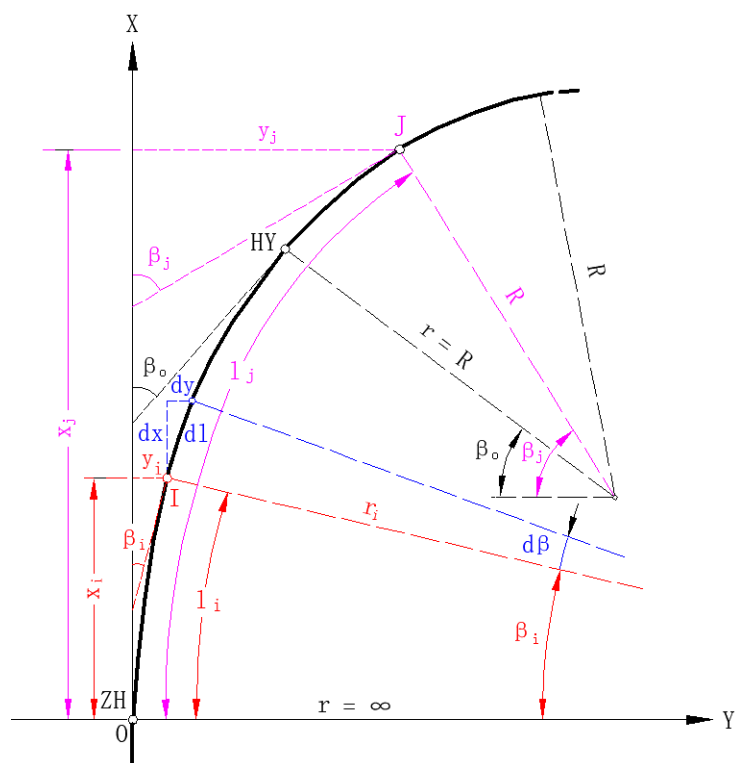


图 7.5 缓和曲线的坐标计算

圆曲线长:

$$L_C = R \cdot (\alpha - 2\beta_0)$$

外矢距:

$$E = (R + p) \cdot \left(\sec \frac{\alpha}{2} - 1 \right)$$

1. 缓和曲线在 ZH 切线直角坐标系中的坐标计算 (ZH 段)

以 ZH 为坐标系原点, ZH 至 JD 方向为 x 轴, 过 ZH 点垂直于 ZH-JD 方向为 y 轴建立坐标, 如图7.4所示。则 ZH 部分缓和曲线上各点的坐标为:

$$\left. \begin{aligned} x_i &= l_i - \frac{l_i^5}{40R^2l_0^2} + \frac{l_i^9}{3456R^4l_0^4} - \frac{l_i^{13}}{599040R^6l_0^6} + \dots \\ y_i &= \frac{l_i^3}{6Rl_0} - \frac{l_i^7}{336R^3l_0^3} + \frac{l_i^{11}}{42240R^5l_0^5} - \frac{l_i^{15}}{9676800R^7l_0^7} + \dots \end{aligned} \right\} \quad (7.4)$$

以上公式在计算中取前两项或前三项即可。

2. 缓圆点 (HY) 坐标计算公式为:

将 $l_i = l_0$ 代入公式7.4中计算可得到下式:

$$\left. \begin{aligned} x_{HY} &= l_0 - \frac{l_0^3}{40R^2} + \frac{l_0^5}{3456R^4} - \frac{l_0^7}{599040R^6} + \dots \\ y_{HY} &= \frac{l_0^2}{6R} - \frac{l_0^4}{336R^3} + \frac{l_0^6}{42240R^5} - \frac{l_0^8}{9676800R^7} + \dots \end{aligned} \right\} \quad (7.5)$$

3. 圆曲线上点的坐标计算公式为:

$$\left. \begin{aligned} x_j &= R \sin \beta_j + m \\ y_j &= R(1 - \cos \beta_j) + p \end{aligned} \right\} \quad (7.6)$$

式中: $\beta_i = \beta_0 + (l_i - l_0)/R$

4. 缓和曲线在 HZ 切线直角坐标系中的坐标计算 (HZ 段)

如图7.6与如图7.7所示建立以缓直点 HZ 为原点, 过 HZ 至 JD 方向为 x 轴, 过 HZ 点的缓和曲线切线为 y 轴的直角坐标系, 计算另一半曲线任意一点的坐标 (x'_i, y'_i) 。然后, 将坐标转换为以 ZH 点为原点的直角坐标系中。

缓和曲线在 HZ 坐标系中的坐标可以继续使用公式 7.4 计算, 然后由图7.6 可知将其转换为 HZ 坐标系中的坐标计算公式为:

$$\left. \begin{aligned} x'_i &= x_i \\ y'_i &= -y_i \end{aligned} \right\} \quad (7.7)$$

请注意在使用公式 7.4 时应将 l_i 替换为 $L - l_i$ 。

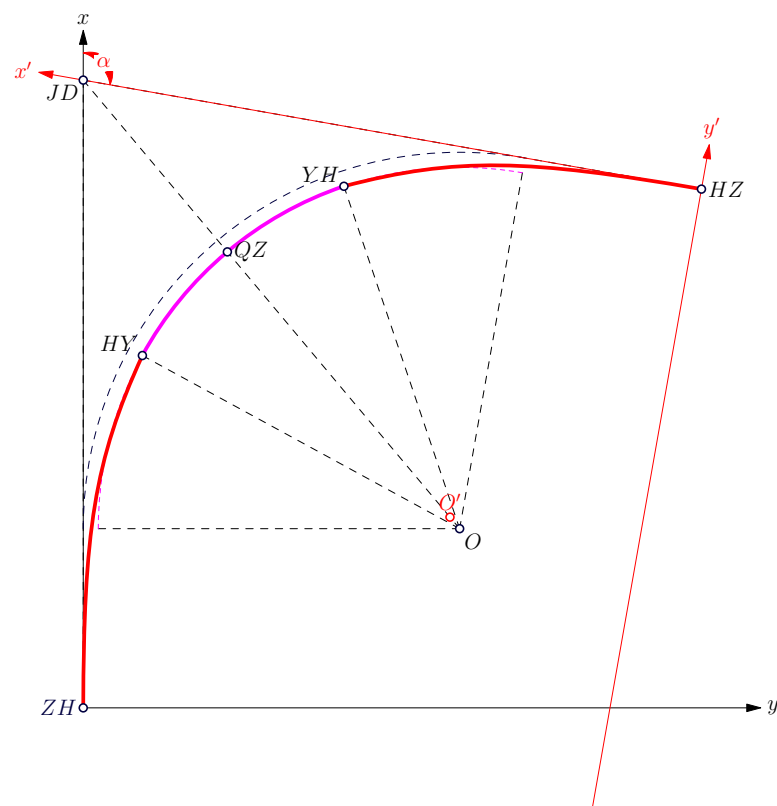


图 7.6 缓和曲线的 HZ 部分坐标计算

为了将 HZ 坐标系中的点坐标转换到 ZH 坐标系中, 我们引用公式7.3来计算, 请注意公式7.3中的 α 为 x' 轴的方位角 (即 x 轴到 x' 轴的水平夹角), 因此图7.6应用到公式7.3中的 α 应为 $\alpha + 180^\circ$ 。如果将其代入公式7.3, 则有转换公式为:

$$\left. \begin{aligned} x_i &= x_{HZ} - x'_i \cos \alpha + y'_i \sin \alpha \\ y_i &= y_{HZ} - x'_i \sin \alpha - y'_i \cos \alpha \end{aligned} \right\} \quad (7.8)$$

公式7.8中的 (x_{HZ}, y_{HZ}) 为 HZ 点在 ZH 坐标系中的坐标, 其值为:

$$\left. \begin{aligned} x_{HZ} &= T(1 + \cos \alpha) \\ y_{HZ} &= T \sin \alpha \end{aligned} \right\} \quad (7.9)$$

也可以不用公式7.8, 直接将 $\alpha + 180^\circ$ 代入到公式7.3中进行计算。

在以上计算中, 我们以曲线右偏为例的, 如果曲线左偏, 同样的方法建立坐标系, x 坐标是相同的, y 坐标乘以 -1 即可。

5. 曲线上点坐标转换为大地坐标的计算公式为:

由于已经将曲线上的点坐标统一到 ZH 坐标系中了, 我们继续引用公式7.3 将 ZH 切线坐标系坐标转换到大地坐标系 (测量坐标系) 中, 公式中的 α 为 $ZH \rightarrow JD$ 的坐标方位角。

7.2.2 有缓和曲线的圆曲线算例

缓和曲线的曲率半径 $R = 1000$, 偏转角 $\alpha = 10.1820$, 右偏, 缓和曲线长 $l_0 = 80$, 直缓点 ZH 的坐标为 (3088256.238, 66798.566), 交点 JD 的坐标为 (3088386.436, 66798.566), 里程桩号为 “K5+5330.198”。计算出的曲线上各点坐标如表7.2所示。

7.3 较为完整的线路里程桩计算

以上曲线要素的计算是按曲线类型单独进行分析编写的, 实际上一条线路是由无数段直线、圆曲线与缓和曲线组成。

表 7.2 有缓和曲线的圆曲线计算示例数据表

序号	里程	X	Y	备注
1	K5+200.000	3088256.238	66798.566	ZH
2	K5+220.000	3088276.238	66798.583	
3	K5+240.000	3088296.238	66798.699	
4	K5+260.000	3088316.235	66799.016	
5	K5+280.000	3088336.225	66799.632	HY
6	K5+300.000	3088356.200	66800.632	
7	K5+320.000	3088376.150	66802.031	
8	K5+329.933	3088386.048	66802.874	QZ
9	K5+339.866	3088395.936	66803.815	
10	K5+359.866	3088415.815	66806.008	
11	K5+379.866	3088435.646	66808.598	YH
12	K5+399.866	3088455.424	66811.568	
13	K5+419.866	3088475.156	66814.834	
14	K5+439.866	3088494.853	66818.297	
15	K5+459.866	3088514.534	66821.858	HZ